Trabajo Práctico III

Optimización de consultas: planes de ejecución, índices B-Tree y hash, y reescritura de consultas

MATERIA	Gestión Avanzada de Datos
CARRERA	Ingeniería en Sistemas de Información
FACULTAD	UTN - Facultad Regional Concepción del Uruguay
ALUMNOS	Juan Brun, Leandro Guiffrey, Manuel Ángel Riedel
DOCENTES	Mg. Ing. Andrés J. Pascal, Mg. Ing. Adrián Nicolás Planas
MOTOR	PostgreSQL 17.10 (PostGIS 3.6 para cargar la base de comercios)
BASES DE DATOS	cdelu (parte a) y optimizacion (partes b, c y d)
FECHA	15 de octubre de 2026

Organización de la entrega

Cada script resuelve uno o varios ítems del enunciado. La parte a corre sobre la base `cdelu`, cargada con el dump de la cátedra; las partes b, c y d generan sus propias tablas en la base `optimizacion`. Los scripts son reejecutables (borran lo que van a crear antes de crearlo), pero dentro de cada parte hay que correrlos en orden, porque los índices se van acumulando como lo plantea el enunciado: el a.8, por ejemplo, se resuelve con los índices hash que dejaron el a.2 y el a.7. Todas las salidas de este documento son las que devolvió PostgreSQL 17 al correr los scripts de punta a punta, en este orden, sobre bases recién creadas.

`00-funciones-medicion.sql`	Funciones para medir planes (en las dos bases)
`a00-preparacion.sql`	Base cdelu: estado inicial y relación comercios-barrios
`a01-calle-mitre.sql`	Ítems a.1 y a.2
`a02-producto-principal.sql`	Ítems a.3, a.4 y a.5
`a03-barrio.sql`	Ítems a.6 y a.7
`a04-centro-galarza-kiosco.sql`	Ítem a.8
`a05-pares-cercanos.sql`	Ítem a.9
`a06-calle-y-rubro.sql`	Ítems a.10 y a.11
`a07-consultas-propias.sql`	Ítem a.12 (tres consultas)
`b00-preparacion.sql`	Tablas articulos y precios del enunciado
`b01-rubro-5.sql`	Ítem b.1
`b02-precio-actual.sql`	Ítems b.2 y b.3
`b03-precios-rango.sql`	Ítem b.4
`b04-promedios.sql`	Ítems b.5 y b.6
`b05-maximo-rubro-15.sql`	Ítem b.7
`b06-maximos-promedios.sql`	Ítems b.8 y b.9
`c00-personas.sql`	Tabla personas (1.000.000 de filas)
`c01-ciudad.sql`	Ítems c.1, c.2 y c.3
`c02-edad.sql`	Ítems c.4 y c.5
`c03-agregados.sql`	Ítem c.6
`d00-clientes-pedidos.sql`	Ítem d.1
`d01-top-clientes.sql`	Ítems d.2 y d.3
`d02-indices-pedidos.sql`	Ítems d.4 y d.5

Para reproducir todo desde cero:

```
createdb cdelu
```

```
psql -d cdelu -c "CREATE EXTENSION postgis"
psql -d cdelu -f "CdelU - Comercios.sql"
psql -d cdelu -f 00-funciones-medicion.sql
psql -d cdelu -f a00-preparacion.sql          # y así hasta a07

createdb optimizacion
psql -d optimizacion -f 00-funciones-medicion.sql
psql -d optimizacion -f b00-preparacion.sql   # y así hasta d02, en orden
```

Las tablas de las partes b, c y d se generan con `random()`, pero cada script fija antes la semilla con `setseed(0.2026)`: los datos salen siempre iguales y los resultados de este documento se pueden reproducir fila por fila. Las bases completas ocupan unos 2 GB y se generan en menos de un minuto.

Cómo se midió

El enunciado pide resolver cada consulta con `EXPLAIN ANALYZE` y describir el plan. Cada script muestra el plan con `EXPLAIN (ANALYZE, BUFFERS)`: además de ejecutar la consulta y dar los tiempos reales de cada nodo, la opción `BUFFERS` dice cuántas páginas de 8 kB se encontraron en la caché de PostgreSQL (`shared hit`) y cuántas hubo que pedir al sistema operativo (`read`). Los planes se leen de adentro hacia afuera: los nodos más indentados son los accesos a las tablas y los de arriba combinan sus resultados. En cada nodo, `cost=inicio..total` es el costo estimado (en unidades abstractas, no en milisegundos), `rows` la cantidad de filas estimada y, entre paréntesis, la real; cuando estas dos difieren mucho el planificador decidió con información equivocada.

Para comparar variantes de una misma consulta un único `EXPLAIN ANALYZE` no alcanza. La primera ejecución puede leer de disco y las siguientes de memoria, y en tablas chicas como comercios la diferencia entre dos planes es del mismo orden que el ruido de una corrida a otra. Por eso el script `00-funciones-medicion.sql` define dos funciones y un procedimiento:

- `fn_medir(consulta, n)` ejecuta la consulta una vez para calentar la caché (esa corrida se descarta) y después `n` veces con `EXPLAIN (ANALYZE, TIMING OFF, FORMAT JSON)`, y devuelve el mínimo, la **mediana** y el máximo del tiempo de ejecución. `TIMING OFF` evita cronometrar cada nodo, que es lo que más infla el tiempo total cuando se procesan millones de filas.
- `fn_resumen_plan` convierte el plan en JSON en una línea, por ejemplo `Hash Join [Seq Scan, Hash [Seq Scan]]`: cada nodo con sus hijos entre corchetes y el índice usado entre paréntesis. Así se ve qué plan usó cada variante sin repetir el EXPLAIN completo.
- `pr_medir(item, variante, consulta, n)` guarda el resultado en la tabla `medicion`. Al final de cada script se listan las mediciones de sus ítems; en las salidas esa lista aparece como tabla.

Se repitió cada consulta 50 veces en la parte a (tardan microsegundos) y entre 5 y 20 en las demás. Los tiempos son, entonces, con la caché caliente, que es la situación de una consulta que se ejecuta seguido. Los números que se citan en el texto son siempre medianas.

El servidor es un PostgreSQL 17.10 de Homebrew en una Mac con Apple M5 (10 núcleos, 16 GB de memoria y disco SSD), con la configuración por defecto: `shared_buffers` 128 MB, `work_mem` 4 MB, `random_page_cost` 4, `max_parallel_workers_per_gather` 2, y sin JIT (esta compilación no lo incluye, así que los planes no tienen la sección de compilación JIT). Las tablas de las partes b a d son más grandes que `shared_buffers`, de ahí que en sus planes aparezcan muchas páginas `read`: el sistema operativo las sirve desde su propia caché, no desde el disco.

En varios ítems se usa `SET enable_seqscan = off` (o `enable_hashjoin`, `enable_indexscan`, etc.) para ver qué hubiera pasado con un plan que el planificador no eligió. Esos parámetros no prohíben el método: le suman un costo enorme, así que el planificador lo evita mientras tenga alternativa. Se usan sólo dentro de la sesión, para comparar, y no como solución.

Sobre el borrador del grupo

El trabajo parte del documento que habíamos empezado en grupo (`Trabajo practico 3 - Optimización de Consultas.pdf`), que tenía resueltos del a.1 al a.7. Se mantuvo su enfoque y se volvieron a correr esas consultas, con estas observaciones:

- Las salidas del borrador son de PostgreSQL 18: las líneas `Index Searches` y las filas con decimales (`rows=46.00`) son novedades de esa versión. Acá se usó PostgreSQL 17, y los planes y costos estimados coinciden exactamente con los del borrador (por ejemplo `Seq Scan ... cost=0.00..83.31 rows=46` en el a.1).
- En el a.4 el borrador concluía que el índice B-Tree empeoraba la consulta (2,961 ms contra 1,573 ms). Esa comparación usa una sola corrida, y en la del índice el plan muestra `Buffers: shared hit=1 read=4`: las páginas del índice recién creado todavía no estaban en caché. Con la mediana de 50 ejecuciones el B-Tree resulta dos veces más rápido (0,083 ms contra 0,162 ms).
- En el a.5 el borrador muestra el plan de la consulta por calle Mitre (el del a.2), no el del agrupamiento por producto principal. Rehecho, el resultado es otro: el índice hash no sirve para `GROUP BY` y el plan vuelve a ser el del a.3.
- Del a.8 en adelante, y las partes b, c y d, estaban pendientes.

Observaciones sobre los datos de comercios

Antes de resolver la parte a hubo que entender cómo se relacionan las tablas del dump, que no tiene claves foráneas:

- `comercios.barrio` no referencia a `barrios.gid` sino a `barrios.type`, una columna de texto con el código de barrio. Se comprobó con PostGIS en `a00-preparacion.sql`: para los comercios que tienen ubicación en `poiscomercios`, el barrio cuyo polígono contiene el punto es siempre el que se obtiene uniendo por `type`. Además, los códigos más usados (0 y del 31 al 52, el 70 % de los comercios) ni siquiera existen como `gid`. Por eso el "barrio 1" del a.6 es *Cantera 25 / San José* (3 comercios) y el barrio Centro del a.8 tiene código 0 (1.063 comercios, la mitad de la tabla).
- El rubro de cada comercio es `clanae5`, código del nomenclador `clanae`. El de kioscos es 47119, *Kioscos y Polirrubros*. Tres comercios tienen códigos que no están en `clanae`.
- 47 comercios tienen la calle con espacios al final (`'Galarza '`, `'8 de junio '`). Una igualdad exacta no los encuentra; se verificó que en los ítems a.8 y a.10 ninguno de ellos cumple las demás condiciones, así que los resultados no cambian. Hay además 511 comercios sin número y 712 sin superficie.

0. Funciones de medición

Se crean en las dos bases antes que cualquier otro script. No producen salida: definen `fn_resumen_plan`, `fn_medir`, la tabla `medicion` y el procedimiento `pr_medir`, descriptos en "Cómo se midió".

SCRIPT 00-funciones-medicion.sql

```
SET search_path TO public;

-- 0. Funciones auxiliares para comparar planes. Se crean en las dos bases del
--    práctico (cdelu y optimizacion). Un solo EXPLAIN ANALYZE no alcanza para
--    comparar tiempos: la primera corrida puede leer de disco y las siguientes
--    de caché, y en tablas chicas la diferencia entre dos planes es menor que el
--    ruido de una corrida a otra. fn_medir ejecuta la consulta varias veces y
--    devuelve el resumen del plan y la mediana del tiempo de ejecución.

-- 0.a) Resumen de una línea del árbol del plan, a partir del EXPLAIN en JSON:
--      cada nodo con sus hijos entre corchetes, el índice usado entre paréntesis
--      y las subconsultas marcadas como InitPlan (se evalúan una vez) o SubPlan
--      (se evalúan por cada fila del nodo padre).

CREATE OR REPLACE FUNCTION fn_resumen_plan(p_nodo jsonb)
RETURNS text
LANGUAGE plpgsql IMMUTABLE AS $$
DECLARE
    v_texto TEXT;
    v_hijos TEXT[] := '{}';
    v_hijo  JSONB;
BEGIN
    v_texto := p_nodo->>'Node Type';

    IF v_texto = 'Aggregate' THEN
        v_texto := CASE p_nodo->>'Strategy'
                       WHEN 'Hashed' THEN 'HashAggregate'
                       WHEN 'Sorted' THEN 'GroupAggregate'
                       WHEN 'Mixed'  THEN 'MixedAggregate'
                       ELSE 'Aggregate'
                   END;
    END IF;

    IF COALESCE(p_nodo->>'Join Type', 'Inner') <> 'Inner' THEN
        IF v_texto = 'Nested Loop' THEN
            v_texto := 'Nested Loop ' || (p_nodo->>'Join Type') || ' Join';
        ELSE
            v_texto := replace(v_texto, ' Join', ' ' || (p_nodo->>'Join Type') || ' Join');
        END IF;
    END IF;

    IF p_nodo->>'Partial Mode' = 'Partial' THEN
        v_texto := 'Partial ' || v_texto;
    ELSIF p_nodo->>'Partial Mode' = 'Finalize' THEN
        v_texto := 'Finalize ' || v_texto;
    END IF;

    IF p_nodo->>'Scan Direction' = 'Backward' THEN
        v_texto := v_texto || ' Backward';
    END IF;

    IF (p_nodo->>'Parallel Aware')::boolean THEN
        v_texto := 'Parallel ' || v_texto;
    END IF;

    IF p_nodo ? 'Index Name' THEN
        v_texto := v_texto || ' (' || (p_nodo->>'Index Name') || ')';
    END IF;

    FOR v_hijo IN SELECT jsonb_array_elements(p_nodo->'Plans') LOOP
        IF v_hijo->>'Parent Relationship' IN ('InitPlan', 'SubPlan') THEN
            v_hijos := v_hijos || ((v_hijo->>'Parent Relationship') || ' ' || fn_resumen_plan(v_hijo));
        ELSE
            v_hijos := v_hijos || fn_resumen_plan(v_hijo);
        END IF;
    END LOOP;

    IF cardinality(v_hijos) > 0 THEN
        v_texto := v_texto || ' [' || array_to_string(v_hijos, ', ') || ']';
    END IF;
```

```
    RETURN v_texto;
END;
$$;

-- 0.b) Ejecuta la consulta una vez para calentar la caché (esa corrida se descarta)
--      y después p_repeticiones veces con EXPLAIN (ANALYZE, TIMING OFF). Devuelve el
--      plan elegido y el mínimo, la mediana y el máximo del tiempo de ejecución en
--      milisegundos. TIMING OFF evita medir cada nodo, que es lo que más distorsiona
--      el tiempo total en consultas que procesan muchas filas.

CREATE OR REPLACE FUNCTION fn_medir(p_consulta TEXT, p_repeticiones INTEGER DEFAULT 20)
RETURNS TABLE (plan TEXT, min_ms NUMERIC, mediana_ms NUMERIC, max_ms NUMERIC)
LANGUAGE plpgsql AS $$
DECLARE
    v_explain JSONB;
    v_tiempos NUMERIC[] := '{}';
BEGIN
    EXECUTE 'EXPLAIN (ANALYZE, TIMING OFF, FORMAT JSON) ' || p_consulta;

    FOR i IN 1..p_repeticiones LOOP
        EXECUTE 'EXPLAIN (ANALYZE, TIMING OFF, FORMAT JSON) ' || p_consulta
           INTO v_explain;
        v_tiempos := v_tiempos || (v_explain->0->>'Execution Time')::NUMERIC;
    END LOOP;

    plan := fn_resumen_plan(v_explain->0->'Plan');

    SELECT ROUND(MIN(t), 3),
           ROUND((PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY t))::NUMERIC, 3),
           ROUND(MAX(t), 3)
      INTO min_ms, mediana_ms, max_ms
    FROM unnest(v_tiempos) AS t;

    RETURN NEXT;
END;
$$;

-- 0.c) Registro de las mediciones, para poder mostrar al final de cada script
--      la comparación entre las variantes de una misma consulta.

CREATE TABLE IF NOT EXISTS medicion (
    id          SERIAL PRIMARY KEY,
    item        TEXT    NOT NULL,
    variante    TEXT    NOT NULL,
    plan        TEXT,
    min_ms      NUMERIC,
    mediana_ms  NUMERIC,
    max_ms      NUMERIC
);

CREATE OR REPLACE PROCEDURE pr_medir(p_item TEXT, p_variante TEXT,
                                     p_consulta TEXT, p_repeticiones INTEGER DEFAULT 20)
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO medicion (item, variante, plan, min_ms, mediana_ms, max_ms)
    SELECT p_item, p_variante, m.plan, m.min_ms, m.mediana_ms, m.max_ms
    FROM fn_medir(p_consulta, p_repeticiones) m;
END;
$$;
```

Parte a. Base de comercios de Concepción del Uruguay

La tabla `comercios` tiene 2.025 filas en 58 páginas (464 kB). Es tan chica que entra entera en la caché después de la primera lectura, y todos sus planes se miden en microsegundos. Las diferencias relativas entre planes son reales y se repiten en cada corrida, pero en valores absolutos son de décimas de milisegundo: lo interesante en esta parte es por qué el planificador elige cada plan, más que cuánto se gana.

a.0 Preparación de la base cdelu

El script deja la tabla comercios como vino en el dump: borra los índices que crean los scripts siguientes, vacía la tabla de mediciones y actualiza las estadísticas con `VACUUM ANALYZE`. Las consultas de control muestran el tamaño de las tablas, que los únicos índices de partida son las claves primarias (y el índice GiST geográfico de barrios), y la comprobación de que `comercios.barrio` se corresponde con `barrios.type`: la columna `barrio_por_gid` queda vacía en todas las filas porque esos códigos no existen como `gid`, mientras que `barrio_por_type` coincide siempre con el barrio que contiene geográficamente al comercio.

SCRIPT a00-preparacion.sql

```
SET search_path TO public;

-- a.0 Preparación de la base cdelu. La base se crea y se carga desde la terminal
--     con el dump de la cátedra, que necesita PostGIS porque las tablas de
--     barrios, calles y puntos de interés tienen columnas de tipo geometry:
--
--       createdb cdelu
--       psql -d cdelu -c "CREATE EXTENSION postgis"
--       psql -d cdelu -f "CdelU - Comercios.sql"
--       psql -d cdelu -f 00-funciones-medicion.sql
--
--     Si el servidor no tiene el rol postgres, los ALTER ... OWNER TO postgres del
--     dump fallan sin consecuencias: las tablas quedan a nombre de quien las carga.
--
--     Este script deja la tabla comercios como vino en el dump: borra los índices
--     que crean los scripts siguientes y actualiza las estadísticas.

DROP INDEX IF EXISTS idx_comercios_calle_hash;
DROP INDEX IF EXISTS idx_comercios_pp_btree;
DROP INDEX IF EXISTS idx_comercios_pp_hash;
DROP INDEX IF EXISTS idx_comercios_barrio_hash;
DROP INDEX IF EXISTS idx_comercios_rubro_calle_numero;
DROP INDEX IF EXISTS idx_comercios_calle_clanae;
DROP INDEX IF EXISTS idx_comercios_clanae_superficie;
DROP INDEX IF EXISTS idx_comercios_clanae;

TRUNCATE TABLE medicion;

VACUUM ANALYZE comercios;
VACUUM ANALYZE clanae;
VACUUM ANALYZE barrios;

-- Tamaño de las tablas que usa la parte a: comercios entra entera en 58 páginas
-- de 8 kB, así que después de la primera lectura queda toda en shared_buffers.

SELECT c.relname                              AS tabla,
       c.reltuples::BIGINT                    AS filas,
       c.relpages                             AS paginas,
       pg_size_pretty(pg_relation_size(c.oid)) AS tamanio
FROM pg_class c
WHERE c.relname IN ('comercios', 'clanae', 'barrios')
ORDER BY c.relname;

-- Índices de partida: sólo las claves primarias (y el GiST geográfico de barrios).

SELECT tablename, indexname, indexdef
FROM pg_indexes
WHERE tablename IN ('comercios', 'clanae', 'barrios')
ORDER BY tablename, indexname;

-- comercios.barrio no referencia a barrios.gid sino a barrios.type (guardado como
-- texto). Se comprueba ubicando geográficamente los comercios que tienen punto en
-- poiscomercios: el barrio que contiene al punto coincide con el que se obtiene
-- uniendo por type, nunca con el que se obtendría uniendo por gid.

SELECT pc.barrio,
       bt.label                                  AS barrio_por_type,
       bg.label                                  AS barrio_por_gid,
       MODE() WITHIN GROUP (ORDER BY geo.label)  AS barrio_por_ubicacion,
       COUNT(geo.gid)                            AS comercios_ubicados
FROM poiscomercios pc
JOIN barrios geo ON ST_Contains(geo.the_geom, pc.geom)
LEFT JOIN barrios bt ON bt.type::INTEGER = pc.barrio
LEFT JOIN barrios bg ON bg.gid = pc.barrio
```

```
GROUP BY pc.barrio, bt.label, bg.label
ORDER BY pc.barrio NULLS LAST;
```

SALIDA

```
   tabla   | filas | paginas |  tamanio
-----------+-------+---------+------------
 barrios   |    30 |       1 | 8192 bytes
 clanae    |   146 |       2 | 16 kB
 comercios |  2025 |      58 | 464 kB
(3 rows)

 tablename |         indexname         |                                    indexdef
-----------+---------------------------+-------------------------------------------------------------------------------
 barrios   | barriosok_A_pkey          | CREATE UNIQUE INDEX "barriosok_A_pkey" ON public.barrios USING btree (gid)
 barrios   | barriosok_A_the_geom_gist | CREATE INDEX "barriosok_A_the_geom_gist" ON public.barrios USING gist
                                         (the_geom)
 clanae    | pkid1                     | CREATE UNIQUE INDEX pkid1 ON public.clanae USING btree (id)
 comercios | pkidcomer                 | CREATE UNIQUE INDEX pkidcomer ON public.comercios USING btree (id)
(4 rows)

 barrio |  barrio_por_type   | barrio_por_gid | barrio_por_ubicacion | comercios_ubicados
--------+--------------------+----------------+----------------------+--------------------
      0 | Centro             |                | Centro               |                426
     31 | San vicente        |                | San vicente          |                 64
     32 | Sarmiento          |                | Sarmiento            |                 62
     33 | La quilmes         |                | La quilmes           |                  9
     34 | La concepcion      |                | La concepcion        |                  7
     39 | Santa teresita sur |                | Santa teresita sur   |                 17
     42 | El mirador         |                | El mirador           |                  1
     47 | 12 de octubre      |                | 12 de octubre        |                  1
     48 | La liga            |                | La liga              |                  1
     52 | Los tanques        |                | Los tanques          |                  2
        |                    |                | Centro               |                  5
(11 rows)
```

a.1 y a.2 Comercios de la calle Mitre

a.1, sin índice. El plan es un único nodo `Seq Scan`: PostgreSQL recorre las 58 páginas de la tabla (`shared hit=58`), evalúa el filtro `calle = 'Mitre'` en cada una de las 2.025 filas, descarta 1.979 (`Rows Removed by Filter`) y devuelve 46. La estimación de 46 filas es exacta, porque Mitre está entre los valores más frecuentes que guarda `ANALYZE`. No hay otra opción: no existe ningún índice sobre calle.

a.2, con índice hash. El plan pasa a tener dos nodos. Abajo, el `Bitmap Index Scan` calcula el hash de 'Mitre', va directo al bucket correspondiente del índice (2 páginas) y arma un mapa de bits con las páginas de la tabla donde hay filas de Mitre. Arriba, el `Bitmap Heap Scan` lee sólo esas 33 páginas, en orden físico, y vuelve a verificar la condición (`Recheck Cond`), porque un hash puede tener colisiones y el índice no guarda el valor original. En total lee 35 páginas en lugar de 58, y evalúa 46 filas en lugar de 2.025. El costo estimado baja de 83,31 a 62,26 y la mediana de 0,070 ms a 0,008 ms: unas nueve veces más rápido.

Que use un Bitmap Heap Scan y no un Index Scan común se explica porque las 46 filas están repartidas en 33 páginas distintas: leerlas por orden de página es más barato que saltar de una a otra en el orden en que las devuelve el índice.

Como agregado, la última consulta busca con `LIKE 'Mit%'` y el plan vuelve al Seq Scan: un índice hash sólo resuelve igualdades. Para búsquedas por prefijo o por rango hace falta un B-Tree.

SCRIPT a01-calle-mitre.sql

```
SET search_path TO public;

DELETE FROM medicion WHERE item IN ('a.1', 'a.2');
DROP INDEX IF EXISTS idx_comercios_calle_hash;

-- a.1) Comercios que se encuentran en la calle Mitre, sin índices sobre calle.

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM comercios c
WHERE c.calle = 'Mitre';

CALL pr_medir('a.1', 'Sin índice', $$
    SELECT * FROM comercios c WHERE c.calle = 'Mitre'
$$, 50);

-- a.2) Índice hash sobre calle y la misma consulta.

CREATE INDEX idx_comercios_calle_hash ON comercios USING hash (calle);
ANALYZE comercios;

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM comercios c
WHERE c.calle = 'Mitre';

CALL pr_medir('a.2', 'Índice hash sobre calle', $$
    SELECT * FROM comercios c WHERE c.calle = 'Mitre'
$$, 50);

-- a.2) El índice hash sólo sirve para igualdad: con LIKE por prefijo el
--      planificador no lo puede usar y vuelve al Seq Scan.

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM comercios c
WHERE c.calle LIKE 'Mit%';

SELECT item, variante, mediana_ms, min_ms, max_ms, plan
FROM medicion
WHERE item IN ('a.1', 'a.2')
ORDER BY id;
```

SALIDA

```
                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------
 Seq Scan on comercios c  (cost=0.00..83.31 rows=46 width=281) (actual time=0.007..0.125 rows=46 loops=1)
```

```
   Filter: ((calle)::text = 'Mitre'::text)
   Rows Removed by Filter: 1979
   Buffers: shared hit=58
 Planning:
   Buffers: shared hit=203
 Planning Time: 0.134 ms
 Execution Time: 0.131 ms
(8 rows)

                                                                 QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on comercios c  (cost=4.36..62.26 rows=46 width=281) (actual time=0.007..0.014 rows=46 loops=1)
   Recheck Cond: ((calle)::text = 'Mitre'::text)
   Heap Blocks: exact=33
   Buffers: shared hit=35
   ->  Bitmap Index Scan on idx_comercios_calle_hash  (cost=0.00..4.34 rows=46 width=0)
         (actual time=0.004..0.004 rows=46 loops=1)
         Index Cond: ((calle)::text = 'Mitre'::text)
         Buffers: shared hit=2
 Planning:
   Buffers: shared hit=242
 Planning Time: 0.084 ms
 Execution Time: 0.020 ms
(11 rows)

                                                QUERY PLAN
-----------------------------------------------------------------------------------------------------------
 Seq Scan on comercios c  (cost=0.00..83.31 rows=46 width=281) (actual time=0.002..0.072 rows=46 loops=1)
   Filter: ((calle)::text ~~ 'Mit%'::text)
   Rows Removed by Filter: 1979
   Buffers: shared hit=58
 Planning:
   Buffers: shared hit=15
 Planning Time: 0.035 ms
 Execution Time: 0.075 ms
(8 rows)
```

ítem	variante	mediana ms	mín ms	máx ms	plan
a.1	Sin índice	0.070	0.068	0.088	Seq Scan
a.2	Índice hash sobre calle	0.008	0.008	0.009	Bitmap Heap Scan [Bitmap Index Scan (idx_comercios_calle_hash)]

a.3, a.4 y a.5 Comercios por producto principal

a.3, sin índice. `Seq Scan` sobre toda la tabla y, encima, un `HashAggregate`: por cada fila busca el producto en una tabla hash en memoria e incrementa su contador. Se forman 283 grupos y la tabla hash ocupa 61 kB, muy por debajo de `work_mem`, así que se resuelve en un solo lote (`Batches: 1`). Mediana: 0,162 ms.

a.4, con índice B-Tree. El plan cambia por completo: `Index Only Scan` sobre el índice y `GroupAggregate`. El B-Tree guarda los valores de productoprincipal ordenados, así que las filas de un mismo producto llegan juntas y para agrupar alcanza con contar hasta que cambia el valor, sin tabla hash. Y como la consulta sólo necesita esa columna, se responde leyendo el índice (5 páginas) sin ir a la tabla: `Heap Fetches: 0` indica que no hizo falta visitar ninguna fila para comprobar su visibilidad, gracias al `VACUUM` que se corrió después de crear el índice. Mediana: 0,083 ms, la mitad que sin índice. Es la corrección a la conclusión del borrador: el `read=4` del plan es el costo de la primera lectura del índice recién creado, no del plan.

a.5, con índice hash. Se borró antes el B-Tree, porque si existen los dos el planificador sigue eligiendo el B-Tree. Con sólo el hash, el plan es idéntico al del a.3 (`HashAggregate` sobre `Seq Scan`) y la mediana también (0,156 ms). El índice no se usa porque un hash no guarda el orden de los valores ni los valores mismos (sólo su código hash): no puede entregar las filas agrupadas ni responder la consulta por sí solo. Que el índice es válido se ve en la última consulta, que filtra un producto puntual (`= 'kiosco'`) y sí lo usa.

Conclusión de los tres ítems: para agrupar o para ordenar, el índice útil es el B-Tree; el hash sólo sirve para búsquedas por igualdad.

SCRIPT a02-producto-principal.sql

```
SET search_path TO public;

DELETE FROM medicion WHERE item IN ('a.3', 'a.4', 'a.5');
DROP INDEX IF EXISTS idx_comercios_pp_btree;
DROP INDEX IF EXISTS idx_comercios_pp_hash;
ANALYZE comercios;

-- a.3) Cantidad de comercios agrupados por producto principal, sin índices.

EXPLAIN (ANALYZE, BUFFERS)
SELECT c.productoprincipal, COUNT(*) AS cantidad
FROM comercios c
GROUP BY c.productoprincipal;

CALL pr_medir('a.3', 'Sin índice', $$
    SELECT c.productoprincipal, COUNT(*) FROM comercios c GROUP BY c.productoprincipal
$$, 50);

-- a.4) Índice B-Tree sobre productoprincipal y la misma consulta. VACUUM marca
--      las páginas como visibles para todos, requisito del Index Only Scan.

CREATE INDEX idx_comercios_pp_btree ON comercios USING btree (productoprincipal);
VACUUM ANALYZE comercios;

EXPLAIN (ANALYZE, BUFFERS)
SELECT c.productoprincipal, COUNT(*) AS cantidad
FROM comercios c
GROUP BY c.productoprincipal;

CALL pr_medir('a.4', 'Índice B-Tree', $$
    SELECT c.productoprincipal, COUNT(*) FROM comercios c GROUP BY c.productoprincipal
$$, 50);

-- a.5) Índice hash sobre productoprincipal. Se borra antes el B-Tree: si los dos
--      existen, el planificador sigue usando el B-Tree y no se ve el efecto del hash.

DROP INDEX idx_comercios_pp_btree;
CREATE INDEX idx_comercios_pp_hash ON comercios USING hash (productoprincipal);
ANALYZE comercios;

EXPLAIN (ANALYZE, BUFFERS)
SELECT c.productoprincipal, COUNT(*) AS cantidad
FROM comercios c
GROUP BY c.productoprincipal;

CALL pr_medir('a.5', 'Índice hash', $$
```

```
    SELECT c.productoprincipal, COUNT(*) FROM comercios c GROUP BY c.productoprincipal
$$, 50);

-- a.5) El hash no se usa para agrupar, pero sí para buscar un producto puntual.

EXPLAIN (ANALYZE, BUFFERS)
SELECT c.productoprincipal, COUNT(*) AS cantidad
FROM comercios c
WHERE c.productoprincipal = 'kiosco'
GROUP BY c.productoprincipal;

SELECT item, variante, mediana_ms, min_ms, max_ms, plan
FROM medicion
WHERE item IN ('a.3', 'a.4', 'a.5')
ORDER BY id;
```

SALIDA

```
                                                    QUERY PLAN
-------------------------------------------------------------------------------------------------------------------
 HashAggregate  (cost=88.38..91.20 rows=283 width=21) (actual time=0.191..0.201 rows=283 loops=1)
   Group Key: productoprincipal
   Batches: 1  Memory Usage: 61kB
   Buffers: shared hit=58
   ->  Seq Scan on comercios c  (cost=0.00..78.25 rows=2025 width=13) (actual time=0.001..0.042 rows=2025 loops=1)
         Buffers: shared hit=58
 Planning:
   Buffers: shared hit=18
 Planning Time: 0.033 ms
 Execution Time: 0.215 ms
(10 rows)

                                                              QUERY PLAN
-------------------------------------------------------------------------------------------------------------------
 GroupAggregate  (cost=0.28..63.61 rows=283 width=21) (actual time=0.010..0.127 rows=283 loops=1)
   Group Key: productoprincipal
   Buffers: shared hit=1 read=4
   ->  Index Only Scan using idx_comercios_pp_btree on comercios c  (cost=0.28..50.65 rows=2025 width=13)
         (actual time=0.008..0.057 rows=2025 loops=1)
         Heap Fetches: 0
         Buffers: shared hit=1 read=4
 Planning:
   Buffers: shared hit=10
 Planning Time: 0.020 ms
 Execution Time: 0.138 ms
(10 rows)

                                                    QUERY PLAN
-------------------------------------------------------------------------------------------------------------------
 HashAggregate  (cost=88.38..91.20 rows=283 width=21) (actual time=0.176..0.185 rows=283 loops=1)
   Group Key: productoprincipal
   Batches: 1  Memory Usage: 61kB
   Buffers: shared hit=58
   ->  Seq Scan on comercios c  (cost=0.00..78.25 rows=2025 width=13) (actual time=0.001..0.041 rows=2025 loops=1)
         Buffers: shared hit=58
 Planning:
   Buffers: shared hit=9
 Planning Time: 0.012 ms
 Execution Time: 0.194 ms
(10 rows)

                                                          QUERY PLAN
-------------------------------------------------------------------------------------------------------------------
 GroupAggregate  (cost=4.77..65.95 rows=1 width=21) (actual time=0.018..0.019 rows=1 loops=1)
   Buffers: shared hit=45
   ->  Bitmap Heap Scan on comercios c  (cost=4.77..65.69 rows=99 width=13)
         (actual time=0.008..0.016 rows=99 loops=1)
         Recheck Cond: ((productoprincipal)::text = 'kiosco'::text)
         Heap Blocks: exact=43
         Buffers: shared hit=45
         ->  Bitmap Index Scan on idx_comercios_pp_hash  (cost=0.00..4.74 rows=99 width=0)
               (actual time=0.004..0.004 rows=99 loops=1)
               Index Cond: ((productoprincipal)::text = 'kiosco'::text)
               Buffers: shared hit=2
 Planning Time: 0.015 ms
 Execution Time: 0.022 ms
(11 rows)
```

ítem	variante	mediana ms	mín ms	máx ms	plan
a.3	Sin índice	0.162	0.161	0.169	HashAggregate [Seq Scan]
a.4	Índice B-Tree	0.083	0.083	0.100	GroupAggregate [Index Only Scan (idx_comercios_pp_btree)]
a.5	Índice hash	0.156	0.154	0.184	HashAggregate [Seq Scan]

a.6 y a.7 Comercios del barrio 1

a.6, sin índice. Como en el a.1, un `Seq Scan` que recorre las 2.025 filas y devuelve 3. La estimación es exacta (`rows=3`). Mediana: 0,077 ms.

a.7, con índice hash sobre barrio. `Bitmap Index Scan` sobre el índice y `Bitmap Heap Scan` que lee sólo las 3 páginas con filas del barrio 1 (`Heap Blocks: exact=3`, 5 páginas en total). El costo estimado baja de 83,31 a 14,01 y la mediana de 0,077 ms a 0,003 ms: el barrio 1 es muy selectivo (0,15 % de la tabla) y es el caso ideal para un índice.

El script agrega una prueba que muestra que el planificador no usa el índice a ciegas: con el mismo índice, la consulta por el barrio 0 (Centro) vuelve al `Seq Scan`. Las estadísticas (`pg_stats.most_common_vals`) dicen que el barrio 0 tiene el 52,5 % de los comercios; para traer la mitad de la tabla el planificador estima más barato leerla entera en orden que ir y venir por el índice. Forzando el índice se ve que, en este caso, el planificador se equivoca por poco: 0,065 ms contra 0,091 ms. La razón es que el modelo de costos por defecto (`random_page_cost = 4`) supone que un acceso no secuencial cuesta cuatro veces más que uno secuencial, como en un disco rotativo, y acá las 58 páginas están en memoria. La diferencia es de microsegundos; la misma situación vuelve a aparecer, con más peso, en el a.9 y en el c.6.

SCRIPT a03-barrio.sql

```
SET search_path TO public;

DELETE FROM medicion WHERE item IN ('a.6', 'a.7');
DROP INDEX IF EXISTS idx_comercios_barrio_hash;
ANALYZE comercios;

-- a.6) Comercios que se encuentran en el barrio 1. El código 1 corresponde a
--      barrios.type = '1', "Cantera 25/ san jose" (ver a00-preparacion.sql).

SELECT b.type AS codigo, b.label AS barrio
FROM barrios b
WHERE b.type::INTEGER = 1;

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM comercios c
WHERE c.barrio = 1;

CALL pr_medir('a.6', 'Sin índice', $$
    SELECT * FROM comercios c WHERE c.barrio = 1
$$, 50);

-- a.7) Índice hash sobre barrio y la misma consulta.

CREATE INDEX idx_comercios_barrio_hash ON comercios USING hash (barrio);
ANALYZE comercios;

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM comercios c
WHERE c.barrio = 1;

CALL pr_medir('a.7', 'Índice hash, barrio 1', $$
    SELECT * FROM comercios c WHERE c.barrio = 1
$$, 50);

-- a.7) El mismo índice no se usa para el barrio 0 (Centro): las estadísticas dicen
--      que tiene más de la mitad de los comercios, y para traer la mitad de la tabla
--      conviene leerla entera en orden.

SELECT v.valor AS barrio, ROUND(v.frecuencia::NUMERIC * 100, 2) AS porcentaje
FROM pg_stats s
CROSS JOIN LATERAL unnest(s.most_common_vals::TEXT::INTEGER[], s.most_common_freqs)
     AS v(valor, frecuencia)
WHERE s.tablename = 'comercios'
  AND s.attname = 'barrio'
ORDER BY v.frecuencia DESC
LIMIT 5;

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM comercios c
WHERE c.barrio = 0;
```

```
CALL pr_medir('a.7', 'Índice hash, barrio 0', $$
    SELECT * FROM comercios c WHERE c.barrio = 0
$$, 50);

SET enable_seqscan = off;
CALL pr_medir('a.7', 'Barrio 0 forzando el índice', $$
    SELECT * FROM comercios c WHERE c.barrio = 0
$$, 50);
RESET enable_seqscan;

SELECT item, variante, mediana_ms, min_ms, max_ms, plan
FROM medicion
WHERE item IN ('a.6', 'a.7')
ORDER BY id;
```

SALIDA

```
 codigo |        barrio
--------+----------------------
 1      | Cantera 25/ san jose
(1 row)

                                                QUERY PLAN
-----------------------------------------------------------------------------------------------------------
 Seq Scan on comercios c  (cost=0.00..83.31 rows=3 width=281) (actual time=0.062..0.088 rows=3 loops=1)
   Filter: (barrio = 1)
   Rows Removed by Filter: 2022
   Buffers: shared hit=58
 Planning:
   Buffers: shared hit=237
 Planning Time: 0.081 ms
 Execution Time: 0.091 ms
(8 rows)

                                                        QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on comercios c  (cost=4.02..14.01 rows=3 width=281) (actual time=0.006..0.006 rows=3 loops=1)
   Recheck Cond: (barrio = 1)
   Heap Blocks: exact=3
   Buffers: shared hit=5
   ->  Bitmap Index Scan on idx_comercios_barrio_hash  (cost=0.00..4.02 rows=3 width=0)
         (actual time=0.004..0.004 rows=3 loops=1)
         Index Cond: (barrio = 1)
         Buffers: shared hit=2
 Planning:
   Buffers: shared hit=240
 Planning Time: 0.083 ms
 Execution Time: 0.012 ms
(11 rows)

 barrio | porcentaje
--------+------------
      0 |      52.49
     31 |       9.58
     32 |       7.56
     39 |       4.15
     33 |       2.77
(5 rows)

                                                QUERY PLAN
-----------------------------------------------------------------------------------------------------------
 Seq Scan on comercios c  (cost=0.00..83.31 rows=1063 width=281) (actual time=0.001..0.088 rows=1063 loops=1)
   Filter: (barrio = 0)
   Rows Removed by Filter: 962
   Buffers: shared hit=58
 Planning Time: 0.017 ms
 Execution Time: 0.107 ms
(6 rows)
```

ítem	variante	mediana ms	mín ms	máx ms	plan
a.6	Sin índice	0.077	0.076	0.083	Seq Scan
a.7	Índice hash, barrio 1	0.003	0.003	0.004	Bitmap Heap Scan [Bitmap Index Scan (idx_comercios_barrio_hash)]
a.7	Índice hash, barrio 0	0.091	0.087	0.108	Seq Scan
a.7	Barrio 0 forzando el índice	0.065	0.065	0.070	Bitmap Heap Scan [Bitmap Index Scan (idx_comercios_barrio_hash)]

a.8 Kioscos del barrio Centro sobre calle Galarza

El barrio y el rubro vienen dados por nombre, así que la consulta natural une comercios con barrios (por `type`, convertido a entero) y con clanae (rubro cuyo nombre empieza con "kiosco"). Se plantearon tres versiones con el mismo resultado, 7 comercios:

Versión 1, joins. El planificador empieza por barrios (un `Seq Scan` que encuentra la fila de Centro), y por cada barrio encontrado (uno) busca los comercios con un `Bitmap Heap Scan` que combina los dos índices hash existentes con un `BitmapAnd`: arma un mapa de bits con las páginas del barrio (índice de barrio), otro con las de Galarza (índice de calle) y los intersecta, que es exactamente la operación AND de los índices bitmap vista en teoría. Después, por cada comercio, busca su rubro en clanae por clave primaria, con un `Memoize` que guarda los rubros ya buscados (54 aciertos de 101 búsquedas). Mediana: 0,084 ms.

El plan tiene un error de estimación que conviene notar: al planificar no se sabe qué código de barrio va a salir de la tabla barrios, así que para `barrio = b.type::integer` estima 58 filas, un promedio, cuando el Centro tiene 1.063. Con esa estimación, usar el índice de barrio le parece barato.

Versión 2, subconsultas. El código del barrio se resuelve en una subconsulta que se evalúa una sola vez (`InitPlan`) y el rubro con `IN`. El plan es parecido, también con `BitmapAnd`, pero clanae se lee una sola vez (un `Seq Scan` que encuentra el único rubro de kioscos) en lugar de buscar el rubro de cada uno de los 101 comercios candidatos. Mediana: 0,051 ms.

Versión 3, códigos literales (barrio 0, CLANAE 47119), como si la aplicación ya los tuviera. Ahora el planificador sabe que el barrio 0 es la mitad de la tabla y no usa ese índice: va sólo por el hash de calle (156 filas de Galarza) y filtra barrio y rubro sobre ellas. Mediana: 0,019 ms, más de cuatro veces más rápido que la versión 1.

La mejora más grande para esta consulta llega en el a.11: con el B-Tree sobre (calle, clanae5) la versión 3 baja a 0,005 ms.

SCRIPT a04-centro-galarza-kiosco.sql

```
SET search_path TO public;

DELETE FROM medicion WHERE item = 'a.8';

-- a.8) Comercios del barrio Centro, en calle Galarza, con el código CLANAE del
--      rubro kioscos. El barrio y el rubro llegan por nombre, así que la consulta
--      natural resuelve los códigos con joins a barrios y clanae. Al llegar a este
--      punto existen los índices hash sobre calle (a.2) y sobre barrio (a.7).

SELECT cl.id, cl.nombre
FROM clanae cl
WHERE cl.nombre ILIKE 'kiosco%';

-- Versión 1: joins con barrios y clanae.

EXPLAIN (ANALYZE, BUFFERS)
SELECT c.id, c.nombre, c.calle, c.numero, b.label AS barrio, cl.nombre AS rubro
FROM comercios c
JOIN barrios b  ON b.type::INTEGER = c.barrio
JOIN clanae  cl ON cl.id = c.clanae5
WHERE b.label = 'Centro'
  AND c.calle = 'Galarza'
  AND cl.nombre ILIKE 'kiosco%'
ORDER BY c.numero;

CALL pr_medir('a.8', 'V1: joins con barrios y clanae', $$
    SELECT c.id, c.nombre, c.calle, c.numero, b.label AS barrio, cl.nombre AS rubro
    FROM comercios c
    JOIN barrios b  ON b.type::INTEGER = c.barrio
    JOIN clanae  cl ON cl.id = c.clanae5
    WHERE b.label = 'Centro' AND c.calle = 'Galarza' AND cl.nombre ILIKE 'kiosco%'
    ORDER BY c.numero
$$, 50);

-- Versión 2: los códigos se resuelven en subconsultas no correlacionadas.

EXPLAIN (ANALYZE, BUFFERS)
SELECT c.id, c.nombre, c.calle, c.numero
FROM comercios c
```

```
WHERE c.barrio  = (SELECT b.type::INTEGER FROM barrios b WHERE b.label = 'Centro')
  AND c.calle   = 'Galarza'
  AND c.clanae5 IN (SELECT cl.id FROM clanae cl WHERE cl.nombre ILIKE 'kiosco%')
ORDER BY c.numero;

CALL pr_medir('a.8', 'V2: subconsultas', $$
    SELECT c.id, c.nombre, c.calle, c.numero
    FROM comercios c
    WHERE c.barrio  = (SELECT b.type::INTEGER FROM barrios b WHERE b.label = 'Centro')
      AND c.calle   = 'Galarza'
      AND c.clanae5 IN (SELECT cl.id FROM clanae cl WHERE cl.nombre ILIKE 'kiosco%')
    ORDER BY c.numero
$$, 50);

-- Versión 3: códigos literales (barrio 0, CLANAE 47119), como si la aplicación
-- ya los tuviera resueltos.

EXPLAIN (ANALYZE, BUFFERS)
SELECT c.id, c.nombre, c.calle, c.numero
FROM comercios c
WHERE c.barrio  = 0
  AND c.calle   = 'Galarza'
  AND c.clanae5 = '47119'
ORDER BY c.numero;

CALL pr_medir('a.8', 'V3: códigos literales', $$
    SELECT c.id, c.nombre, c.calle, c.numero
    FROM comercios c
    WHERE c.barrio = 0 AND c.calle = 'Galarza' AND c.clanae5 = '47119'
    ORDER BY c.numero
$$, 50);

-- Resultado (igual en las tres versiones).

SELECT c.id, c.nombre, c.calle, c.numero, c.productoprincipal
FROM comercios c
WHERE c.barrio = 0 AND c.calle = 'Galarza' AND c.clanae5 = '47119'
ORDER BY c.numero;

SELECT item, variante, mediana_ms, min_ms, max_ms, plan
FROM medicion
WHERE item = 'a.8'
ORDER BY id;
```

SALIDA

```
  id   |        nombre
-------+-----------------------
 47119 | Kioscos y Polirrubros
(1 row)

                                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=25.42..25.42 rows=1 width=75) (actual time=0.144..0.145 rows=7 loops=1)
   Sort Key: c.numero
   Sort Method: quicksort  Memory: 25kB
   Buffers: shared hit=135
   ->  Nested Loop  (cost=10.05..25.41 rows=1 width=75) (actual time=0.045..0.138 rows=7 loops=1)
         Buffers: shared hit=132
         ->  Nested Loop  (cost=9.90..24.24 rows=4 width=51) (actual time=0.030..0.064 rows=101 loops=1)
               Buffers: shared hit=38
               ->  Seq Scan on barrios b  (cost=0.00..1.38 rows=1 width=14)
                     (actual time=0.002..0.004 rows=1 loops=1)
                     Filter: ((label)::text = 'Centro'::text)
                     Rows Removed by Filter: 29
                     Buffers: shared hit=1
               ->  Bitmap Heap Scan on comercios c  (cost=9.90..22.83 rows=4 width=43)
                     (actual time=0.027..0.056 rows=101 loops=1)
                     Recheck Cond: ((barrio = (b.type)::integer) AND ((calle)::text = 'Galarza'::text))
                     Heap Blocks: exact=30
                     Buffers: shared hit=37
                     ->  BitmapAnd  (cost=9.90..9.90 rows=4 width=0) (actual time=0.024..0.024 rows=0 loops=1)
                           Buffers: shared hit=7
                           ->  Bitmap Index Scan on idx_comercios_barrio_hash
                                 (cost=0.00..4.44 rows=58 width=0) (actual time=0.017..0.017 rows=1063 loops=1)
                                 Index Cond: (barrio = (b.type)::integer)
                                 Buffers: shared hit=5
                           ->  Bitmap Index Scan on idx_comercios_calle_hash
                                 (cost=0.00..5.17 rows=156 width=0) (actual time=0.005..0.005 rows=156 loops=1)
```

```
                              Index Cond: ((calle)::text = 'Galarza'::text)
                              Buffers: shared hit=2
        ->  Memoize  (cost=0.15..0.28 rows=1 width=36) (actual time=0.001..0.001 rows=0 loops=101)
              Cache Key: c.clanae5
              Cache Mode: logical
              Hits: 54  Misses: 47  Evictions: 0  Overflows: 0  Memory Usage: 4kB
              Buffers: shared hit=94
              ->  Index Scan using pkid1 on clanae cl  (cost=0.14..0.27 rows=1 width=36)
                    (actual time=0.001..0.001 rows=0 loops=47)
                    Index Cond: ((id)::text = (c.clanae5)::text)
                    Filter: ((nombre)::text ~~* 'kiosco%'::text)
                    Rows Removed by Filter: 1
                    Buffers: shared hit=94
 Planning:
   Buffers: shared hit=253 read=1
 Planning Time: 0.377 ms
 Execution Time: 0.161 ms
(38 rows)

                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=28.03..28.04 rows=1 width=33) (actual time=0.063..0.064 rows=7 loops=1)
   Sort Key: c.numero
   Sort Method: quicksort  Memory: 25kB
   Buffers: shared hit=38
   InitPlan 1
     ->  Seq Scan on barrios b  (cost=0.00..1.38 rows=1 width=4) (actual time=0.001..0.002 rows=1 loops=1)
           Filter: ((label)::text = 'Centro'::text)
           Rows Removed by Filter: 29
           Buffers: shared hit=1
   ->  Nested Loop  (cost=9.86..26.64 rows=1 width=33) (actual time=0.027..0.062 rows=7 loops=1)
         Join Filter: ((c.clanae5)::text = (cl.id)::text)
         Rows Removed by Join Filter: 94
         Buffers: shared hit=38
         ->  Seq Scan on clanae cl  (cost=0.00..3.83 rows=1 width=6) (actual time=0.007..0.030 rows=1 loops=1)
               Filter: ((nombre)::text ~~* 'kiosco%'::text)
               Rows Removed by Filter: 145
               Buffers: shared hit=2
         ->  Bitmap Heap Scan on comercios c  (cost=9.86..22.77 rows=4 width=39)
               (actual time=0.015..0.026 rows=101 loops=1)
               Recheck Cond: ((barrio = (InitPlan 1).col1) AND ((calle)::text = 'Galarza'::text))
               Heap Blocks: exact=30
               Buffers: shared hit=35
               ->  BitmapAnd  (cost=9.86..9.86 rows=4 width=0) (actual time=0.013..0.013 rows=0 loops=1)
                     Buffers: shared hit=5
                     ->  Bitmap Index Scan on idx_comercios_barrio_hash  (cost=0.00..4.43 rows=58 width=0)
                           (actual time=0.009..0.009 rows=1063 loops=1)
                           Index Cond: (barrio = (InitPlan 1).col1)
                           Buffers: shared hit=4
                     ->  Bitmap Index Scan on idx_comercios_calle_hash  (cost=0.00..5.17 rows=156 width=0)
                           (actual time=0.002..0.002 rows=156 loops=1)
                           Index Cond: ((calle)::text = 'Galarza'::text)
                           Buffers: shared hit=1
 Planning:
   Buffers: shared hit=4
 Planning Time: 0.110 ms
 Execution Time: 0.070 ms
(33 rows)

                                                                                              QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=65.96..65.97 rows=5 width=33) (actual time=0.032..0.032 rows=7 loops=1)
   Sort Key: numero
   Sort Method: quicksort  Memory: 25kB
   Buffers: shared hit=45
   ->  Bitmap Heap Scan on comercios c  (cost=5.17..65.90 rows=5 width=33)
         (actual time=0.006..0.031 rows=7 loops=1)
         Recheck Cond: ((calle)::text = 'Galarza'::text)
         Filter: ((barrio = 0) AND ((clanae5)::text = '47119'::text))
         Rows Removed by Filter: 149
         Heap Blocks: exact=44
         Buffers: shared hit=45
         ->  Bitmap Index Scan on idx_comercios_calle_hash  (cost=0.00..5.17 rows=156 width=0)
               (actual time=0.002..0.002 rows=156 loops=1)
               Index Cond: ((calle)::text = 'Galarza'::text)
               Buffers: shared hit=1
 Planning Time: 0.012 ms
 Execution Time: 0.034 ms
(15 rows)

  id  |           nombre            |  calle  | numero | productoprincipal
------+-----------------------------+---------+--------+-------------------
 1954 | tilcara                     | Galarza |    658 | kiosco
```

```
   90 | kiosco pucara               | Galarza |    736 | kiosco
 1226 | los nietos                  | Galarza |   1160 | kiosco
  492 | a toda hora                 | Galarza |   1229 | kiosco
  113 | sin identificacion comercial | Galarza |   1287 | kiosco
 1366 | el fer                      | Galarza |        | kiosco
 1516 | kiosco la muni              | Galarza |        | kiosco
(7 rows)
```

ítem	variante	mediana ms	mín ms	máx ms	plan
a.8	V1: joins con barrios y clanae	0.084	0.080	0.103	Sort [Nested Loop [Nested Loop [Seq Scan, Bitmap Heap Scan [BitmapAnd [Bitmap Index Scan (idx_comercios_barrio_hash), Bitmap Index Scan (idx_comercios_calle_hash)]]], Memoize [Index Scan (pkid1)]]]
a.8	V2: subconsultas	0.051	0.051	0.061	Sort [InitPlan Seq Scan, Nested Loop [Seq Scan, Bitmap Heap Scan [BitmapAnd [Bitmap Index Scan (idx_comercios_barrio_hash), Bitmap Index Scan (idx_comercios_calle_hash)]]]]
a.8	V3: códigos literales	0.019	0.019	0.021	Sort [Bitmap Heap Scan [Bitmap Index Scan (idx_comercios_calle_hash)]]

a.9 Pares de comercios cercanos del mismo rubro

Es un join de la tabla consigo misma: dos comercios forman par si tienen el mismo `clanae5`, la misma calle y números a 50 o menos de distancia. La condición `c2.id > c1.id` evita emparejar un comercio consigo mismo y que cada par aparezca dos veces. Hay 382 pares.

Plan sin índices. El planificador elige un `Merge Join`: lee las dos copias de la tabla con `Seq Scan`, ordena cada una por (clanae5, calle) con un `Sort` y las recorre en paralelo emparejando los grupos iguales. Las condiciones de igualdad son la `Merge Cond`; el rango sobre el número y la de los ids no se pueden usar para emparejar y quedan como `Join Filter`, que descarta 6.856 de los 7.238 pares de mismo rubro y calle. Mediana: 4,42 ms, casi todo en los dos `Sort`.

Ese plan no es el mejor disponible. Prohibiendo el Merge Join, el planificador usa un `Hash Join` sobre las mismas condiciones, que no ordena nada, y tarda 0,74 ms: seis veces menos. Sin embargo el costo estimado del Hash Join es 1.275,88 contra 516,36 del Merge Join. El modelo de costos no refleja lo caro que resulta, en esta base, ordenar texto: la base usa la collation `en_US.UTF-8`, que compara cadenas con reglas del idioma y no byte a byte. La prueba aislada lo confirma: la misma ordenación de calle y rubro tarda 1,63 ms con la collation de la base y 0,46 ms con la collation `"C"`.

Con índice. Se creó un B-Tree compuesto sobre (clanae5, calle, numero), con las dos igualdades primero y el rango al final, que es el orden en que un B-Tree puede aprovechar las tres condiciones. Con la configuración por defecto el planificador lo ignora y repite el plan anterior (4,40 ms): con `random_page_cost = 4` recorrer la tabla por el índice le resulta caro. Con `random_page_cost = 1.1`, el valor recomendado para SSD y el que corresponde a datos en memoria, el planificador sí lo usa: el `Merge Join` lee las dos entradas ya ordenadas desde el índice y se ahorra los dos `Sort`. Mediana: 1,22 ms, 3,6 veces más rápido que el plan original.

Conclusión: para esta consulta conviene el índice compuesto junto con un `random_page_cost` acorde al hardware. El índice se borra al final del script porque también cubre la consulta del a.10 y no dejaría ver el efecto del índice que pide el a.11.

SCRIPT a05-pares-cercanos.sql

```
SET search_path TO public;

DELETE FROM medicion WHERE item = 'a.9';
DROP INDEX IF EXISTS idx_comercios_rubro_calle_numero;
ANALYZE comercios;

-- a.9) Pares de comercios del mismo rubro (clanae5) que están cerca uno del otro:
--      misma calle y número a no más de 50 de distancia. La condición c2.id > c1.id
--      hace que cada par salga una sola vez y que un comercio no se empareje consigo.

EXPLAIN (ANALYZE, BUFFERS)
SELECT c1.clanae5, c1.calle,
       c1.id AS id_1, c1.nombre AS comercio_1, c1.numero AS numero_1,
       c2.id AS id_2, c2.nombre AS comercio_2, c2.numero AS numero_2
FROM comercios c1
JOIN comercios c2 ON c2.clanae5 = c1.clanae5
                 AND c2.calle   = c1.calle
                 AND c2.numero BETWEEN c1.numero - 50 AND c1.numero + 50
                 AND c2.id > c1.id;

CALL pr_medir('a.9', 'Sin índice (plan elegido)', $$
    SELECT c1.id, c1.nombre, c2.id, c2.nombre
    FROM comercios c1
    JOIN comercios c2 ON c2.clanae5 = c1.clanae5 AND c2.calle = c1.calle
                     AND c2.numero BETWEEN c1.numero - 50 AND c1.numero + 50
                     AND c2.id > c1.id
$$, 50);

-- El Merge Join gasta casi todo su tiempo en los dos Sort. Prohibiéndolo, el
-- planificador cae en el Hash Join, que no ordena nada. Se muestra primero el
-- costo estimado de cada alternativa sin ejecutarlas.

EXPLAIN
SELECT c1.id, c1.nombre, c2.id, c2.nombre
FROM comercios c1
JOIN comercios c2 ON c2.clanae5 = c1.clanae5 AND c2.calle = c1.calle
                 AND c2.numero BETWEEN c1.numero - 50 AND c1.numero + 50
```

```
                AND c2.id > c1.id;


SET enable_mergejoin = off;

EXPLAIN
SELECT c1.id, c1.nombre, c2.id, c2.nombre
FROM comercios c1
JOIN comercios c2 ON c2.clanae5 = c1.clanae5 AND c2.calle = c1.calle
                AND c2.numero BETWEEN c1.numero - 50 AND c1.numero + 50
                AND c2.id > c1.id;

CALL pr_medir('a.9', 'Sin índice, forzando Hash Join', $$
    SELECT c1.id, c1.nombre, c2.id, c2.nombre
    FROM comercios c1
    JOIN comercios c2 ON c2.clanae5 = c1.clanae5 AND c2.calle = c1.calle
                    AND c2.numero BETWEEN c1.numero - 50 AND c1.numero + 50
                    AND c2.id > c1.id
$$, 50);
RESET enable_mergejoin;

-- Los Sort son caros porque comparan texto con la collation de la base
-- (en_US.UTF-8). La misma ordenación con la collation "C" (comparación byte a byte):

SELECT datcollate FROM pg_database WHERE datname = current_database();

CALL pr_medir('a.9', 'Sólo el Sort, collation en_US', $$
    SELECT calle, clanae5 FROM comercios ORDER BY calle, clanae5
$$, 50);

CALL pr_medir('a.9', 'Sólo el Sort, collation C', $$
    SELECT calle, clanae5 FROM comercios ORDER BY calle COLLATE "C", clanae5 COLLATE "C"
$$, 50);

-- Índice B-Tree compuesto que cubre las tres condiciones del join: las dos
-- igualdades primero y el rango sobre numero al final.

CREATE INDEX idx_comercios_rubro_calle_numero ON comercios (clanae5, calle, numero);
VACUUM ANALYZE comercios;

EXPLAIN (ANALYZE, BUFFERS)
SELECT c1.clanae5, c1.calle,
       c1.id AS id_1, c1.nombre AS comercio_1, c1.numero AS numero_1,
       c2.id AS id_2, c2.nombre AS comercio_2, c2.numero AS numero_2
FROM comercios c1
JOIN comercios c2 ON c2.clanae5 = c1.clanae5
                AND c2.calle   = c1.calle
                AND c2.numero BETWEEN c1.numero - 50 AND c1.numero + 50
                AND c2.id > c1.id;

CALL pr_medir('a.9', 'Con índice (plan elegido)', $$
    SELECT c1.id, c1.nombre, c2.id, c2.nombre
    FROM comercios c1
    JOIN comercios c2 ON c2.clanae5 = c1.clanae5 AND c2.calle = c1.calle
                    AND c2.numero BETWEEN c1.numero - 50 AND c1.numero + 50
                    AND c2.id > c1.id
$$, 50);

-- Con random_page_cost = 1.1 (valor recomendado para SSD, y acá todo está en
-- memoria) el planificador deja de penalizar el acceso por índice y lo elige.

SET random_page_cost = 1.1;

EXPLAIN (ANALYZE, BUFFERS)
SELECT c1.clanae5, c1.calle,
       c1.id AS id_1, c1.nombre AS comercio_1, c1.numero AS numero_1,
       c2.id AS id_2, c2.nombre AS comercio_2, c2.numero AS numero_2
FROM comercios c1
JOIN comercios c2 ON c2.clanae5 = c1.clanae5
                AND c2.calle   = c1.calle
                AND c2.numero BETWEEN c1.numero - 50 AND c1.numero + 50
                AND c2.id > c1.id;

CALL pr_medir('a.9', 'Con índice, random_page_cost 1.1', $$
    SELECT c1.id, c1.nombre, c2.id, c2.nombre
    FROM comercios c1
    JOIN comercios c2 ON c2.clanae5 = c1.clanae5 AND c2.calle = c1.calle
                    AND c2.numero BETWEEN c1.numero - 50 AND c1.numero + 50
```

```
                    AND c2.id > c1.id
$$, 50);

RESET random_page_cost;

-- Resultado: cantidad de pares y los primeros diez.

SELECT COUNT(*) AS pares
FROM comercios c1
JOIN comercios c2 ON c2.clanae5 = c1.clanae5
                 AND c2.calle   = c1.calle
                 AND c2.numero BETWEEN c1.numero - 50 AND c1.numero + 50
                 AND c2.id > c1.id;

SELECT c1.clanae5, c1.calle,
       c1.nombre AS comercio_1, c1.numero AS numero_1,
       c2.nombre AS comercio_2, c2.numero AS numero_2
FROM comercios c1
JOIN comercios c2 ON c2.clanae5 = c1.clanae5
                 AND c2.calle   = c1.calle
                 AND c2.numero BETWEEN c1.numero - 50 AND c1.numero + 50
                 AND c2.id > c1.id
ORDER BY c1.calle, c1.numero, c2.numero
LIMIT 10;

-- El índice de este punto se borra: también cubre la consulta del a.10 (clanae5 y
-- calle por igualdad) y no dejaría ver el efecto del índice que pide el a.11.

DROP INDEX idx_comercios_rubro_calle_numero;

SELECT item, variante, mediana_ms, min_ms, max_ms, plan
FROM medicion
WHERE item = 'a.9'
ORDER BY id;
```

SALIDA

```
                                                              QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------------------
 Merge Join  (cost=378.92..516.36 rows=151 width=62) (actual time=3.715..4.562 rows=382 loops=1)
   Merge Cond: (((c1.clanae5)::text = (c2.clanae5)::text) AND ((c1.calle)::text = (c2.calle)::text))
   Join Filter: ((c2.id > c1.id) AND (c2.numero >= (c1.numero - 50)) AND (c2.numero <= (c1.numero + 50)))
   Rows Removed by Join Filter: 6856
   Buffers: shared hit=116
   ->  Sort  (cost=189.46..194.52 rows=2025 width=39) (actual time=1.827..1.861 rows=2025 loops=1)
         Sort Key: c1.clanae5, c1.calle
         Sort Method: quicksort  Memory: 168kB
         Buffers: shared hit=58
         ->  Seq Scan on comercios c1  (cost=0.00..78.25 rows=2025 width=39)
               (actual time=0.001..0.096 rows=2025 loops=1)
               Buffers: shared hit=58
   ->  Sort  (cost=189.46..194.52 rows=2025 width=39) (actual time=1.878..1.998 rows=7241 loops=1)
         Sort Key: c2.clanae5, c2.calle
         Sort Method: quicksort  Memory: 167kB
         Buffers: shared hit=58
         ->  Seq Scan on comercios c2  (cost=0.00..78.25 rows=2025 width=39)
               (actual time=0.001..0.090 rows=2025 loops=1)
               Buffers: shared hit=58
 Planning:
   Buffers: shared hit=161
 Planning Time: 0.257 ms
 Execution Time: 4.575 ms
(21 rows)

                                                 QUERY PLAN
-------------------------------------------------------------------------------------------------------------
 Merge Join  (cost=378.92..516.36 rows=151 width=38)
   Merge Cond: (((c1.clanae5)::text = (c2.clanae5)::text) AND ((c1.calle)::text = (c2.calle)::text))
   Join Filter: ((c2.id > c1.id) AND (c2.numero >= (c1.numero - 50)) AND (c2.numero <= (c1.numero + 50)))
   ->  Sort  (cost=189.46..194.52 rows=2025 width=39)
         Sort Key: c1.clanae5, c1.calle
         ->  Seq Scan on comercios c1  (cost=0.00..78.25 rows=2025 width=39)
   ->  Sort  (cost=189.46..194.52 rows=2025 width=39)
         Sort Key: c2.clanae5, c2.calle
         ->  Seq Scan on comercios c2  (cost=0.00..78.25 rows=2025 width=39)
(9 rows)

                                                 QUERY PLAN
-------------------------------------------------------------------------------------------------------------
```

```
 Hash Join  (cost=108.62..1275.88 rows=151 width=38)
   Hash Cond: (((c1.clanae5)::text = (c2.clanae5)::text) AND ((c1.calle)::text = (c2.calle)::text))
   Join Filter: ((c2.id > c1.id) AND (c2.numero >= (c1.numero - 50)) AND (c2.numero <= (c1.numero + 50)))
   ->  Seq Scan on comercios c1  (cost=0.00..78.25 rows=2025 width=39)
   ->  Hash  (cost=78.25..78.25 rows=2025 width=39)
         ->  Seq Scan on comercios c2  (cost=0.00..78.25 rows=2025 width=39)
(6 rows)

 datcollate
-------------
 en_US.UTF-8
(1 row)

                                                                 QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------------------------
 Merge Join  (cost=378.92..516.36 rows=151 width=62) (actual time=4.015..4.858 rows=382 loops=1)
   Merge Cond: (((c1.clanae5)::text = (c2.clanae5)::text) AND ((c1.calle)::text = (c2.calle)::text))
   Join Filter: ((c2.id > c1.id) AND (c2.numero >= (c1.numero - 50)) AND (c2.numero <= (c1.numero + 50)))
   Rows Removed by Join Filter: 6856
   Buffers: shared hit=116
   ->  Sort  (cost=189.46..194.52 rows=2025 width=39) (actual time=1.986..2.040 rows=2025 loops=1)
         Sort Key: c1.clanae5, c1.calle
         Sort Method: quicksort  Memory: 168kB
         Buffers: shared hit=58
         ->  Seq Scan on comercios c1  (cost=0.00..78.25 rows=2025 width=39)
               (actual time=0.001..0.094 rows=2025 loops=1)
               Buffers: shared hit=58
   ->  Sort  (cost=189.46..194.52 rows=2025 width=39) (actual time=2.020..2.140 rows=7241 loops=1)
         Sort Key: c2.clanae5, c2.calle
         Sort Method: quicksort  Memory: 167kB
         Buffers: shared hit=58
         ->  Seq Scan on comercios c2  (cost=0.00..78.25 rows=2025 width=39)
               (actual time=0.001..0.092 rows=2025 loops=1)
               Buffers: shared hit=58
 Planning:
   Buffers: shared hit=39 read=3
 Planning Time: 0.229 ms
 Execution Time: 4.868 ms
(21 rows)

                                                                            QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------------------------
 Merge Join  (cost=0.56..349.88 rows=151 width=62) (actual time=0.017..1.343 rows=382 loops=1)
   Merge Cond: (((c1.clanae5)::text = (c2.clanae5)::text) AND ((c1.calle)::text = (c2.calle)::text))
   Join Filter: ((c2.id > c1.id) AND (c2.numero >= (c1.numero - 50)) AND (c2.numero <= (c1.numero + 50)))
   Rows Removed by Join Filter: 6856
   Buffers: shared hit=3823 read=9
   ->  Index Scan using idx_comercios_rubro_calle_numero on comercios c1
         (cost=0.28..108.75 rows=2025 width=39) (actual time=0.002..0.157 rows=2025 loops=1)
         Buffers: shared hit=1916
   ->  Materialize  (cost=0.28..113.81 rows=2025 width=39) (actual time=0.002..0.368 rows=7241 loops=1)
         Buffers: shared hit=1907 read=9
         ->  Index Scan using idx_comercios_rubro_calle_numero on comercios c2
               (cost=0.28..108.75 rows=2025 width=39) (actual time=0.001..0.178 rows=2025 loops=1)
               Buffers: shared hit=1907 read=9
 Planning:
   Buffers: shared hit=12
 Planning Time: 0.175 ms
 Execution Time: 1.356 ms
(15 rows)

 pares
-------
   382
(1 row)

 clanae5 |    calle    |     comercio_1     | numero_1 | comercio_2  | numero_2
---------+-------------+--------------------+----------+-------------+----------
 47719   | 14 de julio | manequin boutique  |       27 | nogal       |       39
 47719   | 14 de julio | manequin boutique  |       27 | pomelo jean |       63
 47613   | 14 de julio | libreria argentina |       27 | libelula    |       68
 47719   | 14 de julio | manequin boutique  |       27 | magna       |       72
 47719   | 14 de julio | manequin boutique  |       27 | amarenas    |       73
 47719   | 14 de julio | nogal              |       39 | magna       |       72
 47719   | 14 de julio | nogal              |       39 | esphia      |       78
 47719   | 14 de julio | pomelo jean        |       63 | nogal       |       39
 47719   | 14 de julio | pomelo jean        |       63 | magna       |       72
 47719   | 14 de julio | pomelo jean        |       63 | esphia      |       78
(10 rows)
```

ítem	variante	mediana ms	mín ms	máx ms	plan
a.9	Sin índice (plan elegido)	4.424	4.197	4.986	Merge Join [Sort [Seq Scan], Sort [Seq Scan]]
a.9	Sin índice, forzando Hash Join	0.739	0.722	0.797	Hash Join [Seq Scan, Hash [Seq Scan]]
a.9	Sólo el Sort, collation en_US	1.627	1.583	1.831	Sort [Seq Scan]
a.9	Sólo el Sort, collation C	0.460	0.446	0.503	Sort [Seq Scan]
a.9	Con índice (plan elegido)	4.397	4.206	4.741	Merge Join [Sort [Seq Scan], Sort [Seq Scan]]
a.9	Con índice, random_page_cost 1.1	1.219	1.163	1.302	Merge Join [Index Scan (idx_comercios_rubro_calle_numero), Materialize [Index Scan (idx_comercios_rubro_calle_numero)]]

a.10 y a.11 Comercios de la calle 8 de junio con CLANAE 47119

a.10. Sigue existiendo el índice hash sobre calle del a.2, y el planificador lo usa: `Bitmap Index Scan` por calle = '8 de junio' (42 filas en 27 páginas) y el rubro se verifica como `Filter` en el `Bitmap Heap Scan`, que descarta 39 de las 42. Mediana: 0,008 ms.

a.11. Con el B-Tree compuesto sobre (calle, clanae5) el plan cambió: el `Bitmap Index Scan` ahora es sobre el índice nuevo y las dos condiciones aparecen juntas en `Index Cond`. El índice ya entrega sólo las 3 filas que cumplen ambas, y el `Bitmap Heap Scan` lee 3 páginas en lugar de 27. Mediana: 0,004 ms, la mitad. Que el planificador lo prefiera sobre el hash, que sigue existiendo, se debe a que estima 3 filas en lugar de 42.

Las dos consultas siguientes muestran la regla del prefijo izquierdo de los índices compuestos: el mismo índice sirve para filtrar sólo por calle (su primera columna) pero no para filtrar sólo por clanae5, que vuelve al `Seq Scan`. Por último, la versión 3 del a.8 también aprovecha el índice nuevo (0,019 ms antes, 0,005 ms ahora). La tabla final de la salida junta las mediciones del a.8 y de estos dos ítems.

SCRIPT a06-calle-y-rubro.sql

```
SET search_path TO public;

DELETE FROM medicion WHERE item IN ('a.10', 'a.11');
DROP INDEX IF EXISTS idx_comercios_calle_clanae;
ANALYZE comercios;

-- a.10) Comercios de la calle 8 de junio con CLANAE 47119 (kioscos y polirrubros).
--       Sigue existiendo el índice hash sobre calle creado en el a.2.

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM comercios c
WHERE c.calle = '8 de junio'
  AND c.clanae5 = '47119';

CALL pr_medir('a.10', 'Hash sobre calle', $$
    SELECT * FROM comercios c WHERE c.calle = '8 de junio' AND c.clanae5 = '47119'
$$, 50);

-- a.11) B-Tree compuesto sobre (calle, clanae5) y la misma consulta.

CREATE INDEX idx_comercios_calle_clanae ON comercios USING btree (calle, clanae5);
ANALYZE comercios;

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM comercios c
WHERE c.calle = '8 de junio'
  AND c.clanae5 = '47119';

CALL pr_medir('a.11', 'B-Tree (calle, clanae5)', $$
    SELECT * FROM comercios c WHERE c.calle = '8 de junio' AND c.clanae5 = '47119'
$$, 50);

-- a.11) El índice compuesto sirve también si se filtra sólo por calle (prefijo
--       izquierdo), pero no si se filtra sólo por clanae5, la segunda columna.

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM comercios c
WHERE c.calle = '8 de junio';

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM comercios c
WHERE c.clanae5 = '47119';

-- a.11) La consulta del a.8 (versión con códigos literales) también lo aprovecha.

EXPLAIN (ANALYZE, BUFFERS)
SELECT c.id, c.nombre, c.calle, c.numero
FROM comercios c
WHERE c.barrio  = 0
  AND c.calle   = 'Galarza'
  AND c.clanae5 = '47119'
```

```
ORDER BY c.numero;

CALL pr_medir('a.11', 'Consulta del a.8 (V3) con el B-Tree', $$
    SELECT c.id, c.nombre, c.calle, c.numero
    FROM comercios c
    WHERE c.barrio = 0 AND c.calle = 'Galarza' AND c.clanae5 = '47119'
    ORDER BY c.numero
$$, 50);

-- Resultado.

SELECT c.id, c.nombre, c.calle, c.numero, c.clanae5, c.productoprincipal
FROM comercios c
WHERE c.calle = '8 de junio'
  AND c.clanae5 = '47119'
ORDER BY c.numero;

SELECT item, variante, mediana_ms, min_ms, max_ms, plan
FROM medicion
WHERE item IN ('a.8', 'a.10', 'a.11')
ORDER BY id;
```

SALIDA

```
                                                                 QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on comercios c  (cost=4.32..60.96 rows=3 width=281) (actual time=0.009..0.014 rows=3 loops=1)
   Recheck Cond: ((calle)::text = '8 de junio'::text)
   Filter: ((clanae5)::text = '47119'::text)
   Rows Removed by Filter: 39
   Heap Blocks: exact=27
   Buffers: shared hit=29
   ->  Bitmap Index Scan on idx_comercios_calle_hash  (cost=0.00..4.32 rows=42 width=0)
         (actual time=0.006..0.006 rows=42 loops=1)
         Index Cond: ((calle)::text = '8 de junio'::text)
         Buffers: shared hit=2
 Planning:
   Buffers: shared hit=207
 Planning Time: 0.076 ms
 Execution Time: 0.020 ms
(13 rows)

                                                                 QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on comercios c  (cost=4.31..14.31 rows=3 width=281) (actual time=0.007..0.007 rows=3 loops=1)
   Recheck Cond: (((calle)::text = '8 de junio'::text) AND ((clanae5)::text = '47119'::text))
   Heap Blocks: exact=3
   Buffers: shared hit=3 read=2
   ->  Bitmap Index Scan on idx_comercios_calle_clanae  (cost=0.00..4.31 rows=3 width=0)
         (actual time=0.006..0.006 rows=3 loops=1)
         Index Cond: (((calle)::text = '8 de junio'::text) AND ((clanae5)::text = '47119'::text))
         Buffers: shared read=2
 Planning:
   Buffers: shared hit=208 read=1
 Planning Time: 0.083 ms
 Execution Time: 0.011 ms
(11 rows)

                                                                 QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on comercios c  (cost=4.33..60.86 rows=42 width=281) (actual time=0.004..0.008 rows=42 loops=1)
   Recheck Cond: ((calle)::text = '8 de junio'::text)
   Heap Blocks: exact=27
   Buffers: shared hit=28
   ->  Bitmap Index Scan on idx_comercios_calle_hash  (cost=0.00..4.32 rows=42 width=0)
         (actual time=0.002..0.002 rows=42 loops=1)
         Index Cond: ((calle)::text = '8 de junio'::text)
         Buffers: shared hit=1
 Planning Time: 0.011 ms
 Execution Time: 0.012 ms
(9 rows)

                                                 QUERY PLAN
-----------------------------------------------------------------------------------------------------------
 Seq Scan on comercios c  (cost=0.00..83.31 rows=125 width=281) (actual time=0.001..0.060 rows=125 loops=1)
   Filter: ((clanae5)::text = '47119'::text)
   Rows Removed by Filter: 1900
   Buffers: shared hit=58
 Planning Time: 0.010 ms
 Execution Time: 0.065 ms
```

```
(6 rows)

                                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=32.15..32.17 rows=5 width=33) (actual time=0.008..0.008 rows=7 loops=1)
   Sort Key: numero
   Sort Method: quicksort  Memory: 25kB
   Buffers: shared hit=9 read=1
   ->  Bitmap Heap Scan on comercios c  (cost=4.38..32.10 rows=5 width=33)
         (actual time=0.005..0.006 rows=7 loops=1)
         Recheck Cond: (((calle)::text = 'Galarza'::text) AND ((clanae5)::text = '47119'::text))
         Filter: (barrio = 0)
         Rows Removed by Filter: 1
         Heap Blocks: exact=8
         Buffers: shared hit=9 read=1
         ->  Bitmap Index Scan on idx_comercios_calle_clanae  (cost=0.00..4.38 rows=10 width=0)
               (actual time=0.004..0.004 rows=8 loops=1)
               Index Cond: (((calle)::text = 'Galarza'::text) AND ((clanae5)::text = '47119'::text))
               Buffers: shared hit=1 read=1
 Planning:
   Buffers: shared hit=5
 Planning Time: 0.019 ms
 Execution Time: 0.010 ms
(17 rows)

  id  |    nombre    |   calle    | numero | clanae5 | productoprincipal
------+--------------+------------+--------+---------+-------------------
 1468 | la tienda    | 8 de junio |    350 | 47119   | polirrubro
    1 | maria teresa | 8 de junio |    816 | 47119   | kiosco
  716 | vb           | 8 de junio |        | 47119   | kiosco
(3 rows)
```

ítem	variante	mediana ms	mín ms	máx ms	plan
a.8	V1: joins con barrios y clanae	0.084	0.080	0.103	Sort [Nested Loop [Nested Loop [Seq Scan, Bitmap Heap Scan [BitmapAnd [Bitmap Index Scan (idx_comercios_barrio_hash), Bitmap Index Scan (idx_comercios_calle_hash)]]], Memoize [Index Scan (pkid1)]]]
a.8	V2: subconsultas	0.051	0.051	0.061	Sort [InitPlan Seq Scan, Nested Loop [Seq Scan, Bitmap Heap Scan [BitmapAnd [Bitmap Index Scan (idx_comercios_barrio_hash), Bitmap Index Scan (idx_comercios_calle_hash)]]]]
a.8	V3: códigos literales	0.019	0.019	0.021	Sort [Bitmap Heap Scan [Bitmap Index Scan (idx_comercios_calle_hash)]]
a.10	Hash sobre calle	0.008	0.007	0.009	Bitmap Heap Scan [Bitmap Index Scan (idx_comercios_calle_hash)]
a.11	B-Tree (calle, clanae5)	0.004	0.004	0.005	Bitmap Heap Scan [Bitmap Index Scan (idx_comercios_calle_clanae)]
a.11	Consulta del a.8 (V3) con el B-Tree	0.005	0.005	0.005	Sort [Bitmap Heap Scan [Bitmap Index Scan (idx_comercios_calle_clanae)]]

a.12 Tres consultas propias

a.12.1 Subconsulta correlacionada: comercios con superficie mayor al promedio de su rubro. La forma directa calcula el promedio en una subconsulta que depende de la fila exterior. El plan lo muestra como `SubPlan`: por cada una de las 2.025 filas de comercios (`loops=2025`) ejecuta un `Seq Scan` completo de la tabla para promediar las de su rubro. Son 117.508 lecturas de página para una tabla de 58, y la mediana es de 142 ms.

Se probaron dos reescrituras. Con un CTE que calcula una vez el promedio de cada rubro (`GROUP BY`) y un join con comercios, el plan es un `Hash Join` entre la tabla y los 134 promedios: 0,50 ms, unas 280 veces más rápido. Con una función de ventana (`AVG(...) OVER (PARTITION BY clanae5)`) se hace en una sola pasada, pero el `WindowAgg` necesita las filas ordenadas por rubro y el `Sort` la deja en 1,25 ms. Un índice sobre (clanae5, superficie) mejora la versión correlacionada, porque cada subconsulta pasa a ser un `Index Only Scan` de las filas de su rubro (5,66 ms), pero sigue lejos de la reescritura, que con el mismo índice baja a 0,36 ms. El índice ayuda; cambiar la forma de la consulta ayuda mucho más.

a.12.2 Joins y agregación: las diez combinaciones barrio y rubro con más comercios. La versión directa une comercios con barrios y con clanae (dos `Hash Join`) y agrupa por los nombres: el `HashAggregate` recibe unas 1.900 filas con textos largos. La reescritura agrupa primero en comercios por los códigos (barrio, clanae5), que son cortos, y después une las 550 combinaciones resultantes con las tablas de nombres. Baja de 0,459 ms a 0,326 ms. Es correcta porque cada código tiene un solo nombre; agrupar por la clave y no por la descripción es, además, la forma correcta de agrupar.

a.12.3 Anti-join: rubros del nomenclador sin comercios (15). Con `NOT IN` el plan es un `Seq Scan` de clanae con un `hashed SubPlan`: carga los rubros de comercios en una tabla hash y descarta los que aparecen. Con `NOT EXISTS` el planificador lo reconoce como un anti-join (`Hash Right Anti Join`). Con un índice sobre clanae5 pasa a `Nested Loop Anti Join`: por cada rubro busca en el índice y corta en la primera coincidencia (`rows=1` por iteración). Las tres variantes tardan lo mismo (entre 0,115 y 0,130 ms), porque clanae tiene sólo 146 filas. La diferencia importante es de corrección: si la subconsulta devuelve algún NULL, `x NOT IN (..., NULL)` nunca es verdadero y la consulta no devuelve nada. El script lo prueba dentro de una transacción que se deshace: con un solo comercio sin rubro, `NOT IN` devuelve 0 rubros y `NOT EXISTS` sigue devolviendo 15. Por eso `NOT EXISTS` es la forma recomendable.

Un detalle de los planes: para leer los códigos de rubro de comercios el planificador usa un `Index Only Scan` sobre el índice (calle, clanae5) del a.11, aunque clanae5 sea su segunda columna. No busca nada en él: lo recorre entero porque es más chico que la tabla y ya tiene la columna que necesita.

SCRIPT a07-consultas-propias.sql

```
SET search_path TO public;

DELETE FROM medicion WHERE item LIKE 'a.12%';
DROP INDEX IF EXISTS idx_comercios_clanae_superficie;
DROP INDEX IF EXISTS idx_comercios_clanae;
ANALYZE comercios;

-- a.12.1) Subconsulta correlacionada. Comercios cuya superficie supera el
--         promedio de superficie de su rubro (los NULL no entran al promedio).

EXPLAIN (ANALYZE, BUFFERS)
SELECT c.id, c.nombre, c.clanae5, c.superficie
FROM comercios c
WHERE c.superficie > (SELECT AVG(c2.superficie)
                      FROM comercios c2
                      WHERE c2.clanae5 = c.clanae5);

CALL pr_medir('a.12.1', 'Subconsulta correlacionada', $$
    SELECT c.id, c.nombre, c.clanae5, c.superficie
    FROM comercios c
    WHERE c.superficie > (SELECT AVG(c2.superficie) FROM comercios c2
                          WHERE c2.clanae5 = c.clanae5)
$$, 20);

-- Reescritura 1: el promedio de cada rubro se calcula una sola vez en un CTE y
-- se une con comercios.

EXPLAIN (ANALYZE, BUFFERS)
WITH promedio_rubro AS (
    SELECT c2.clanae5, AVG(c2.superficie) AS promedio
```

```
    FROM comercios c2
    GROUP BY c2.clanae5
)
SELECT c.id, c.nombre, c.clanae5, c.superficie
FROM comercios c
JOIN promedio_rubro p ON p.clanae5 = c.clanae5
WHERE c.superficie > p.promedio;

CALL pr_medir('a.12.1', 'CTE con GROUP BY y join', $$
    WITH promedio_rubro AS (
        SELECT c2.clanae5, AVG(c2.superficie) AS promedio
        FROM comercios c2 GROUP BY c2.clanae5
    )
    SELECT c.id, c.nombre, c.clanae5, c.superficie
    FROM comercios c
    JOIN promedio_rubro p ON p.clanae5 = c.clanae5
    WHERE c.superficie > p.promedio
$$, 50);

-- Reescritura 2: función de ventana, una sola pasada sobre comercios.

CALL pr_medir('a.12.1', 'Función de ventana', $$
    SELECT id, nombre, clanae5, superficie
    FROM (SELECT c.id, c.nombre, c.clanae5, c.superficie,
                 AVG(c.superficie) OVER (PARTITION BY c.clanae5) AS promedio
          FROM comercios c) t
    WHERE superficie > promedio
$$, 50);

-- Índice sobre (clanae5, superficie): la subconsulta correlacionada pasa a
-- resolverse con un Index Only Scan por fila en lugar de un Seq Scan por fila.

CREATE INDEX idx_comercios_clanae_superficie ON comercios (clanae5, superficie);
VACUUM ANALYZE comercios;

EXPLAIN (ANALYZE, BUFFERS)
SELECT c.id, c.nombre, c.clanae5, c.superficie
FROM comercios c
WHERE c.superficie > (SELECT AVG(c2.superficie)
                      FROM comercios c2
                      WHERE c2.clanae5 = c.clanae5);

CALL pr_medir('a.12.1', 'Correlacionada con índice', $$
    SELECT c.id, c.nombre, c.clanae5, c.superficie
    FROM comercios c
    WHERE c.superficie > (SELECT AVG(c2.superficie) FROM comercios c2
                          WHERE c2.clanae5 = c.clanae5)
$$, 50);

CALL pr_medir('a.12.1', 'CTE con índice', $$
    WITH promedio_rubro AS (
        SELECT c2.clanae5, AVG(c2.superficie) AS promedio
        FROM comercios c2 GROUP BY c2.clanae5
    )
    SELECT c.id, c.nombre, c.clanae5, c.superficie
    FROM comercios c
    JOIN promedio_rubro p ON p.clanae5 = c.clanae5
    WHERE c.superficie > p.promedio
$$, 50);

DROP INDEX idx_comercios_clanae_superficie;

-- Las tres formas devuelven las mismas filas.

SELECT COUNT(*) AS comercios_sobre_el_promedio
FROM comercios c
WHERE c.superficie > (SELECT AVG(c2.superficie)
                      FROM comercios c2
                      WHERE c2.clanae5 = c.clanae5);

-- a.12.2) Joins y agregación. Las diez combinaciones barrio y rubro con más
--         comercios, con los nombres de ambos.

EXPLAIN (ANALYZE, BUFFERS)
SELECT b.label AS barrio, cl.nombre AS rubro, COUNT(*) AS comercios
FROM comercios c
JOIN barrios b  ON b.type::INTEGER = c.barrio
JOIN clanae  cl ON cl.id = c.clanae5
```

```
GROUP BY b.label, cl.nombre
ORDER BY comercios DESC, barrio, rubro
LIMIT 10;

CALL pr_medir('a.12.2', 'Join y después agrupar', $$
    SELECT b.label AS barrio, cl.nombre AS rubro, COUNT(*) AS comercios
    FROM comercios c
    JOIN barrios b  ON b.type::INTEGER = c.barrio
    JOIN clanae  cl ON cl.id = c.clanae5
    GROUP BY b.label, cl.nombre
    ORDER BY comercios DESC, barrio, rubro
    LIMIT 10
$$, 50);

-- Reescritura: se agrupa primero por los códigos, en la propia tabla comercios, y
-- recién después se buscan los nombres. El join recibe una fila por combinación
-- (unas 550) en lugar de una por comercio (unas 1900), y se agrupa por claves
-- cortas en lugar de por textos largos.

EXPLAIN (ANALYZE, BUFFERS)
SELECT b.label AS barrio, cl.nombre AS rubro, t.comercios
FROM (SELECT c.barrio, c.clanae5, COUNT(*) AS comercios
      FROM comercios c
      GROUP BY c.barrio, c.clanae5) t
JOIN barrios b  ON b.type::INTEGER = t.barrio
JOIN clanae  cl ON cl.id = t.clanae5
ORDER BY t.comercios DESC, barrio, rubro
LIMIT 10;

CALL pr_medir('a.12.2', 'Agrupar y después join', $$
    SELECT b.label AS barrio, cl.nombre AS rubro, t.comercios
    FROM (SELECT c.barrio, c.clanae5, COUNT(*) AS comercios
          FROM comercios c GROUP BY c.barrio, c.clanae5) t
    JOIN barrios b  ON b.type::INTEGER = t.barrio
    JOIN clanae  cl ON cl.id = t.clanae5
    ORDER BY t.comercios DESC, barrio, rubro
    LIMIT 10
$$, 50);

SELECT b.label AS barrio, cl.nombre AS rubro, t.comercios
FROM (SELECT c.barrio, c.clanae5, COUNT(*) AS comercios
      FROM comercios c
      GROUP BY c.barrio, c.clanae5) t
JOIN barrios b  ON b.type::INTEGER = t.barrio
JOIN clanae  cl ON cl.id = t.clanae5
ORDER BY t.comercios DESC, barrio, rubro
LIMIT 10;

-- a.12.3) Subconsulta de anti-join. Rubros del nomenclador CLANAE que no tienen
--         ningún comercio cargado, con NOT IN y con NOT EXISTS.

EXPLAIN (ANALYZE, BUFFERS)
SELECT cl.id, cl.nombre
FROM clanae cl
WHERE cl.id NOT IN (SELECT c.clanae5 FROM comercios c);

CALL pr_medir('a.12.3', 'NOT IN', $$
    SELECT cl.id, cl.nombre FROM clanae cl
    WHERE cl.id NOT IN (SELECT c.clanae5 FROM comercios c)
$$, 50);

EXPLAIN (ANALYZE, BUFFERS)
SELECT cl.id, cl.nombre
FROM clanae cl
WHERE NOT EXISTS (SELECT 1 FROM comercios c WHERE c.clanae5 = cl.id);

CALL pr_medir('a.12.3', 'NOT EXISTS', $$
    SELECT cl.id, cl.nombre FROM clanae cl
    WHERE NOT EXISTS (SELECT 1 FROM comercios c WHERE c.clanae5 = cl.id)
$$, 50);

-- Con un índice sobre comercios.clanae5 el anti-join pasa a Nested Loop: por cada
-- rubro busca en el índice y corta en la primera coincidencia.

CREATE INDEX idx_comercios_clanae ON comercios (clanae5);
VACUUM ANALYZE comercios;

EXPLAIN (ANALYZE, BUFFERS)
```

```
SELECT cl.id, cl.nombre
FROM clanae cl
WHERE NOT EXISTS (SELECT 1 FROM comercios c WHERE c.clanae5 = cl.id);

CALL pr_medir('a.12.3', 'NOT EXISTS con índice', $$
    SELECT cl.id, cl.nombre FROM clanae cl
    WHERE NOT EXISTS (SELECT 1 FROM comercios c WHERE c.clanae5 = cl.id)
$$, 50);

DROP INDEX idx_comercios_clanae;

-- NOT IN y NOT EXISTS sólo son equivalentes si la subconsulta no devuelve NULL.
-- Con un solo comercio sin rubro, NOT IN deja de devolver filas (x NOT IN (..., NULL)
-- nunca es verdadero), mientras que NOT EXISTS no cambia. Se prueba dentro de una
-- transacción que se deshace.

BEGIN;
UPDATE comercios SET clanae5 = NULL WHERE id = 1;

SELECT 'NOT IN' AS forma, COUNT(*) AS rubros_sin_comercios
FROM clanae cl
WHERE cl.id NOT IN (SELECT c.clanae5 FROM comercios c)
UNION ALL
SELECT 'NOT EXISTS', COUNT(*)
FROM clanae cl
WHERE NOT EXISTS (SELECT 1 FROM comercios c WHERE c.clanae5 = cl.id);
ROLLBACK;

SELECT cl.id, cl.nombre
FROM clanae cl
WHERE NOT EXISTS (SELECT 1 FROM comercios c WHERE c.clanae5 = cl.id)
ORDER BY cl.id;

SELECT item, variante, mediana_ms, min_ms, max_ms, plan
FROM medicion
WHERE item LIKE 'a.12%'
ORDER BY id;
```

SALIDA

```
                                                        QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------
 Seq Scan on comercios c  (cost=0.00..168897.44 rows=675 width=29) (actual time=0.372..141.138 rows=447 loops=1)
   Filter: ((superficie)::numeric > (SubPlan 1))
   Rows Removed by Filter: 1578
   Buffers: shared hit=117508
   SubPlan 1
     ->  Aggregate  (cost=83.35..83.36 rows=1 width=32) (actual time=0.069..0.069 rows=1 loops=2025)
           Buffers: shared hit=117450
           ->  Seq Scan on comercios c2  (cost=0.00..83.31 rows=15 width=4)
                 (actual time=0.004..0.065 rows=53 loops=2025)
                 Filter: ((clanae5)::text = (c.clanae5)::text)
                 Rows Removed by Filter: 1972
                 Buffers: shared hit=117450
 Planning:
   Buffers: shared hit=40
 Planning Time: 0.060 ms
 Execution Time: 141.156 ms
(15 rows)

                                                           QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------
 Hash Join  (cost=91.72..175.37 rows=675 width=29) (actual time=0.271..0.549 rows=447 loops=1)
   Hash Cond: ((c.clanae5)::text = (c2.clanae5)::text)
   Join Filter: ((c.superficie)::numeric > (avg(c2.superficie)))
   Rows Removed by Join Filter: 1578
   Buffers: shared hit=116
   ->  Seq Scan on comercios c  (cost=0.00..78.25 rows=2025 width=29) (actual time=0.001..0.042 rows=2025 loops=1)
         Buffers: shared hit=58
   ->  Hash  (cost=90.05..90.05 rows=134 width=38) (actual time=0.265..0.265 rows=134 loops=1)
         Buckets: 1024  Batches: 1  Memory Usage: 15kB
         Buffers: shared hit=58
         ->  HashAggregate  (cost=88.38..90.05 rows=134 width=38) (actual time=0.246..0.258 rows=134 loops=1)
               Group Key: c2.clanae5
               Batches: 1  Memory Usage: 64kB
               Buffers: shared hit=58
               ->  Seq Scan on comercios c2  (cost=0.00..78.25 rows=2025 width=10)
                     (actual time=0.000..0.041 rows=2025 loops=1)
                     Buffers: shared hit=58
```

```
 Planning:
   Buffers: shared hit=153
 Planning Time: 0.111 ms
 Execution Time: 0.565 ms
(20 rows)

                                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Seq Scan on comercios c  (cost=0.00..9383.12 rows=675 width=29) (actual time=0.034..7.429 rows=447 loops=1)
   Filter: ((superficie)::numeric > (SubPlan 1))
   Rows Removed by Filter: 1578
   Buffers: shared hit=4235 read=6
   SubPlan 1
     ->  Aggregate  (cost=4.58..4.59 rows=1 width=32) (actual time=0.003..0.004 rows=1 loops=2025)
           Buffers: shared hit=4177 read=6
           ->  Index Only Scan using idx_comercios_clanae_superficie on comercios c2
                 (cost=0.28..4.54 rows=15 width=4) (actual time=0.001..0.002 rows=53 loops=2025)
                 Index Cond: (clanae5 = (c.clanae5)::text)
                 Heap Fetches: 0
                 Buffers: shared hit=4177 read=6
 Planning:
   Buffers: shared hit=22
 Planning Time: 0.033 ms
 Execution Time: 7.440 ms
(15 rows)

 comercios_sobre_el_promedio
-----------------------------
                         447
(1 row)

                                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=187.40..187.42 rows=10 width=50) (actual time=0.576..0.577 rows=10 loops=1)
   Buffers: shared hit=64
   ->  Sort  (cost=187.40..191.72 rows=1728 width=50) (actual time=0.576..0.576 rows=10 loops=1)
         Sort Key: (count(*)) DESC, b.label, cl.nombre
         Sort Method: top-N heapsort  Memory: 26kB
         Buffers: shared hit=64
         ->  HashAggregate  (cost=132.78..150.06 rows=1728 width=50) (actual time=0.510..0.534 rows=550 loops=1)
               Group Key: b.label, cl.nombre
               Batches: 1  Memory Usage: 129kB
               Buffers: shared hit=61
               ->  Hash Join  (cost=6.96..119.82 rows=1728 width=42) (actual time=0.029..0.336 rows=1878 loops=1)
                     Hash Cond: ((c.clanae5)::text = (cl.id)::text)
                     Buffers: shared hit=61
                     ->  Hash Join  (cost=1.68..109.86 rows=1728 width=18)
                           (actual time=0.012..0.208 rows=1881 loops=1)
                           Hash Cond: (c.barrio = (b.type)::integer)
                           Buffers: shared hit=59
                           ->  Seq Scan on comercios c  (cost=0.00..78.25 rows=2025 width=10)
                                 (actual time=0.001..0.043 rows=2025 loops=1)
                                 Buffers: shared hit=58
                           ->  Hash  (cost=1.30..1.30 rows=30 width=14) (actual time=0.010..0.010 rows=30 loops=1)
                                 Buckets: 1024  Batches: 1  Memory Usage: 10kB
                                 Buffers: shared hit=1
                                 ->  Seq Scan on barrios b  (cost=0.00..1.30 rows=30 width=14)
                                       (actual time=0.004..0.007 rows=30 loops=1)
                                       Buffers: shared hit=1
                     ->  Hash  (cost=3.46..3.46 rows=146 width=36) (actual time=0.017..0.017 rows=146 loops=1)
                           Buckets: 1024  Batches: 1  Memory Usage: 18kB
                           Buffers: shared hit=2
                           ->  Seq Scan on clanae cl  (cost=0.00..3.46 rows=146 width=36)
                                 (actual time=0.003..0.008 rows=146 loops=1)
                                 Buffers: shared hit=2
 Planning:
   Buffers: shared hit=131
 Planning Time: 0.255 ms
 Execution Time: 0.585 ms
(32 rows)

                                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=106.55..106.58 rows=10 width=50) (actual time=0.383..0.384 rows=10 loops=1)
   Buffers: shared hit=61
   ->  Sort  (cost=106.55..106.61 rows=22 width=50) (actual time=0.383..0.383 rows=10 loops=1)
         Sort Key: (count(*)) DESC, b.label, cl.nombre
         Sort Method: top-N heapsort  Memory: 26kB
         Buffers: shared hit=61
         ->  Hash Join  (cost=100.40..106.08 rows=22 width=50) (actual time=0.250..0.343 rows=550 loops=1)
               Hash Cond: ((c.clanae5)::text = (cl.id)::text)
               Buffers: shared hit=61
               ->  Hash Join  (cost=95.11..100.71 rows=30 width=26) (actual time=0.236..0.295 rows=553 loops=1)
```

```
                    Hash Cond: (c.barrio = (b.type)::integer)
                    Buffers: shared hit=59
                    ->  HashAggregate  (cost=93.44..95.46 rows=202 width=18)
                          (actual time=0.232..0.255 rows=627 loops=1)
                          Group Key: c.barrio, c.clanae5
                          Batches: 1  Memory Usage: 121kB
                          Buffers: shared hit=58
                          ->  Seq Scan on comercios c  (cost=0.00..78.25 rows=2025 width=10)
                                (actual time=0.000..0.040 rows=2025 loops=1)
                                Buffers: shared hit=58
                    ->  Hash  (cost=1.30..1.30 rows=30 width=14) (actual time=0.004..0.004 rows=30 loops=1)
                          Buckets: 1024  Batches: 1  Memory Usage: 10kB
                          Buffers: shared hit=1
                          ->  Seq Scan on barrios b  (cost=0.00..1.30 rows=30 width=14)
                                (actual time=0.001..0.001 rows=30 loops=1)
                                Buffers: shared hit=1
              ->  Hash  (cost=3.46..3.46 rows=146 width=36) (actual time=0.013..0.013 rows=146 loops=1)
                    Buckets: 1024  Batches: 1  Memory Usage: 18kB
                    Buffers: shared hit=2
                    ->  Seq Scan on clanae cl  (cost=0.00..3.46 rows=146 width=36)
                          (actual time=0.001..0.005 rows=146 loops=1)
                          Buffers: shared hit=2
 Planning:
   Buffers: shared hit=2
 Planning Time: 0.036 ms
 Execution Time: 0.390 ms
(32 rows)

 barrio |                                      rubro                                       | comercios
--------+----------------------------------------------------------------------------------+-----------
 Centro | Venta de prendas y accesorios de vestir                                          |       146
 Centro | Kioscos y Polirrubros                                                            |        55
 Centro | Peluquerías                                                                      |        45
 Centro | Almacenes y Dietéticas                                                           |        37
 Centro | Servicios de preparación de comidas para llevar (Delivery de pizzas, rotiserías) |        31
 Centro | Papelerías y Librerías escolares                                                 |        27
 Centro | Restaurantes, bares, pizzerías y parrillas                                       |        24
 Centro | Farmacias                                                                        |        23
 Centro | Bazares y Regalerías                                                             |        20
 Centro | Venta de indumentaria para bebés y niños                                         |        20
(10 rows)

                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Seq Scan on clanae cl  (cost=71.72..75.54 rows=73 width=36) (actual time=0.202..0.210 rows=15 loops=1)
   Filter: (NOT (ANY ((id)::text = ((hashed SubPlan 1).col1)::text)))
   Rows Removed by Filter: 131
   Buffers: shared hit=6 read=5
   SubPlan 1
     ->  Index Only Scan using idx_comercios_calle_clanae on comercios c
           (cost=0.28..66.65 rows=2025 width=6) (actual time=0.026..0.099 rows=2025 loops=1)
           Heap Fetches: 0
           Buffers: shared hit=4 read=5
 Planning Time: 0.012 ms
 Execution Time: 0.214 ms
(10 rows)

                                                                                QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Hash Right Anti Join  (cost=5.56..97.93 rows=12 width=36) (actual time=0.141..0.143 rows=15 loops=1)
   Hash Cond: ((c.clanae5)::text = (cl.id)::text)
   Buffers: shared hit=11
   ->  Index Only Scan using idx_comercios_calle_clanae on comercios c
         (cost=0.28..66.65 rows=2025 width=6) (actual time=0.002..0.054 rows=2025 loops=1)
         Heap Fetches: 0
         Buffers: shared hit=9
   ->  Hash  (cost=3.46..3.46 rows=146 width=36) (actual time=0.014..0.014 rows=146 loops=1)
         Buckets: 1024  Batches: 1  Memory Usage: 18kB
         Buffers: shared hit=2
         ->  Seq Scan on clanae cl  (cost=0.00..3.46 rows=146 width=36) (actual time=0.001..0.005 rows=146 loops=1)
               Buffers: shared hit=2
 Planning:
   Buffers: shared hit=4
 Planning Time: 0.052 ms
 Execution Time: 0.146 ms
(15 rows)

                                                                              QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Nested Loop Anti Join  (cost=0.28..53.43 rows=12 width=36) (actual time=0.011..0.124 rows=15 loops=1)
   Buffers: shared hit=294 read=1
   ->  Seq Scan on clanae cl  (cost=0.00..3.46 rows=146 width=36) (actual time=0.001..0.004 rows=146 loops=1)
         Buffers: shared hit=2
```

```
   ->  Index Only Scan using idx_comercios_clanae on comercios c  (cost=0.28..0.68 rows=15 width=6)
         (actual time=0.001..0.001 rows=1 loops=146)
         Index Cond: (clanae5 = (cl.id)::text)
         Heap Fetches: 0
         Buffers: shared hit=292 read=1
 Planning:
   Buffers: shared hit=20 read=3
 Planning Time: 0.074 ms
 Execution Time: 0.126 ms
(12 rows)

   forma    | rubros_sin_comercios
------------+----------------------
 NOT IN     |                    0
 NOT EXISTS |                   15
(2 rows)

  id   |                              nombre
-------+---------------------------------------------------------------------
 45250 | Tapizado de autos
 47190 | Venta de Artículos Varios
 47715 | Venta de prendas de cuero
 47748 | Venta de cuadros y obras de arte
 47781 | Venta de muebles usados
 49211 | Empresas de transporte urbano (Colectivos)
 49213 | Transporte escolar
 49217 | Servicio de transporte automotor internacional de pasajeros
 49218 | Servicio de transporte automotor turístico de pasajeros
 49222 | Servicio de transporte automotor de mercaderías a granel
 52201 | Servicios de almacenamiento y depósito en silos
 52202 | Servicios de almacenamiento y depósito en cámaras frigoríficas
 52209 | Servicios de almacenamiento y depósito no clasificados previamente
 52300 | Servicios de gestión y logística para el transporte de mercaderías
 77110 | Alquiler de autos
(15 rows)
```

ítem	variante	mediana ms	mín ms	máx ms	plan
a.12.1	Subconsulta correlacionada	141.982	140.742	143.422	Seq Scan [SubPlan Aggregate [Seq Scan]]
a.12.1	CTE con GROUP BY y join	0.496	0.476	0.545	Hash Join [Seq Scan, Hash [HashAggregate [Seq Scan]]]
a.12.1	Función de ventana	1.245	1.175	1.374	Subquery Scan [WindowAgg [Sort [Seq Scan]]]
a.12.1	Correlacionada con índice	5.657	5.279	5.977	Seq Scan [SubPlan Aggregate [Index Only Scan (idx_comercios_clanae_superficie)]]
a.12.1	CTE con índice	0.356	0.343	0.396	Hash Join [Seq Scan, Hash [GroupAggregate [Index Only Scan (idx_comercios_clanae_superficie)]]]
a.12.2	Join y después agrupar	0.459	0.444	0.497	Limit [Sort [HashAggregate [Hash Join [Hash Join [Seq Scan, Hash [Seq Scan]], Hash [Seq Scan]]]]]
a.12.2	Agrupar y después join	0.326	0.315	0.370	Limit [Sort [Hash Join [Hash Join [HashAggregate [Seq Scan], Hash [Seq Scan]], Hash [Seq Scan]]]]
a.12.3	NOT IN	0.130	0.121	0.155	Seq Scan [SubPlan Index Only Scan (idx_comercios_calle_clanae)]
a.12.3	NOT EXISTS	0.115	0.111	0.138	Hash Right Anti Join [Index Only Scan (idx_comercios_calle_clanae), Hash [Seq Scan]]
a.12.3	NOT EXISTS con índice	0.120	0.118	0.132	Nested Loop Anti Join [Seq Scan, Index Only Scan (idx_comercios_clanae)]

Parte b. Artículos y precios

Las dos tablas se crean exactamente como las define el enunciado, con la función `random_string` dada. `articulos` tiene 500.000 filas (72 MB) y `precios` 8.000.000 (460 MB). Hay dos características de los datos que condicionan todos los planes de esta parte y que conviene tener presentes:

- `id_articulo` se genera entre 1 y 99.001, así que sólo uno de cada cinco artículos tiene precios, y cada uno de ellos tiene unos 80. Las fechas van del 1 de enero de 2022 al 26 de septiembre de 2024 (1.000 días) y los precios de 100 a 5.100.
- `CREATE TABLE AS` copia los datos pero no crea claves primarias ni índices: las dos tablas empiezan sin ningún índice. Y como las filas se generan al azar, ninguna columna salvo `id` tiene relación con el orden físico (las correlaciones de `pg_stats` son prácticamente 0).

b.0 Preparación de articulos y precios

Además de crear las tablas, el script fija la semilla, actualiza las estadísticas y muestra los tamaños, la distribución de los datos y la correlación de cada columna con el orden físico, que se usa en el b.1.

SCRIPT b00-preparacion.sql

```
SET search_path TO public;

-- b.0 Tablas articulos y precios, tal como las define el enunciado, en la base
--     optimizacion (compartida con las partes c y d):
--
--       createdb optimizacion
--       psql -d optimizacion -f 00-funciones-medicion.sql
--
--     setseed fija la semilla de random() para que los datos generados sean
--     siempre los mismos y las salidas de este documento se puedan reproducir.

DROP TABLE IF EXISTS precios;
DROP TABLE IF EXISTS articulos;
DELETE FROM medicion WHERE item LIKE 'b.%';

CREATE OR REPLACE FUNCTION public.random_string(length integer)
RETURNS text
LANGUAGE plpgsql AS $BODY$
declare
    chars text[] :=
        '{0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,S,T,U,V,W,X,Y,Z,a,b,c,d,e,f,g,h,i,j,k,l,m,
        n,o,p,q,r,s,t,u,v,w,x,y,z}';
    result text := '';
    i integer := 0;
begin
    if length < 0 then
        raise exception 'Given length cannot be less than 0';
    end if;
    for i in 1..length loop
        result := result || chars[1+random()*(array_length(chars, 1)-1)];
    end loop;
    return result;
end;
$BODY$;

SELECT setseed(0.2026);

create table articulos as
    select id::int8, random_string(100) as descripcion, round(random() * 30)::int8 as rubro
    from generate_series(1, 500000) as id;

create table precios as
    select id::int8,
           round(random() * 99000)::int8+1 as id_articulo,
           timestamp '2022-01-01 00:00:00' + (random() * (interval '1000 days')) as fecha,
           100.00+round(random() * 5000)::numeric(11,2) as precio
    from generate_series(1, 8000000) as id;

VACUUM ANALYZE articulos;
VACUUM ANALYZE precios;

-- Tamaño de las tablas. Ninguna de las dos tiene clave primaria ni índices:
-- CREATE TABLE AS sólo copia los datos.

SELECT c.relname                                AS tabla,
       c.reltuples::BIGINT                      AS filas,
       c.relpages                               AS paginas,
       pg_size_pretty(pg_relation_size(c.oid)) AS tamanio
FROM pg_class c
WHERE c.relname IN ('articulos', 'precios')
ORDER BY c.relname;

-- Características de los datos que condicionan los planes: id_articulo va de 1 a
-- 99.001, así que sólo uno de cada cinco artículos tiene precios; cada uno tiene
-- en promedio unos 80. Los rubros van de 0 a 30.

SELECT MIN(id_articulo)                         AS min_articulo,
       MAX(id_articulo)                         AS max_articulo,
       COUNT(DISTINCT id_articulo)              AS articulos_con_precio,
       ROUND(COUNT(*)::NUMERIC / COUNT(DISTINCT id_articulo), 1) AS precios_por_articulo,
```

```
       MIN(fecha)::DATE                         AS primera_fecha,
       MAX(fecha)::DATE                         AS ultima_fecha,
       MIN(precio)                              AS precio_min,
       MAX(precio)                              AS precio_max
FROM precios;

SELECT rubro, COUNT(*) AS articulos
FROM articulos
WHERE rubro IN (0, 5, 15, 30)
GROUP BY rubro
ORDER BY rubro;

-- Correlación entre el orden físico y el valor de cada columna, según las
-- estadísticas del planificador (1 = misma secuencia, 0 = sin relación).

SELECT tablename, attname, n_distinct, ROUND(correlation::NUMERIC, 3) AS correlacion
FROM pg_stats
WHERE tablename IN ('articulos', 'precios')
ORDER BY tablename, attname;
```

SALIDA

```
 setseed
---------

(1 row)

   tabla   |  filas  | paginas | tamanio
-----------+---------+---------+---------
 articulos |  500000 |    9152 | 72 MB
 precios   | 8000473 |   58880 | 460 MB
(2 rows)

 min_articulo | max_articulo | articulos_con_precio | precios_por_articulo | primera_fecha | ultima_fecha |
     precio_min | precio_max
--------------+--------------+----------------------+----------------------+---------------+--------------+--------
            1 |        99001 |                99001 |                 80.8 | 2022-01-01    | 2024-09-26   |
               100.00 |    5100.00
(1 row)

 rubro | articulos
-------+-----------
     0 |      8255
     5 |     16663
    15 |     16863
    30 |      8376
(4 rows)

 tablename |   attname   | n_distinct | correlacion
-----------+-------------+------------+-------------
 articulos | descripcion |         -1 |       0.004
 articulos | id          |         -1 |       1.000
 articulos | rubro       |         31 |       0.029
 precios   | fecha       |         -1 |      -0.004
 precios   | id          |         -1 |       1.000
 precios   | id_articulo |     100079 |       0.004
 precios   | precio      |       5001 |       0.007
(7 rows)
```

b.1 Artículos del rubro 5

Sin índice. `Gather` con un `Parallel Seq Scan`: el líder y dos procesos auxiliares (`Workers Launched: 2`) se reparten las 9.152 páginas de la tabla, cada uno filtra su parte (descarta unas 161.000 filas) y el `Gather` junta los resultados. Son 16.663 artículos, el 3,3 %. Mediana: 7,63 ms.

Con un B-Tree sobre rubro. `Bitmap Index Scan` sobre el índice y `Bitmap Heap Scan`. La mediana baja a 1,92 ms, pero el plan muestra un problema: `Heap Blocks: exact=7682`, es decir, para traer el 3 % de las filas tiene que leer el 84 % de las páginas. Los artículos de cada rubro están desparramados por toda la tabla (la correlación de rubro es 0,03) y casi todas las páginas tienen alguno del rubro 5. La ganancia viene de no tener que evaluar las 500.000 filas, no de leer menos.

Con CLUSTER. `CLUSTER articulos USING idx_articulos_rubro` reescribe la tabla en el orden del índice (tardó 1,2 s). Después la correlación de rubro pasa a 1 y el planificador cambia a un `Index Scan` común, que lee los 16.663 artículos en 321 páginas consecutivas. Mediana: 0,444 ms, 17 veces más rápido que sin índice.

CLUSTER tiene dos costos que hay que considerar: bloquea la tabla mientras se reescribe, y el orden no se mantiene, porque las filas que se inserten después van al final. En una tabla que se modifica seguido habría que repetirlo periódicamente. Además, sólo se puede ordenar físicamente por un criterio: después del CLUSTER la correlación de `id` cae de 1 a 0,03.

SCRIPT b01-rubro-5.sql

```
SET search_path TO public;

DELETE FROM medicion WHERE item = 'b.1';
DROP INDEX IF EXISTS idx_articulos_rubro;
ANALYZE articulos;

-- b.1) Artículos correspondientes al rubro 5, sin índices.

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM articulos a
WHERE a.rubro = 5;

CALL pr_medir('b.1', 'Sin índice', $$
    SELECT * FROM articulos a WHERE a.rubro = 5
$$, 10);

-- Índice B-Tree sobre rubro. El rubro 5 tiene el 3,3 % de los artículos.

CREATE INDEX idx_articulos_rubro ON articulos (rubro);
ANALYZE articulos;

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM articulos a
WHERE a.rubro = 5;

CALL pr_medir('b.1', 'Índice B-Tree sobre rubro', $$
    SELECT * FROM articulos a WHERE a.rubro = 5
$$, 10);

-- Aun con el índice se leen casi todas las páginas de la tabla: los artículos de
-- cada rubro están desparramados (correlación de rubro cercana a 0). CLUSTER
-- reescribe la tabla en el orden del índice y deja juntos los del mismo rubro.

\timing on
CLUSTER articulos USING idx_articulos_rubro;
\timing off
ANALYZE articulos;

SELECT attname, ROUND(correlation::NUMERIC, 3) AS correlacion
FROM pg_stats
WHERE tablename = 'articulos' AND attname IN ('id', 'rubro')
ORDER BY attname;

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM articulos a
WHERE a.rubro = 5;
```

```
CALL pr_medir('b.1', 'Índice B-Tree + CLUSTER', $$
    SELECT * FROM articulos a WHERE a.rubro = 5
$$, 10);

SELECT COUNT(*) AS articulos_rubro_5
FROM articulos a
WHERE a.rubro = 5;

SELECT a.id, LEFT(a.descripcion, 40) AS descripcion, a.rubro
FROM articulos a
WHERE a.rubro = 5
ORDER BY a.id
LIMIT 5;

SELECT item, variante, mediana_ms, min_ms, max_ms, plan
FROM medicion
WHERE item = 'b.1'
ORDER BY id;
```

SALIDA

```
                                                                 QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------
 Gather  (cost=1000.00..14456.17 rows=17000 width=117) (actual time=0.071..7.620 rows=16663 loops=1)
   Workers Planned: 2
   Workers Launched: 2
   Buffers: shared hit=2656 read=6496
   ->  Parallel Seq Scan on articulos a  (cost=0.00..11756.17 rows=7083 width=117)
         (actual time=0.018..5.723 rows=5554 loops=3)
         Filter: (rubro = 5)
         Rows Removed by Filter: 161112
         Buffers: shared hit=2656 read=6496
 Planning:
   Buffers: shared hit=19
 Planning Time: 0.030 ms
 Execution Time: 7.904 ms
(12 rows)

                                                                   QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on articulos a  (cost=188.69..9751.33 rows=16550 width=117)
     (actual time=0.882..7.280 rows=16663 loops=1)
   Recheck Cond: (rubro = 5)
   Heap Blocks: exact=7682
   Buffers: shared hit=3443 read=4259
   ->  Bitmap Index Scan on idx_articulos_rubro  (cost=0.00..184.55 rows=16550 width=0)
         (actual time=0.418..0.418 rows=16663 loops=1)
         Index Cond: (rubro = 5)
         Buffers: shared hit=3 read=17
 Planning:
   Buffers: shared hit=14 read=1
 Planning Time: 0.035 ms
 Execution Time: 7.564 ms
(11 rows)

Time: 1164.694 ms (00:01.165)
 attname | correlacion
---------+-------------
 id      |       0.033
 rubro   |       1.000
(2 rows)

                                                                QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------
 Index Scan using idx_articulos_rubro on articulos a  (cost=0.42..663.12 rows=16783 width=117)
     (actual time=0.006..0.847 rows=16663 loops=1)
   Index Cond: (rubro = 5)
   Buffers: shared read=321
 Planning:
   Buffers: shared hit=14 read=1
 Planning Time: 0.019 ms
 Execution Time: 1.117 ms
(7 rows)

 articulos_rubro_5
-------------------
             16663
(1 row)
```

```
 id |               descripcion                | rubro
----+------------------------------------------+-------
 10 | iU5xYzEB4BjRTakTxN7H8dnFPxk2qSgGG7TGf97L |     5
 38 | HlwfhPK1xxhAgU8aHAWAc88mlPAWNj1SBrHSYIWu |     5
 45 | kKEoug7kXexou3idy37Q8NUASahpqG6qPoOPRX5n |     5
 66 | LW2sfDswr6WqCOIbs5MILyJkcOc9LDjlG7h4qZZu |     5
 96 | pXiBJhi0SvXPevc9igyvaTwY2AUv9QHtIypOv9QL |     5
(5 rows)
```

ítem	variante	mediana ms	mín ms	máx ms	plan
b.1	Sin índice	7.630	7.508	7.985	`Gather [Parallel Seq Scan]`
b.1	Índice B-Tree sobre rubro	1.920	1.864	2.071	`Bitmap Heap Scan [Bitmap Index Scan (idx_articulos_rubro)]`
b.1	Índice B-Tree + CLUSTER	0.444	0.431	0.455	`Index Scan (idx_articulos_rubro)`

b.2 y b.3 Precio actual del artículo 10, con y sin descripción

El precio actual es el de fecha más reciente. Se agregó la condición `fecha <= now()` porque un precio cargado con fecha futura todavía no está vigente; con estos datos no cambia el resultado ($ 4.451, del 9 de septiembre de 2024), pero hace correcta la consulta.

b.2 sin índice. El plan es `Limit` sobre `Gather Merge` sobre `Sort` sobre `Parallel Seq Scan`. Cada proceso recorre su tercio de los 8 millones de precios para encontrar los 79 del artículo 10, cada uno ordena los suyos quedándose sólo con el más reciente (`top-N heapsort`, porque el `LIMIT 1` le permite no ordenar todo) y el `Gather Merge` combina los tres resultados manteniendo el orden. Lee las 58.880 páginas de la tabla: 66 ms.

Con un índice sobre id_articulo. El `Bitmap Index Scan` encuentra los 79 precios en el índice, el `Bitmap Heap Scan` lee las 79 páginas donde están (una por precio, porque están desparramados) y se ordenan para quedarse con el primero. Mediana: 0,015 ms, 4.400 veces menos.

Con el índice compuesto (id_articulo, fecha DESC) INCLUDE (precio). Dentro de cada artículo las entradas están ordenadas de la fecha más nueva a la más vieja, así que la primera que cumple `fecha <= now()` es la respuesta y el `Limit` corta ahí. Las dos condiciones están en `Index Cond` y, como el precio está incluido en el índice, es un `Index Only Scan` que no toca la tabla: lee 7 páginas del índice. Mediana: 0,003 ms. `INCLUDE` agrega columnas a las hojas del índice sin que formen parte de la clave: sirven para responder la consulta, no para buscar.

El índice compuesto ocupa 309 MB, dos tercios de la tabla, mientras que el simple ocupa 53 MB. La diferencia no es sólo por tener más columnas: desde PostgreSQL 13 el B-Tree deduplica las claves repetidas y guarda cada `id_articulo` una vez con la lista de sus filas, y en el compuesto cada entrada es distinta. Como el compuesto sirve también para cualquier búsqueda por `id_articulo` (es su primera columna), el simple queda redundante y se borra.

b.3, con la descripción. Hay que buscar además el artículo 10 en articulos, que no tiene clave primaria. El plan es un `Nested Loop` que toma el precio actual del índice anterior y busca el artículo con un `Parallel Seq Scan` de las 9.152 páginas de articulos: 7,70 ms, casi todo en esa búsqueda. Agregando la clave primaria que le falta a la tabla, la búsqueda pasa a ser un `Index Scan` sobre `articulos_pkey` y la consulta baja a 0,004 ms.

Se agregó una versión con `LATERAL`, que para un solo artículo rinde igual pero es la forma general: por cada artículo de la consulta exterior ejecuta la subconsulta del precio actual, que con el índice compuesto es una sola lectura del índice. Es la forma de pedir el precio actual de muchos artículos a la vez sin recorrer todos sus precios.

SCRIPT b02-precio-actual.sql

```
SET search_path TO public;

DELETE FROM medicion WHERE item IN ('b.2', 'b.3');
DROP INDEX IF EXISTS idx_precios_articulo;
DROP INDEX IF EXISTS idx_precios_articulo_fecha;
ALTER TABLE articulos DROP CONSTRAINT IF EXISTS articulos_pkey;
ANALYZE precios;

-- b.2) Precio actual del artículo 10: el de fecha más reciente que no sea
--      posterior al momento de la consulta (un precio con fecha futura todavía no
--      está vigente).

EXPLAIN (ANALYZE, BUFFERS)
SELECT p.precio, p.fecha
FROM precios p
WHERE p.id_articulo = 10
  AND p.fecha <= now()
ORDER BY p.fecha DESC
LIMIT 1;

CALL pr_medir('b.2', 'Sin índice', $$
    SELECT p.precio, p.fecha FROM precios p
    WHERE p.id_articulo = 10 AND p.fecha <= now()
    ORDER BY p.fecha DESC LIMIT 1
$$, 5);

-- Índice simple sobre id_articulo: encuentra los ~80 precios del artículo, pero
-- todavía hay que ir a la tabla por cada uno y ordenarlos por fecha.

CREATE INDEX idx_precios_articulo ON precios (id_articulo);
ANALYZE precios;
```

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT p.precio, p.fecha
FROM precios p
WHERE p.id_articulo = 10
  AND p.fecha <= now()
ORDER BY p.fecha DESC
LIMIT 1;

CALL pr_medir('b.2', 'Índice (id_articulo)', $$
    SELECT p.precio, p.fecha FROM precios p
    WHERE p.id_articulo = 10 AND p.fecha <= now()
    ORDER BY p.fecha DESC LIMIT 1
$$, 20);

-- Índice compuesto (id_articulo, fecha DESC) que además incluye el precio: las
-- entradas de cada artículo ya están ordenadas de la más reciente a la más vieja,
-- así que alcanza con leer la primera, y como el precio está en el índice no hace
-- falta ir a la tabla (Index Only Scan).

CREATE INDEX idx_precios_articulo_fecha ON precios (id_articulo, fecha DESC) INCLUDE (precio);
VACUUM ANALYZE precios;

EXPLAIN (ANALYZE, BUFFERS)
SELECT p.precio, p.fecha
FROM precios p
WHERE p.id_articulo = 10
  AND p.fecha <= now()
ORDER BY p.fecha DESC
LIMIT 1;

CALL pr_medir('b.2', 'Índice (id_articulo, fecha DESC) INCLUDE (precio)', $$
    SELECT p.precio, p.fecha FROM precios p
    WHERE p.id_articulo = 10 AND p.fecha <= now()
    ORDER BY p.fecha DESC LIMIT 1
$$, 20);

-- Tamaño de los dos índices. El simple ocupa mucho menos porque el B-Tree
-- deduplica las claves repetidas (cada id_articulo se guarda una vez con la
-- lista de sus filas); en el compuesto cada entrada es distinta.

SELECT c.relname AS indice, pg_size_pretty(pg_relation_size(c.oid)) AS tamanio
FROM pg_class c
WHERE c.relname IN ('precios', 'idx_precios_articulo', 'idx_precios_articulo_fecha')
ORDER BY pg_relation_size(c.oid) DESC;

-- El índice simple queda redundante: el compuesto sirve para cualquier búsqueda
-- por id_articulo porque es su primera columna.

DROP INDEX idx_precios_articulo;

SELECT p.id_articulo, p.precio, p.fecha
FROM precios p
WHERE p.id_articulo = 10
  AND p.fecha <= now()
ORDER BY p.fecha DESC
LIMIT 1;

-- b.3) Precio actual y descripción del artículo 10. La tabla articulos no tiene
--      clave primaria (CREATE TABLE AS no la crea), así que para encontrar el
--      artículo 10 hay que recorrerla entera.

EXPLAIN (ANALYZE, BUFFERS)
SELECT a.id, a.descripcion, p.precio, p.fecha
FROM articulos a
JOIN precios p ON p.id_articulo = a.id
WHERE a.id = 10
  AND p.fecha <= now()
ORDER BY p.fecha DESC
LIMIT 1;

CALL pr_medir('b.3', 'Sin clave primaria en articulos', $$
    SELECT a.id, a.descripcion, p.precio, p.fecha
    FROM articulos a JOIN precios p ON p.id_articulo = a.id
    WHERE a.id = 10 AND p.fecha <= now()
    ORDER BY p.fecha DESC LIMIT 1
$$, 10);
```

```
ALTER TABLE articulos ADD PRIMARY KEY (id);
ANALYZE articulos;

EXPLAIN (ANALYZE, BUFFERS)
SELECT a.id, a.descripcion, p.precio, p.fecha
FROM articulos a
JOIN precios p ON p.id_articulo = a.id
WHERE a.id = 10
  AND p.fecha <= now()
ORDER BY p.fecha DESC
LIMIT 1;

CALL pr_medir('b.3', 'Con clave primaria en articulos', $$
    SELECT a.id, a.descripcion, p.precio, p.fecha
    FROM articulos a JOIN precios p ON p.id_articulo = a.id
    WHERE a.id = 10 AND p.fecha <= now()
    ORDER BY p.fecha DESC LIMIT 1
$$, 20);

-- La misma consulta con LATERAL. Para un solo artículo rinde igual, pero es la
-- forma que generaliza: el precio actual de muchos artículos a la vez, con una
-- lectura de una entrada del índice por artículo.

EXPLAIN (ANALYZE, BUFFERS)
SELECT a.id, a.descripcion, pa.precio, pa.fecha
FROM articulos a
CROSS JOIN LATERAL (
    SELECT p.precio, p.fecha
    FROM precios p
    WHERE p.id_articulo = a.id
      AND p.fecha <= now()
    ORDER BY p.fecha DESC
    LIMIT 1
) pa
WHERE a.id = 10;

CALL pr_medir('b.3', 'LATERAL con clave primaria', $$
    SELECT a.id, a.descripcion, pa.precio, pa.fecha
    FROM articulos a
    CROSS JOIN LATERAL (SELECT p.precio, p.fecha FROM precios p
                        WHERE p.id_articulo = a.id AND p.fecha <= now()
                        ORDER BY p.fecha DESC LIMIT 1) pa
    WHERE a.id = 10
$$, 20);

SELECT a.id, a.descripcion, pa.precio, pa.fecha
FROM articulos a
CROSS JOIN LATERAL (
    SELECT p.precio, p.fecha
    FROM precios p
    WHERE p.id_articulo = a.id
      AND p.fecha <= now()
    ORDER BY p.fecha DESC
    LIMIT 1
) pa
WHERE a.id = 10;

SELECT item, variante, mediana_ms, min_ms, max_ms, plan
FROM medicion
WHERE item IN ('b.2', 'b.3')
ORDER BY id;
```

SALIDA

```
                                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=118218.92..118219.03 rows=1 width=13) (actual time=63.603..64.340 rows=1 loops=1)
   Buffers: shared hit=2704 read=56288
   ->  Gather Merge  (cost=118218.92..118226.62 rows=66 width=13) (actual time=63.603..64.340 rows=1 loops=1)
         Workers Planned: 2
         Workers Launched: 2
         Buffers: shared hit=2704 read=56288
         ->  Sort  (cost=117218.89..117218.98 rows=33 width=13) (actual time=62.683..62.683 rows=1 loops=3)
               Sort Key: fecha DESC
               Sort Method: top-N heapsort  Memory: 25kB
               Buffers: shared hit=2704 read=56288
               Worker 0:  Sort Method: top-N heapsort  Memory: 25kB
               Worker 1:  Sort Method: top-N heapsort  Memory: 25kB
```

```
               ->  Parallel Seq Scan on precios p  (cost=0.00..117218.73 rows=33 width=13)
                     (actual time=1.531..62.658 rows=26 loops=3)
                     Filter: ((id_articulo = 10) AND (fecha <= now()))
                     Rows Removed by Filter: 2666640
                     Buffers: shared hit=2592 read=56288
 Planning:
   Buffers: shared hit=34
 Planning Time: 0.061 ms
 Execution Time: 64.352 ms
(20 rows)

                                                                                                  QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=329.59..329.60 rows=1 width=13) (actual time=0.134..0.134 rows=1 loops=1)
   Buffers: shared hit=10 read=75
   ->  Sort  (cost=329.59..329.80 rows=83 width=13) (actual time=0.134..0.134 rows=1 loops=1)
         Sort Key: fecha DESC
         Sort Method: top-N heapsort  Memory: 25kB
         Buffers: shared hit=10 read=75
         ->  Bitmap Heap Scan on precios p  (cost=5.08..329.18 rows=83 width=13)
               (actual time=0.022..0.128 rows=79 loops=1)
               Recheck Cond: (id_articulo = 10)
               Filter: (fecha <= now())
               Heap Blocks: exact=79
               Buffers: shared hit=10 read=75
               ->  Bitmap Index Scan on idx_precios_articulo  (cost=0.00..5.05 rows=83 width=0)
                     (actual time=0.016..0.016 rows=79 loops=1)
                     Index Cond: (id_articulo = 10)
                     Buffers: shared hit=3 read=3
 Planning:
   Buffers: shared hit=29 read=1
 Planning Time: 0.074 ms
 Execution Time: 0.139 ms
(18 rows)

                                                                                                      QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=0.43..0.50 rows=1 width=13) (actual time=0.263..0.263 rows=1 loops=1)
   Buffers: shared hit=4 read=3
   ->  Index Only Scan using idx_precios_articulo_fecha on precios p  (cost=0.43..6.10 rows=83 width=13)
         (actual time=0.263..0.263 rows=1 loops=1)
         Index Cond: ((id_articulo = 10) AND (fecha <= now()))
         Heap Fetches: 0
         Buffers: shared hit=4 read=3
 Planning:
   Buffers: shared hit=27
 Planning Time: 0.088 ms
 Execution Time: 0.271 ms
(10 rows)

           indice           | tamanio
----------------------------+---------
 precios                    | 460 MB
 idx_precios_articulo_fecha | 309 MB
 idx_precios_articulo       | 53 MB
(3 rows)

 id_articulo | precio  |           fecha
-------------+---------+----------------------------
          10 | 4451.00 | 2024-09-09 18:11:41.513997
(1 row)

                                                                                                  QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=1000.43..1141.42 rows=1 width=122) (actual time=6.974..7.648 rows=1 loops=1)
   Buffers: shared hit=628 read=8467
   ->  Nested Loop  (cost=1000.43..12702.40 rows=83 width=122) (actual time=6.974..7.648 rows=1 loops=1)
         Buffers: shared hit=628 read=8467
         ->  Index Only Scan using idx_precios_articulo_fecha on precios p
               (cost=0.43..6.10 rows=83 width=21) (actual time=0.002..0.002 rows=1 loops=1)
               Index Cond: ((id_articulo = 10) AND (fecha <= now()))
               Heap Fetches: 0
               Buffers: shared hit=4
         ->  Materialize  (cost=1000.00..12695.27 rows=1 width=109) (actual time=6.970..7.644 rows=1 loops=1)
               Buffers: shared hit=624 read=8467
               ->  Gather  (cost=1000.00..12695.27 rows=1 width=109) (actual time=6.962..7.635 rows=1 loops=1)
                     Workers Planned: 2
                     Workers Launched: 2
                     Buffers: shared hit=624 read=8467
                     ->  Parallel Seq Scan on articulos a  (cost=0.00..11695.17 rows=1 width=109)
                           (actual time=4.079..5.730 rows=0 loops=3)
                           Filter: (id = 10)
                           Rows Removed by Filter: 166666
```

```
                          Buffers: shared hit=624 read=8467
 Planning:
   Buffers: shared hit=51
 Planning Time: 0.085 ms
 Execution Time: 7.653 ms
(22 rows)

                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=0.86..1.03 rows=1 width=122) (actual time=0.012..0.013 rows=1 loops=1)
   Buffers: shared hit=5 read=3
   ->  Nested Loop  (cost=0.86..15.58 rows=83 width=122) (actual time=0.012..0.012 rows=1 loops=1)
         Buffers: shared hit=5 read=3
         ->  Index Only Scan using idx_precios_articulo_fecha on precios p
               (cost=0.43..6.10 rows=83 width=21) (actual time=0.004..0.004 rows=1 loops=1)
               Index Cond: ((id_articulo = 10) AND (fecha <= now()))
               Heap Fetches: 0
               Buffers: shared hit=4
         ->  Materialize  (cost=0.42..8.45 rows=1 width=109) (actual time=0.007..0.007 rows=1 loops=1)
               Buffers: shared hit=1 read=3
               ->  Index Scan using articulos_pkey on articulos a  (cost=0.42..8.44 rows=1 width=109)
                     (actual time=0.006..0.006 rows=1 loops=1)
                     Index Cond: (id = 10)
                     Buffers: shared hit=1 read=3
 Planning:
   Buffers: shared hit=11 read=1
 Planning Time: 0.046 ms
 Execution Time: 0.018 ms
(17 rows)

                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Nested Loop  (cost=0.86..8.95 rows=1 width=122) (actual time=0.004..0.005 rows=1 loops=1)
   Buffers: shared hit=8
   ->  Index Scan using articulos_pkey on articulos a  (cost=0.42..8.44 rows=1 width=109)
         (actual time=0.001..0.001 rows=1 loops=1)
         Index Cond: (id = 10)
         Buffers: shared hit=4
   ->  Limit  (cost=0.43..0.50 rows=1 width=13) (actual time=0.003..0.003 rows=1 loops=1)
         Buffers: shared hit=4
         ->  Index Only Scan using idx_precios_articulo_fecha on precios p
               (cost=0.43..6.10 rows=83 width=13) (actual time=0.003..0.003 rows=1 loops=1)
               Index Cond: ((id_articulo = a.id) AND (fecha <= now()))
               Heap Fetches: 0
               Buffers: shared hit=4
 Planning Time: 0.019 ms
 Execution Time: 0.008 ms
(13 rows)

 id |                                            descripcion                                                                  |
     precio  |           fecha
----+-----------------------------------------------------------------------------------------------------------------------+-------
 10 | iU5xYzEB4BjRTakTxN7H8dnFPxk2qSgGG7TGf97LPhwUC8niBVXA16DO8jVwHWba0ahSTW7vPYikms5YwAaw9jp2ChsD3d84x9PG |
     4451.00 | 2024-09-09 18:11:41.513997
(1 row)
```

ítem	variante	mediana ms	mín ms	máx ms	plan
b.2	Sin índice	66.316	64.820	69.449	Limit [Gather Merge [Sort [Parallel Seq Scan]]]
b.2	Índice (id_articulo)	0.015	0.015	0.017	Limit [Sort [Bitmap Heap Scan [Bitmap Index Scan (idx_precios_articulo)]]]
b.2	Índice (id_articulo, fecha DESC) INCLUDE (precio)	0.003	0.002	0.003	Limit [Index Only Scan (idx_precios_articulo_fecha)]
b.3	Sin clave primaria en articulos	7.696	7.314	9.630	Limit [Nested Loop [Index Only Scan (idx_precios_articulo_fecha), Materialize [Gather [Parallel Seq Scan]]]]
b.3	Con clave primaria en articulos	0.004	0.004	0.005	Limit [Nested Loop [Index Only Scan (idx_precios_articulo_fecha), Materialize [Index Scan (articulos_pkey)]]]
b.3	LATERAL con clave primaria	0.004	0.004	0.004	Nested Loop [Index Scan (articulos_pkey), Limit [Index Only Scan (idx_precios_articulo_fecha)]]

b.4 Artículos con precios entre 1000 y 1500

Se interpretó como los artículos que tienen al menos un precio registrado en ese rango. El 10 % de los precios cae entre 1.000 y 1.500 (800.521) y, como cada artículo tiene unos 80, casi todos los que tienen precios califican: son 98.971.

Join y DISTINCT. Es la forma más intuitiva: unir artículos con sus precios del rango y eliminar los repetidos. El plan hace un `Parallel Hash Join` (la tabla hash de articulos no entra en `work_mem` y se parte en 16 lotes que pasan por disco), produce 800.000 filas, una por precio, y para quitar duplicados las ordena por las tres columnas del artículo, incluida la descripción de 100 caracteres: es un `Sort` con `external merge` de unos 35 MB por proceso. Mediana: 326 ms.

EXISTS. Lo que se pregunta es si existe algún precio en el rango, y escrito así el planificador lo resuelve como un `Parallel Hash Semi Join`: por cada artículo alcanza con una coincidencia, no se multiplican filas y no hace falta eliminar duplicados. Mediana: 228 ms, un 30 % menos.

EXISTS con índice sobre precio. El rango se resuelve en el índice, pero las 800.000 filas están repartidas por toda la tabla y el `Bitmap Heap Scan` termina leyendo sus 58.824 páginas (`Heap Blocks: exact=58824`). Mejora poco (174 ms): se ahorra evaluar el filtro sobre los 8 millones de filas, pero no lectura. Con una selectividad del 10 % y datos sin orden, un índice que obliga a ir a la tabla no sirve de mucho.

EXISTS con índice (precio, id_articulo). La subconsulta sólo necesita esas dos columnas, así que se resuelve con un `Index Only Scan` del tramo del índice que corresponde al rango, sin tocar la tabla precios. Los ids encontrados se deduplican con un `HashAggregate` y se unen con articulos. Mediana: 107 ms, tres veces más rápido que la versión inicial.

SCRIPT b03-precios-rango.sql

```
SET search_path TO public;

DELETE FROM medicion WHERE item = 'b.4';
DROP INDEX IF EXISTS idx_precios_precio;
DROP INDEX IF EXISTS idx_precios_precio_articulo;
ANALYZE precios;

-- b.4) Artículos cuyos precios están entre 1000 y 1500: los que tienen al menos un
--      precio registrado en ese rango. Un 10 % de los precios cae en el rango, y
--      como cada artículo tiene unos 80, casi todos los que tienen precios califican.

-- Versión 1: join y DISTINCT para no repetir el artículo una vez por precio.

EXPLAIN (ANALYZE, BUFFERS)
SELECT DISTINCT a.id, a.descripcion, a.rubro
FROM articulos a
JOIN precios p ON p.id_articulo = a.id
WHERE p.precio BETWEEN 1000 AND 1500;

CALL pr_medir('b.4', 'JOIN + DISTINCT', $$
    SELECT DISTINCT a.id, a.descripcion, a.rubro
    FROM articulos a JOIN precios p ON p.id_articulo = a.id
    WHERE p.precio BETWEEN 1000 AND 1500
$$, 5);

-- Versión 2: EXISTS. Lo que se pregunta es si existe algún precio en el rango, y
-- así lo resuelve el motor: un semi-join que no multiplica filas ni necesita
-- eliminar duplicados después.

EXPLAIN (ANALYZE, BUFFERS)
SELECT a.id, a.descripcion, a.rubro
FROM articulos a
WHERE EXISTS (SELECT 1
              FROM precios p
              WHERE p.id_articulo = a.id
                AND p.precio BETWEEN 1000 AND 1500);

CALL pr_medir('b.4', 'EXISTS (semi-join)', $$
    SELECT a.id, a.descripcion, a.rubro FROM articulos a
    WHERE EXISTS (SELECT 1 FROM precios p
                  WHERE p.id_articulo = a.id AND p.precio BETWEEN 1000 AND 1500)
$$, 5);
```

```
-- Índice sobre precio: el rango se resuelve en el índice, pero las 800.000 filas
-- están repartidas por toda la tabla y el Bitmap Heap Scan termina leyendo todas
-- sus páginas.

CREATE INDEX idx_precios_precio ON precios (precio);
ANALYZE precios;

EXPLAIN (ANALYZE, BUFFERS)
SELECT a.id, a.descripcion, a.rubro
FROM articulos a
WHERE EXISTS (SELECT 1
              FROM precios p
              WHERE p.id_articulo = a.id
                AND p.precio BETWEEN 1000 AND 1500);

CALL pr_medir('b.4', 'EXISTS + índice (precio)', $$
    SELECT a.id, a.descripcion, a.rubro FROM articulos a
    WHERE EXISTS (SELECT 1 FROM precios p
                  WHERE p.id_articulo = a.id AND p.precio BETWEEN 1000 AND 1500)
$$, 5);

-- Índice (precio, id_articulo): tiene las dos columnas que la subconsulta
-- necesita, así que se resuelve con un Index Only Scan sobre el 10 % del índice
-- sin tocar la tabla precios.

DROP INDEX idx_precios_precio;
CREATE INDEX idx_precios_precio_articulo ON precios (precio, id_articulo);
VACUUM ANALYZE precios;

EXPLAIN (ANALYZE, BUFFERS)
SELECT a.id, a.descripcion, a.rubro
FROM articulos a
WHERE EXISTS (SELECT 1
              FROM precios p
              WHERE p.id_articulo = a.id
                AND p.precio BETWEEN 1000 AND 1500);

CALL pr_medir('b.4', 'EXISTS + índice (precio, id_articulo)', $$
    SELECT a.id, a.descripcion, a.rubro FROM articulos a
    WHERE EXISTS (SELECT 1 FROM precios p
                  WHERE p.id_articulo = a.id AND p.precio BETWEEN 1000 AND 1500)
$$, 5);

SELECT COUNT(*) AS articulos_con_precios_en_rango
FROM articulos a
WHERE EXISTS (SELECT 1
              FROM precios p
              WHERE p.id_articulo = a.id
                AND p.precio BETWEEN 1000 AND 1500);

SELECT item, variante, mediana_ms, min_ms, max_ms, plan
FROM medicion
WHERE item = 'b.4'
ORDER BY id;
```

SALIDA

```
                                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------------------------
 Unique  (cost=186249.72..272662.42 rows=500000 width=117) (actual time=299.743..346.460 rows=98971 loops=1)
   Buffers: shared hit=6489 read=61610, temp read=23542 written=23731
   ->  Gather Merge  (cost=186249.72..267596.20 rows=675496 width=117)
         (actual time=299.743..339.925 rows=114309 loops=1)
         Workers Planned: 2
         Workers Launched: 2
         Buffers: shared hit=6489 read=61610, temp read=23542 written=23731
         ->  Unique  (cost=185249.70..188627.18 rows=337748 width=117)
               (actual time=297.867..328.550 rows=38103 loops=3)
               Buffers: shared hit=6489 read=61610, temp read=23542 written=23731
               ->  Sort  (cost=185249.70..186094.07 rows=337748 width=117)
                     (actual time=297.867..316.710 rows=266840 loops=3)
                     Sort Key: a.id, a.descripcion, a.rubro
                     Sort Method: external merge  Disk: 35288kB
                     Buffers: shared hit=6489 read=61610, temp read=23542 written=23731
                     Worker 0:  Sort Method: external merge  Disk: 30960kB
                     Worker 1:  Sort Method: external merge  Disk: 35712kB
                     ->  Parallel Hash Join  (cost=17441.50..133455.50 rows=337748 width=117)
                           (actual time=216.026..242.647 rows=266840 loops=3)
```

```
                          Hash Cond: (p.id_articulo = a.id)
                          Buffers: shared hit=6359 read=61610, temp read=10797 written=10956
                          ->  Parallel Seq Scan on precios p  (cost=0.00..108824.40 rows=337748 width=8)
                                (actual time=0.036..147.159 rows=266840 loops=3)
                                Filter: ((precio >= '1000'::numeric) AND (precio <= '1500'::numeric))
                                Rows Removed by Filter: 2399826
                                Buffers: shared hit=4176 read=54648
                          ->  Parallel Hash  (cost=11174.33..11174.33 rows=208333 width=117)
                                (actual time=46.654..46.654 rows=166667 loops=3)
                                Buckets: 65536  Batches: 16  Memory Usage: 5216kB
                                Buffers: shared hit=2129 read=6962, temp written=8092
                                ->  Parallel Seq Scan on articulos a
                                      (cost=0.00..11174.33 rows=208333 width=117)
                                      (actual time=0.015..6.178 rows=166667 loops=3)
                                      Buffers: shared hit=2129 read=6962
 Planning:
   Buffers: shared hit=124 read=4
 Planning Time: 0.217 ms
 Execution Time: 348.842 ms
(30 rows)

                                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------
 Gather  (cost=115366.25..146174.69 rows=96893 width=117) (actual time=214.114..239.750 rows=98971 loops=1)
   Workers Planned: 2
   Workers Launched: 2
   Buffers: shared hit=7845 read=60266, temp read=10956 written=11020
   ->  Parallel Hash Semi Join  (cost=114366.25..135485.39 rows=40372 width=117)
         (actual time=215.318..232.633 rows=32990 loops=3)
         Hash Cond: (a.id = p.id_articulo)
         Buffers: shared hit=7845 read=60266, temp read=10956 written=11020
         ->  Parallel Seq Scan on articulos a  (cost=0.00..11174.33 rows=208333 width=117)
               (actual time=0.112..6.021 rows=166667 loops=3)
               Buffers: shared hit=2801 read=6290
         ->  Parallel Hash  (cost=108824.40..108824.40 rows=337748 width=8)
               (actual time=173.442..173.442 rows=266840 loops=3)
               Buckets: 262144  Batches: 8  Memory Usage: 6080kB
               Buffers: shared hit=4848 read=53976, temp written=2428
               ->  Parallel Seq Scan on precios p  (cost=0.00..108824.40 rows=337748 width=8)
                     (actual time=0.018..150.145 rows=266840 loops=3)
                     Filter: ((precio >= '1000'::numeric) AND (precio <= '1500'::numeric))
                     Rows Removed by Filter: 2399826
                     Buffers: shared hit=4848 read=53976
 Planning:
   Buffers: shared hit=12
 Planning Time: 0.059 ms
 Execution Time: 241.428 ms
(20 rows)

                                                                        QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------
 Hash Join  (cost=91814.47..108308.08 rows=97987 width=117) (actual time=205.085..244.305 rows=98971 loops=1)
   Hash Cond: (a.id = p.id_articulo)
   Buffers: shared hit=59 read=70046 written=671
   ->  Seq Scan on articulos a  (cost=0.00..14091.00 rows=500000 width=117)
         (actual time=0.004..16.080 rows=500000 loops=1)
         Buffers: shared read=9091
   ->  Hash  (cost=90589.87..90589.87 rows=97968 width=8) (actual time=205.072..205.072 rows=98971 loops=1)
         Buckets: 131072  Batches: 1  Memory Usage: 4891kB
         Buffers: shared hit=59 read=60955 written=671
         ->  HashAggregate  (cost=89610.19..90589.87 rows=97968 width=8)
               (actual time=198.771..201.713 rows=98971 loops=1)
               Group Key: p.id_articulo
               Batches: 1  Memory Usage: 7185kB
               Buffers: shared hit=59 read=60955 written=671
               ->  Bitmap Heap Scan on precios p  (cost=16878.41..87623.36 rows=794730 width=8)
                     (actual time=23.396..152.438 rows=800521 loops=1)
                     Recheck Cond: ((precio >= '1000'::numeric) AND (precio <= '1500'::numeric))
                     Heap Blocks: exact=58824
                     Buffers: shared hit=59 read=60955 written=671
                     ->  Bitmap Index Scan on idx_precios_precio
                           (cost=0.00..16679.73 rows=794730 width=0)
                           (actual time=18.993..18.993 rows=800521 loops=1)
                           Index Cond: ((precio >= '1000'::numeric) AND (precio <= '1500'::numeric))
                           Buffers: shared read=2190
 Planning:
   Buffers: shared hit=43 read=1
 Planning Time: 0.164 ms
 Execution Time: 245.998 ms
(23 rows)

                                                                            QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------
```

```
 Hash Join  (cost=32216.72..48703.74 rows=97395 width=117) (actual time=95.040..131.854 rows=98971 loops=1)
   Hash Cond: (a.id = p.id_articulo)
   Buffers: shared hit=395275 read=12131, temp read=49 written=117
   ->  Seq Scan on articulos a  (cost=0.00..14091.00 rows=500000 width=117)
         (actual time=0.003..15.804 rows=500000 loops=1)
         Buffers: shared hit=30 read=9061
   ->  Hash  (cost=30999.52..30999.52 rows=97376 width=8) (actual time=95.031..95.031 rows=98971 loops=1)
         Buckets: 131072  Batches: 1  Memory Usage: 4891kB
         Buffers: shared hit=395245 read=3070, temp read=49 written=117
         ->  HashAggregate  (cost=30025.76..30999.52 rows=97376 width=8)
               (actual time=85.848..91.980 rows=98971 loops=1)
               Group Key: p.id_articulo
               Batches: 5  Memory Usage: 10033kB  Disk Usage: 672kB
               Buffers: shared hit=395245 read=3070, temp read=49 written=117
               ->  Index Only Scan using idx_precios_precio_articulo on precios p
                     (cost=0.43..28045.61 rows=792059 width=8) (actual time=0.011..43.081 rows=800521 loops=1)
                     Index Cond: ((precio >= '1000'::numeric) AND (precio <= '1500'::numeric))
                     Heap Fetches: 0
                     Buffers: shared hit=395245 read=3070
 Planning:
   Buffers: shared hit=20 read=9
 Planning Time: 0.475 ms
 Execution Time: 133.698 ms
(20 rows)

 articulos_con_precios_en_rango
--------------------------------
                          98971
(1 row)
```

ítem	variante	mediana ms	mín ms	máx ms	plan
b.4	JOIN + DISTINCT	326.449	314.617	337.744	Unique [Gather Merge [Unique [Sort [Parallel Hash Join [Parallel Seq Scan, Parallel Hash [Parallel Seq Scan]]]]]]
b.4	EXISTS (semi-join)	228.334	220.339	233.119	Gather [Parallel Hash Semi Join [Parallel Seq Scan, Parallel Hash [Parallel Seq Scan]]]
b.4	EXISTS + índice (precio)	174.306	173.657	175.281	Hash Join [Seq Scan, Hash [HashAggregate [Bitmap Heap Scan [Bitmap Index Scan (idx_precios_precio)]]]]
b.4	EXISTS + índice (precio, id_articulo)	107.207	106.447	108.099	Hash Join [Seq Scan, Hash [HashAggregate [Index Only Scan (idx_precios_precio_articulo)]]]

b.5 y b.6 Precio promedio por artículo y por rubro

b.5. Hay que leer los 8 millones de precios y formar 99.001 grupos. A esta altura existen los índices de los ítems anteriores, y el planificador usa el (id_articulo, fecha DESC) INCLUDE (precio): un `Parallel Index Only Scan` que entrega los precios ya ordenados por artículo, cada proceso agrupa su parte con un `Partial GroupAggregate` y el `Finalize GroupAggregate` combina los resultados parciales del mismo artículo. Mediana: 505 ms.

Para comparar con la situación sin índices se deshabilitaron los accesos por índice en la sesión. El plan pasa a `Parallel Seq Scan` con `Partial HashAggregate`, y la tabla hash de 99.000 grupos no entra en los 4 MB de `work_mem`: se parte en lotes que escriben unos 68 MB a disco por proceso (`Disk Usage`), y el `Sort` posterior también es `external merge`. Mediana: 663 ms. Con `work_mem = 64MB` todo se resuelve en memoria y baja a 588 ms.

Las tres variantes están en el mismo orden de magnitud porque las tres tienen que procesar los 8 millones de precios. Si el promedio se consulta seguido, lo que cambia el resultado es no recalcularlo cada vez: una **vista materializada** guarda, por artículo, la cantidad, la suma, el promedio y el máximo de sus precios (99.001 filas). Crearla tardó 0,6 s; consultarla, 2,6 ms, casi 200 veces menos que la mejor variante sobre la tabla. La contracara es que la vista no se actualiza sola: hay que ejecutar `REFRESH MATERIALIZED VIEW` cuando cambian los precios. Con el índice único sobre `id_articulo` se puede usar `REFRESH ... CONCURRENTLY` (0,76 s), que recalcula sin bloquear a quien esté leyendo la vista.

b.6. Para saber el rubro de cada precio hay que unir los 8 millones de precios con los 500.000 artículos. El plan es un `Parallel Hash Join` (la tabla hash de articulos se construye en paralelo entre los tres procesos), un `Partial HashAggregate` de sólo 31 grupos y la combinación final. Mediana: 636 ms. La tabla hash de los artículos no entra en `work_mem`: el plan muestra `Batches: 4` y lectura y escritura de archivos temporales. Con `work_mem = 64MB` entra entera (`Batches: 1`) y la consulta baja a 405 ms, un 36 % menos.

Se probó también agregar antes del join: primero suma y cantidad por artículo y después el join, que recibe 99.000 filas en lugar de 8 millones. No mejora (575 ms), porque agrupar en 99.000 grupos es más caro que agrupar en 31, y eso compensa lo que se ahorra en el join. Lo que sí vale de esa forma es cómo se calcula el promedio del rubro: tiene que ser `SUM(suma) / SUM(cantidad)`; promediar los promedios de cada artículo daría el mismo peso a un artículo con 40 precios que a uno con 120.

Sobre la vista materializada del b.5 la misma cuenta es un join de 99.001 filas precalculadas con articulos y tarda 16,2 ms, 39 veces menos que la consulta original.

Los promedios por rubro van de 2.593 a 2.605. Es lo esperable: los precios son uniformes entre 100 y 5.100, así que todos los promedios tienden a 2.600, el centro del intervalo. Los rubros 0 y 30 tienen la mitad de precios que los demás porque `round(random() * 30)` les asigna la mitad de probabilidad.

SCRIPT b04-promedios.sql

```
SET search_path TO public;

DELETE FROM medicion WHERE item IN ('b.5', 'b.6');
DROP MATERIALIZED VIEW IF EXISTS mv_precios_por_articulo;

-- b.5) Precio promedio por artículo. Hay que leer los 8 millones de precios y
--      formar unos 99.000 grupos. A esta altura existen los índices
--      (id_articulo, fecha DESC) INCLUDE (precio) del b.2 y (precio, id_articulo)
--      del b.4; el planificador usa el primero, que ya entrega los precios
--      ordenados por artículo.

EXPLAIN (ANALYZE, BUFFERS)
SELECT p.id_articulo, ROUND(AVG(p.precio), 2) AS precio_promedio
FROM precios p
GROUP BY p.id_articulo;

CALL pr_medir('b.5', 'Con índice cubriente (plan elegido)', $$
    SELECT p.id_articulo, ROUND(AVG(p.precio), 2) FROM precios p GROUP BY p.id_articulo
$$, 5);

-- Para comparar con la situación sin índices se deshabilitan los accesos por
-- índice en la sesión: el planificador queda con el Seq Scan.

SET enable_indexscan = off;
SET enable_indexonlyscan = off;
SET enable_bitmapscan = off;
```

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT p.id_articulo, ROUND(AVG(p.precio), 2) AS precio_promedio
FROM precios p
GROUP BY p.id_articulo;

CALL pr_medir('b.5', 'Sin índices, work_mem 4MB', $$
    SELECT p.id_articulo, ROUND(AVG(p.precio), 2) FROM precios p GROUP BY p.id_articulo
$$, 5);

-- El HashAggregate no entra en work_mem (4 MB por defecto) y escribe a disco
-- ("Disk Usage"). Con más memoria para la sesión se resuelve en memoria.

SET work_mem = '64MB';

EXPLAIN (ANALYZE, BUFFERS)
SELECT p.id_articulo, ROUND(AVG(p.precio), 2) AS precio_promedio
FROM precios p
GROUP BY p.id_articulo;

CALL pr_medir('b.5', 'Sin índices, work_mem 64MB', $$
    SELECT p.id_articulo, ROUND(AVG(p.precio), 2) FROM precios p GROUP BY p.id_articulo
$$, 5);

RESET work_mem;
RESET enable_indexscan;
RESET enable_indexonlyscan;
RESET enable_bitmapscan;

-- Las tres variantes leen todos los precios y tardan lo mismo en orden de
-- magnitud. Si el promedio se consulta seguido, lo que cambia el resultado es no
-- recalcularlo: una vista materializada guarda la suma, la cantidad y el
-- promedio de cada artículo, y se actualiza con REFRESH cuando cambian los precios.

\timing on
CREATE MATERIALIZED VIEW mv_precios_por_articulo AS
SELECT p.id_articulo,
       COUNT(*)                AS cantidad,
       SUM(p.precio)           AS suma,
       ROUND(AVG(p.precio), 2) AS promedio,
       MAX(p.precio)           AS maximo
FROM precios p
GROUP BY p.id_articulo;
\timing off

CREATE UNIQUE INDEX idx_mv_precios_articulo ON mv_precios_por_articulo (id_articulo);
VACUUM ANALYZE mv_precios_por_articulo;

EXPLAIN (ANALYZE, BUFFERS)
SELECT m.id_articulo, m.promedio AS precio_promedio
FROM mv_precios_por_articulo m;

CALL pr_medir('b.5', 'Vista materializada', $$
    SELECT m.id_articulo, m.promedio FROM mv_precios_por_articulo m
$$, 20);

-- REFRESH ... CONCURRENTLY (posible gracias al índice único) recalcula la vista
-- sin bloquear las lecturas.

\timing on
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_precios_por_articulo;
\timing off

SELECT COUNT(*) AS articulos, MIN(promedio) AS menor_promedio, MAX(promedio) AS mayor_promedio
FROM mv_precios_por_articulo;

SELECT m.id_articulo, m.cantidad, m.promedio AS precio_promedio
FROM mv_precios_por_articulo m
ORDER BY m.id_articulo
LIMIT 5;

-- b.6) Precio promedio por rubro. Hay que unir cada precio con su artículo para
--      saber el rubro: 8 millones de filas contra 500.000.

EXPLAIN (ANALYZE, BUFFERS)
SELECT a.rubro, ROUND(AVG(p.precio), 2) AS precio_promedio
FROM precios p
JOIN articulos a ON a.id = p.id_articulo
```

```
GROUP BY a.rubro
ORDER BY a.rubro;

CALL pr_medir('b.6', 'Join y AVG, work_mem 4MB', $$
    SELECT a.rubro, ROUND(AVG(p.precio), 2)
    FROM precios p JOIN articulos a ON a.id = p.id_articulo
    GROUP BY a.rubro ORDER BY a.rubro
$$, 5);

-- La tabla hash de articulos no entra en work_mem y el join se parte en lotes que
-- pasan por disco ("Batches" y "temp read/written"). Con 64 MB entra entera.

SET work_mem = '64MB';

EXPLAIN (ANALYZE, BUFFERS)
SELECT a.rubro, ROUND(AVG(p.precio), 2) AS precio_promedio
FROM precios p
JOIN articulos a ON a.id = p.id_articulo
GROUP BY a.rubro
ORDER BY a.rubro;

CALL pr_medir('b.6', 'Join y AVG, work_mem 64MB', $$
    SELECT a.rubro, ROUND(AVG(p.precio), 2)
    FROM precios p JOIN articulos a ON a.id = p.id_articulo
    GROUP BY a.rubro ORDER BY a.rubro
$$, 5);

RESET work_mem;

-- Reescritura agregando antes del join: primero suma y cantidad por artículo y
-- después el join, que recibe 99.000 filas en lugar de 8 millones. El promedio
-- del rubro tiene que ser SUM(suma) / SUM(cantidad); el promedio de los promedios
-- daría el mismo peso a artículos con distinta cantidad de precios.

EXPLAIN (ANALYZE, BUFFERS)
WITH por_articulo AS (
    SELECT p.id_articulo, SUM(p.precio) AS suma, COUNT(*) AS cantidad
    FROM precios p
    GROUP BY p.id_articulo
)
SELECT a.rubro, ROUND(SUM(pa.suma) / SUM(pa.cantidad), 2) AS precio_promedio
FROM por_articulo pa
JOIN articulos a ON a.id = pa.id_articulo
GROUP BY a.rubro
ORDER BY a.rubro;

CALL pr_medir('b.6', 'Agregar por artículo y después join', $$
    WITH por_articulo AS (
        SELECT p.id_articulo, SUM(p.precio) AS suma, COUNT(*) AS cantidad
        FROM precios p GROUP BY p.id_articulo
    )
    SELECT a.rubro, ROUND(SUM(pa.suma) / SUM(pa.cantidad), 2)
    FROM por_articulo pa JOIN articulos a ON a.id = pa.id_articulo
    GROUP BY a.rubro ORDER BY a.rubro
$$, 5);

-- Con la vista materializada del b.5 el join es de 99.000 filas precalculadas.

EXPLAIN (ANALYZE, BUFFERS)
SELECT a.rubro, ROUND(SUM(m.suma) / SUM(m.cantidad), 2) AS precio_promedio
FROM mv_precios_por_articulo m
JOIN articulos a ON a.id = m.id_articulo
GROUP BY a.rubro
ORDER BY a.rubro;

CALL pr_medir('b.6', 'Vista materializada', $$
    SELECT a.rubro, ROUND(SUM(m.suma) / SUM(m.cantidad), 2)
    FROM mv_precios_por_articulo m JOIN articulos a ON a.id = m.id_articulo
    GROUP BY a.rubro ORDER BY a.rubro
$$, 20);

SELECT a.rubro, ROUND(SUM(m.suma) / SUM(m.cantidad), 2) AS precio_promedio,
       SUM(m.cantidad) AS precios
FROM mv_precios_por_articulo m
JOIN articulos a ON a.id = m.id_articulo
GROUP BY a.rubro
ORDER BY a.rubro;
```

```
SELECT item, variante, mediana_ms, min_ms, max_ms, plan
FROM medicion
WHERE item IN ('b.5', 'b.6')
ORDER BY id;
```

SALIDA

```
                                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------------------------
 Finalize GroupAggregate  (cost=1000.46..276042.66 rows=97395 width=40)
     (actual time=3.582..648.164 rows=99001 loops=1)
   Group Key: id_articulo
   Buffers: shared hit=3950052 read=39408 written=16
   ->  Gather Merge  (cost=1000.46..273120.81 rows=194790 width=40)
         (actual time=3.557..610.078 rows=137691 loops=1)
         Workers Planned: 2
         Workers Launched: 2
         Buffers: shared hit=3950052 read=39408 written=16
         ->  Partial GroupAggregate  (cost=0.43..249637.18 rows=97395 width=40)
               (actual time=0.044..434.609 rows=45897 loops=3)
               Group Key: id_articulo
               Buffers: shared hit=3950052 read=39408 written=16
               ->  Parallel Index Only Scan using idx_precios_articulo_fecha on precios p
                     (cost=0.43..231753.21 rows=3333308 width=13) (actual time=0.032..336.028 rows=2666667 loops=3)
                     Heap Fetches: 0
                     Buffers: shared hit=3950052 read=39408 written=16
 Planning:
   Buffers: shared hit=105 read=7
 Planning Time: 0.390 ms
 Execution Time: 649.934 ms
(17 rows)

                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------------------------
 Finalize GroupAggregate  (cost=351199.86..376848.79 rows=97395 width=40)
     (actual time=605.702..686.768 rows=99001 loops=1)
   Group Key: id_articulo
   Buffers: shared hit=16 read=58824, temp read=28449 written=47726
   ->  Gather Merge  (cost=351199.86..373926.94 rows=194790 width=40)
         (actual time=605.697..628.865 rows=297003 loops=1)
         Workers Planned: 2
         Workers Launched: 2
         Buffers: shared hit=16 read=58824, temp read=28449 written=47726
         ->  Sort  (cost=350199.83..350443.32 rows=97395 width=40)
               (actual time=598.087..602.938 rows=99001 loops=3)
               Sort Key: id_articulo
               Sort Method: external merge  Disk: 8048kB
               Buffers: shared hit=16 read=58824, temp read=28449 written=47726
               Worker 0:  Sort Method: external merge  Disk: 8048kB
               Worker 1:  Sort Method: external merge  Disk: 8048kB
               ->  Partial HashAggregate  (cost=305697.12..339466.40 rows=97395 width=40)
                     (actual time=392.088..583.455 rows=99001 loops=3)
                     Group Key: id_articulo
                     Planned Partitions: 4  Batches: 5  Memory Usage: 8241kB  Disk Usage: 68896kB
                     Buffers: shared read=58824, temp read=25431 written=44702
                     Worker 0:  Batches: 5  Memory Usage: 8241kB  Disk Usage: 67368kB
                     Worker 1:  Batches: 5  Memory Usage: 8241kB  Disk Usage: 69432kB
                     ->  Parallel Seq Scan on precios p  (cost=0.00..92157.08 rows=3333308 width=13)
                           (actual time=0.014..67.358 rows=2666667 loops=3)
                           Buffers: shared read=58824
 Planning Time: 0.020 ms
 Execution Time: 690.293 ms
(23 rows)

                                                                QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------------------------
 Finalize HashAggregate  (cost=131980.98..133441.91 rows=97395 width=40)
     (actual time=615.225..637.004 rows=99001 loops=1)
   Group Key: id_articulo
   Batches: 1  Memory Usage: 51217kB
   Buffers: shared hit=672 read=58152
   ->  Gather  (cost=109823.62..130520.06 rows=194790 width=40) (actual time=527.207..548.984 rows=297003 loops=1)
         Workers Planned: 2
         Workers Launched: 2
         Buffers: shared hit=672 read=58152
         ->  Partial HashAggregate  (cost=108823.62..110041.06 rows=97395 width=40)
               (actual time=525.823..539.853 rows=99001 loops=3)
               Group Key: id_articulo
               Batches: 1  Memory Usage: 39953kB
               Buffers: shared hit=672 read=58152
               Worker 0:  Batches: 1  Memory Usage: 39953kB
```

```
               Worker 1:  Batches: 1  Memory Usage: 39953kB
               ->  Parallel Seq Scan on precios p  (cost=0.00..92157.08 rows=3333308 width=13)
                     (actual time=0.016..72.500 rows=2666667 loops=3)
                     Buffers: shared hit=672 read=58152
 Planning Time: 0.019 ms
 Execution Time: 639.038 ms
(18 rows)

Time: 597.476 ms
                                                                 QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------------------------
 Seq Scan on mv_precios_por_articulo m  (cost=0.00..1822.01 rows=99001 width=14)
     (actual time=0.002..2.680 rows=99001 loops=1)
   Buffers: shared hit=832
 Planning:
   Buffers: shared hit=9
 Planning Time: 0.018 ms
 Execution Time: 4.278 ms
(6 rows)

Time: 762.944 ms
 articulos | menor_promedio | mayor_promedio
-----------+----------------+----------------
     99001 |        1868.34 |        3278.32
(1 row)

 id_articulo | cantidad | precio_promedio
-------------+----------+-----------------
           1 |       38 |         2618.03
           2 |       74 |         2599.14
           3 |       77 |         2642.12
           4 |       91 |         2741.81
           5 |       69 |         2788.33
(5 rows)

                                                                                 QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------------------------
 Finalize GroupAggregate  (cost=166941.31..166949.47 rows=31 width=40)
     (actual time=696.337..699.952 rows=31 loops=1)
   Group Key: a.rubro
   Buffers: shared hit=23 read=67914, temp read=36821 written=36856
   ->  Gather Merge  (cost=166941.31..166948.54 rows=62 width=40) (actual time=696.333..699.929 rows=93 loops=1)
         Workers Planned: 2
         Workers Launched: 2
         Buffers: shared hit=23 read=67914, temp read=36821 written=36856
         ->  Sort  (cost=165941.28..165941.36 rows=31 width=40) (actual time=694.747..694.783 rows=31 loops=3)
               Sort Key: a.rubro
               Sort Method: quicksort  Memory: 27kB
               Buffers: shared hit=23 read=67914, temp read=36821 written=36856
               Worker 0:  Sort Method: quicksort  Memory: 27kB
               Worker 1:  Sort Method: quicksort  Memory: 27kB
               ->  Partial HashAggregate  (cost=165940.13..165940.51 rows=31 width=40)
                     (actual time=694.735..694.773 rows=31 loops=3)
                     Group Key: a.rubro
                     Batches: 1  Memory Usage: 32kB
                     Buffers: shared hit=7 read=67914, temp read=36821 written=36856
                     Worker 0:  Batches: 1  Memory Usage: 32kB
                     Worker 1:  Batches: 1  Memory Usage: 32kB
                     ->  Parallel Hash Join  (cost=14796.50..149273.59 rows=3333308 width=13)
                           (actual time=320.291..525.943 rows=2666667 loops=3)
                           Hash Cond: (p.id_articulo = a.id)
                           Buffers: shared hit=7 read=67914, temp read=36821 written=36856
                           ->  Parallel Seq Scan on precios p  (cost=0.00..92157.08 rows=3333308 width=13)
                                 (actual time=0.033..112.249 rows=2666667 loops=3)
                                 Buffers: shared read=58824
                           ->  Parallel Hash  (cost=11174.33..11174.33 rows=208333 width=16)
                                 (actual time=17.845..17.845 rows=166667 loops=3)
                                 Buckets: 262144  Batches: 4  Memory Usage: 7968kB
                                 Buffers: shared hit=1 read=9090, temp written=1668
                                 ->  Parallel Seq Scan on articulos a
                                       (cost=0.00..11174.33 rows=208333 width=16)
                                       (actual time=0.012..7.910 rows=166667 loops=3)
                                       Buffers: shared hit=1 read=9090
 Planning:
   Buffers: shared hit=46 read=12
 Planning Time: 0.427 ms
 Execution Time: 699.964 ms
(33 rows)

                                                                                 QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------------------------
 Finalize GroupAggregate  (cost=132353.31..132361.47 rows=31 width=40)
     (actual time=528.777..529.863 rows=31 loops=1)
```

```
   Group Key: a.rubro
   Buffers: shared hit=1367 read=66570
   ->  Gather Merge  (cost=132353.31..132360.54 rows=62 width=40) (actual time=528.773..529.841 rows=93 loops=1)
         Workers Planned: 2
         Workers Launched: 2
         Buffers: shared hit=1367 read=66570
         ->  Sort  (cost=131353.28..131353.36 rows=31 width=40) (actual time=527.342..527.343 rows=31 loops=3)
               Sort Key: a.rubro
               Sort Method: quicksort  Memory: 27kB
               Buffers: shared hit=1367 read=66570
               Worker 0:  Sort Method: quicksort  Memory: 27kB
               Worker 1:  Sort Method: quicksort  Memory: 27kB
               ->  Partial HashAggregate  (cost=131352.13..131352.51 rows=31 width=40)
                     (actual time=527.329..527.332 rows=31 loops=3)
                     Group Key: a.rubro
                     Batches: 1  Memory Usage: 32kB
                     Buffers: shared hit=1351 read=66570
                     Worker 0:  Batches: 1  Memory Usage: 32kB
                     Worker 1:  Batches: 1  Memory Usage: 32kB
                     ->  Parallel Hash Join  (cost=13778.50..114685.59 rows=3333308 width=13)
                           (actual time=18.499..355.398 rows=2666667 loops=3)
                           Hash Cond: (p.id_articulo = a.id)
                           Buffers: shared hit=1351 read=66570
                           ->  Parallel Seq Scan on precios p  (cost=0.00..92157.08 rows=3333308 width=13)
                                 (actual time=0.035..77.682 rows=2666667 loops=3)
                                 Buffers: shared hit=672 read=58152
                           ->  Parallel Hash  (cost=11174.33..11174.33 rows=208333 width=16)
                                 (actual time=18.331..18.331 rows=166667 loops=3)
                                 Buckets: 524288  Batches: 1  Memory Usage: 27616kB
                                 Buffers: shared hit=673 read=8418
                                 ->  Parallel Seq Scan on articulos a
                                       (cost=0.00..11174.33 rows=208333 width=16)
                                       (actual time=0.020..8.570 rows=166667 loops=3)
                                       Buffers: shared hit=673 read=8418
 Planning:
   Buffers: shared hit=12
 Planning Time: 0.055 ms
 Execution Time: 529.873 ms
(33 rows)

                                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=309544.16..309544.24 rows=31 width=40) (actual time=663.249..663.263 rows=31 loops=1)
   Sort Key: a.rubro
   Sort Method: quicksort  Memory: 25kB
   Buffers: shared hit=3950055 read=48497 written=636, temp read=1365 written=1365
   ->  HashAggregate  (cost=309542.78..309543.40 rows=31 width=40) (actual time=663.241..663.260 rows=31 loops=1)
         Group Key: a.rubro
         Batches: 1  Memory Usage: 32kB
         Buffers: shared hit=3950055 read=48497 written=636, temp read=1365 written=1365
         ->  Hash Join  (cost=287667.80..308812.31 rows=97395 width=48)
               (actual time=600.697..657.594 rows=99001 loops=1)
               Hash Cond: (a.id = pa.id_articulo)
               Buffers: shared hit=3950055 read=48497 written=636, temp read=1365 written=1365
               ->  Seq Scan on articulos a  (cost=0.00..14091.00 rows=500000 width=16)
                     (actual time=0.005..20.776 rows=500000 loops=1)
                     Buffers: shared hit=1 read=9090
               ->  Hash  (cost=285593.36..285593.36 rows=97395 width=48)
                     (actual time=600.656..600.670 rows=99001 loops=1)
                     Buckets: 131072  Batches: 2  Memory Usage: 3729kB
                     Buffers: shared hit=3950054 read=39407 written=636, temp written=266
                     ->  Subquery Scan on pa  (cost=1000.46..285593.36 rows=97395 width=48)
                           (actual time=3.454..591.361 rows=99001 loops=1)
                           Buffers: shared hit=3950054 read=39407 written=636
                           ->  Finalize GroupAggregate  (cost=1000.46..284619.41 rows=97395 width=48)
                                 (actual time=3.454..587.779 rows=99001 loops=1)
                                 Group Key: p.id_articulo
                                 Buffers: shared hit=3950054 read=39407 written=636
                                 ->  Gather Merge  (cost=1000.46..281454.08 rows=194790 width=48)
                                       (actual time=3.440..563.766 rows=136776 loops=1)
                                       Workers Planned: 2
                                       Workers Launched: 2
                                       Buffers: shared hit=3950054 read=39407 written=636
                                       ->  Partial GroupAggregate
                                             (cost=0.43..257970.45 rows=97395 width=48)
                                             (actual time=0.041..401.762 rows=45592 loops=3)
                                             Group Key: p.id_articulo
                                             Buffers: shared hit=3950054 read=39407 written=636
                                             ->  Parallel Index Only Scan using idx_precios_articulo_fecha on
                                                   precios p  (cost=0.43..231753.21 rows=3333308 width=13)
                                                   (actual time=0.027..291.322 rows=2666667 loops=3)
                                                   Heap Fetches: 0
                                                   Buffers: shared hit=3950054 read=39407 written=636
```

```
 Planning:
   Buffers: shared hit=3 read=3
 Planning Time: 0.064 ms
 Execution Time: 663.281 ms
(35 rows)

                                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Finalize GroupAggregate  (cost=14611.30..14619.93 rows=31 width=40) (actual time=20.641..21.511 rows=31 loops=1)
   Group Key: a.rubro
   Buffers: shared hit=96150 read=3272
   ->  Gather Merge  (cost=14611.30..14618.53 rows=62 width=72) (actual time=20.636..21.476 rows=93 loops=1)
         Workers Planned: 2
         Workers Launched: 2
         Buffers: shared hit=96150 read=3272
         ->  Sort  (cost=13611.28..13611.35 rows=31 width=72) (actual time=18.979..18.980 rows=31 loops=3)
               Sort Key: a.rubro
               Sort Method: quicksort  Memory: 28kB
               Buffers: shared hit=96150 read=3272
               Worker 0:  Sort Method: quicksort  Memory: 28kB
               Worker 1:  Sort Method: quicksort  Memory: 28kB
               ->  Partial HashAggregate  (cost=13610.04..13610.51 rows=31 width=72)
                     (actual time=18.967..18.971 rows=31 loops=3)
                     Group Key: a.rubro
                     Batches: 1  Memory Usage: 32kB
                     Buffers: shared hit=96140 read=3272
                     Worker 0:  Batches: 1  Memory Usage: 32kB
                     Worker 1:  Batches: 1  Memory Usage: 32kB
                     ->  Merge Join  (cost=1.20..13300.67 rows=41250 width=22)
                           (actual time=0.645..16.836 rows=33000 loops=3)
                           Merge Cond: (a.id = m.id_articulo)
                           Buffers: shared hit=96140 read=3272
                           ->  Parallel Index Scan using articulos_pkey on articulos a
                                 (cost=0.42..46426.68 rows=208333 width=16)
                                 (actual time=0.028..5.836 rows=33001 loops=3)
                                 Buffers: shared hit=93980 read=2136
                           ->  Index Scan using idx_mv_precios_articulo on mv_precios_por_articulo m
                                 (cost=0.29..3412.31 rows=99001 width=22)
                                 (actual time=0.014..6.488 rows=99001 loops=3)
                                 Buffers: shared hit=2160 read=1136
 Planning:
   Buffers: shared hit=9 read=10
 Planning Time: 0.126 ms
 Execution Time: 21.523 ms
(30 rows)

 rubro | precio_promedio | precios
-------+-----------------+---------
     0 |         2604.89 |  130540
     1 |         2603.03 |  275432
     2 |         2596.39 |  263590
     3 |         2594.91 |  263433
     4 |         2603.29 |  266826
     5 |         2594.99 |  261471
     6 |         2603.61 |  261990
     7 |         2596.06 |  265310
     8 |         2600.29 |  269530
     9 |         2597.05 |  263864
    10 |         2594.13 |  269596
    11 |         2601.88 |  259879
    12 |         2601.56 |  269980
    13 |         2601.55 |  270576
    14 |         2599.26 |  270225
    15 |         2600.66 |  268840
    16 |         2599.66 |  267139
    17 |         2600.31 |  271346
    18 |         2602.24 |  268442
    19 |         2601.55 |  265300
    20 |         2593.43 |  265321
    21 |         2599.31 |  268811
    22 |         2600.35 |  267351
    23 |         2601.03 |  266212
    24 |         2601.31 |  266515
    25 |         2598.61 |  271574
    26 |         2600.30 |  263505
    27 |         2597.58 |  258022
    28 |         2603.84 |  272648
    29 |         2598.75 |  266615
    30 |         2594.08 |  130117
(31 rows)
```

ítem	variante	mediana ms	mín ms	máx ms	plan
b.5	Con índice cubriente (plan elegido)	504.674	496.766	514.533	Finalize GroupAggregate [Gather Merge [Partial GroupAggregate [Parallel Index Only Scan (idx_precios_articulo_fecha)]]]
b.5	Sin índices, work_mem 4MB	662.984	653.090	682.131	Finalize GroupAggregate [Gather Merge [Sort [Partial HashAggregate [Parallel Seq Scan]]]]
b.5	Sin índices, work_mem 64MB	588.246	586.430	604.406	Finalize HashAggregate [Gather [Partial HashAggregate [Parallel Seq Scan]]]
b.5	Vista materializada	2.594	2.194	2.679	Seq Scan
b.6	Join y AVG, work_mem 4MB	635.611	629.285	646.969	Finalize GroupAggregate [Gather Merge [Sort [Partial HashAggregate [Parallel Hash Join [Parallel Seq Scan, Parallel Hash [Parallel Seq Scan]]]]]]
b.6	Join y AVG, work_mem 64MB	404.728	400.053	414.246	Finalize GroupAggregate [Gather Merge [Sort [Partial HashAggregate [Parallel Hash Join [Parallel Seq Scan, Parallel Hash [Parallel Seq Scan]]]]]]
b.6	Agregar por artículo y después join	575.209	561.344	592.161	Sort [HashAggregate [Hash Join [Seq Scan, Hash [Subquery Scan [Finalize GroupAggregate [Gather Merge [Partial GroupAggregate [Parallel Index Only Scan (idx_precios_articulo_fecha)]]]]]]]]
b.6	Vista materializada	16.182	15.684	17.291	Finalize GroupAggregate [Gather Merge [Sort [Partial HashAggregate [Merge Join [Parallel Index Scan (articulos_pkey), Index Scan (idx_mv_precios_articulo)]]]]]

b.7 Máximo precio del rubro 15 posterior a marzo de 2023

"Posterior a marzo de 2023" se tomó como fecha desde el 1 de abril de 2023.

Sin índices (deshabilitados en la sesión). `Parallel Hash Join`: se construye la tabla hash con los 16.863 artículos del rubro 15 (un `Parallel Seq Scan` de articulos) y se recorren en paralelo los precios desde abril de 2023 (4,4 millones), buscando cada uno en la tabla hash. Sobre las 146.000 filas que cruzan, cada proceso calcula su máximo (`Partial Aggregate`) y el `Finalize Aggregate` se queda con el mayor. Mediana: 134 ms.

Con los índices existentes. El plan cambia a `Nested Loop`: los artículos del rubro 15 salen del índice sobre rubro (por el CLUSTER del b.1 están en páginas contiguas), y para cada uno el índice (id_articulo, fecha DESC) INCLUDE (precio) da directamente sus precios desde abril de 2023 con un `Index Only Scan` (16.863 búsquedas, unos 9 precios cada una). Mediana: 12,7 ms, diez veces menos.

Reescritura con ORDER BY ... LIMIT 1. El máximo es el primer precio en orden descendente que cumpla las condiciones. Escrito así, el planificador recorre el índice (precio, id_articulo) del b.4 desde el final (`Index Scan Backward`), es decir, de los precios más altos hacia abajo, y para cada uno verifica la fecha y busca el artículo por clave primaria. El `Limit` corta el `Nested Loop` en la primera fila que cumple: leyó 79 entradas del índice, descartó 38 por fecha, buscó 41 artículos y el último era del rubro 15. Mediana: 0,050 ms, 2.700 veces menos que sin índices. Se agregó `precio IS NOT NULL` porque `MAX` ignora los NULL y, en orden descendente, PostgreSQL los pone primero.

Esta reescritura depende de los datos: es rápida porque el precio máximo posible (5.100) tiene 16 filas que cumplen las condiciones y aparecen enseguida. Si el rubro 15 tuviera sólo precios bajos, el recorrido descendente tendría que atravesar gran parte del índice antes de encontrar uno, y el plan con `MAX` sería mejor. El planificador tiene esto en cuenta en su estimación, que acá acertó.

SCRIPT b05-maximo-rubro-15.sql

```
SET search_path TO public;

DELETE FROM medicion WHERE item = 'b.7';

-- b.7) Máximo precio del rubro 15 posterior a marzo de 2023, es decir, con fecha
--      desde el 1 de abril de 2023.

-- Sin índices (se deshabilitan en la sesión): Hash Join entre los artículos del
-- rubro 15 y los precios posteriores a marzo, los dos leídos completos.

SET enable_indexscan = off;
SET enable_indexonlyscan = off;
SET enable_bitmapscan = off;

EXPLAIN (ANALYZE, BUFFERS)
SELECT MAX(p.precio) AS precio_maximo
FROM precios p
JOIN articulos a ON a.id = p.id_articulo
WHERE a.rubro = 15
  AND p.fecha >= DATE '2023-04-01';

CALL pr_medir('b.7', 'Sin índices', $$
    SELECT MAX(p.precio) FROM precios p JOIN articulos a ON a.id = p.id_articulo
    WHERE a.rubro = 15 AND p.fecha >= DATE '2023-04-01'
$$, 5);

RESET enable_indexscan;
RESET enable_indexonlyscan;
RESET enable_bitmapscan;

-- Con los índices creados hasta acá: los artículos del rubro 15 salen de
-- idx_articulos_rubro (y por el CLUSTER del b.1 están juntos), y para cada uno el
-- índice (id_articulo, fecha DESC) INCLUDE (precio) da directamente sus precios
-- desde abril de 2023, sin ir a la tabla.

EXPLAIN (ANALYZE, BUFFERS)
SELECT MAX(p.precio) AS precio_maximo
FROM precios p
JOIN articulos a ON a.id = p.id_articulo
WHERE a.rubro = 15
  AND p.fecha >= DATE '2023-04-01';

CALL pr_medir('b.7', 'Con índices: Nested Loop', $$
```

```
    SELECT MAX(p.precio) FROM precios p JOIN articulos a ON a.id = p.id_articulo
    WHERE a.rubro = 15 AND p.fecha >= DATE '2023-04-01'
$$, 20);

-- Reescritura: el máximo es el primer precio en orden descendente que cumpla las
-- condiciones. Recorriendo idx_precios_precio_articulo desde el final, el Nested
-- Loop puede cortar en la primera fila que sea del rubro 15 y posterior a marzo.
-- La condición IS NOT NULL mantiene la semántica de MAX, que ignora los NULL (en
-- orden descendente los NULL van primero).

EXPLAIN (ANALYZE, BUFFERS)
SELECT p.precio AS precio_maximo
FROM precios p
JOIN articulos a ON a.id = p.id_articulo
WHERE a.rubro = 15
  AND p.fecha >= DATE '2023-04-01'
  AND p.precio IS NOT NULL
ORDER BY p.precio DESC
LIMIT 1;

CALL pr_medir('b.7', 'ORDER BY precio DESC LIMIT 1', $$
    SELECT p.precio FROM precios p JOIN articulos a ON a.id = p.id_articulo
    WHERE a.rubro = 15 AND p.fecha >= DATE '2023-04-01' AND p.precio IS NOT NULL
    ORDER BY p.precio DESC LIMIT 1
$$, 20);

-- Resultado, con el detalle de cuántos precios alcanzan ese máximo.

SELECT MAX(p.precio) AS precio_maximo
FROM precios p
JOIN articulos a ON a.id = p.id_articulo
WHERE a.rubro = 15
  AND p.fecha >= DATE '2023-04-01';

SELECT COUNT(*) AS precios_con_el_maximo, COUNT(DISTINCT p.id_articulo) AS articulos
FROM precios p
JOIN articulos a ON a.id = p.id_articulo
WHERE a.rubro = 15
  AND p.fecha >= DATE '2023-04-01'
  AND p.precio = 5100;

SELECT item, variante, mediana_ms, min_ms, max_ms, plan
FROM medicion
WHERE item = 'b.7'
ORDER BY id;
```

SALIDA

```
                                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------------------------
 Finalize Aggregate  (cost=118245.37..118245.38 rows=1 width=32) (actual time=163.373..164.138 rows=1 loops=1)
   Buffers: shared hit=2026 read=66055
   ->  Gather  (cost=118245.15..118245.36 rows=2 width=32) (actual time=163.288..164.135 rows=3 loops=1)
         Workers Planned: 2
         Workers Launched: 2
         Buffers: shared hit=2026 read=66055
         ->  Partial Aggregate  (cost=117245.15..117245.16 rows=1 width=32)
               (actual time=162.368..162.368 rows=1 loops=3)
               Buffers: shared hit=2026 read=66055
               ->  Parallel Hash Join  (cost=11785.62..117086.04 rows=63646 width=5)
                     (actual time=6.988..160.441 rows=48808 loops=3)
                     Hash Cond: (p.id_articulo = a.id)
                     Buffers: shared hit=2026 read=66055
                     ->  Parallel Seq Scan on precios p  (cost=0.00..100490.35 rows=1832391 width=13)
                           (actual time=0.024..101.897 rows=1453821 loops=3)
                           Filter: (fecha >= '2023-04-01'::date)
                           Rows Removed by Filter: 1212846
                           Buffers: shared read=58824
                     ->  Parallel Hash  (cost=11695.17..11695.17 rows=7236 width=8)
                           (actual time=6.897..6.897 rows=5621 loops=3)
                           Buckets: 32768  Batches: 1  Memory Usage: 960kB
                           Buffers: shared hit=1860 read=7231
                           ->  Parallel Seq Scan on articulos a  (cost=0.00..11695.17 rows=7236 width=8)
                                 (actual time=3.174..6.530 rows=5621 loops=3)
                                 Filter: (rubro = 15)
                                 Rows Removed by Filter: 161046
                                 Buffers: shared hit=1860 read=7231
 Planning:
   Buffers: shared hit=225 read=21 dirtied=1
```

```
 Planning Time: 0.583 ms
 Execution Time: 164.156 ms
(26 rows)

                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Finalize Aggregate  (cost=47271.78..47271.79 rows=1 width=32) (actual time=17.299..18.268 rows=1 loops=1)
   Buffers: shared hit=120513 read=3577
   ->  Gather  (cost=47271.56..47271.77 rows=2 width=32) (actual time=17.255..18.266 rows=3 loops=1)
         Workers Planned: 2
         Workers Launched: 2
         Buffers: shared hit=120513 read=3577
         ->  Partial Aggregate  (cost=46271.56..46271.57 rows=1 width=32)
               (actual time=15.991..15.992 rows=1 loops=3)
               Buffers: shared hit=120513 read=3577
               ->  Nested Loop  (cost=195.45..46112.45 rows=63646 width=5)
                     (actual time=0.286..14.679 rows=48808 loops=3)
                     Buffers: shared hit=120513 read=3577
                     ->  Parallel Bitmap Heap Scan on articulos a
                           (cost=195.02..9475.10 rows=7236 width=8) (actual time=0.257..0.670 rows=5621 loops=3)
                           Recheck Cond: (rubro = 15)
                           Heap Blocks: exact=104
                           Buffers: shared hit=65 read=263
                           ->  Bitmap Index Scan on idx_articulos_rubro
                                 (cost=0.00..190.68 rows=17367 width=0)
                                 (actual time=0.708..0.708 rows=16863 loops=1)
                                 Index Cond: (rubro = 15)
                                 Buffers: shared hit=3 read=17
                     ->  Index Only Scan using idx_precios_articulo_fecha on precios p
                           (cost=0.43..4.61 rows=45 width=13) (actual time=0.001..0.002 rows=9 loops=16863)
                           Index Cond: ((id_articulo = a.id) AND (fecha >= '2023-04-01'::date))
                           Heap Fetches: 0
                           Buffers: shared hit=120448 read=3314
 Planning:
   Buffers: shared hit=12
 Planning Time: 0.061 ms
 Execution Time: 18.279 ms
(25 rows)

                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=0.86..7.50 rows=1 width=5) (actual time=0.418..0.418 rows=1 loops=1)
   Buffers: shared hit=165 read=81
   ->  Nested Loop  (cost=0.86..1014252.77 rows=152751 width=5) (actual time=0.417..0.418 rows=1 loops=1)
         Buffers: shared hit=165 read=81
         ->  Index Scan Backward using idx_precios_precio_articulo on precios p
               (cost=0.43..518517.45 rows=4397739 width=13) (actual time=0.015..0.152 rows=41 loops=1)
               Index Cond: (precio IS NOT NULL)
               Filter: (fecha >= '2023-04-01'::date)
               Rows Removed by Filter: 38
               Buffers: shared hit=1 read=81
         ->  Memoize  (cost=0.43..0.46 rows=1 width=8) (actual time=0.006..0.006 rows=0 loops=41)
               Cache Key: p.id_articulo
               Cache Mode: logical
               Hits: 0  Misses: 41  Evictions: 0  Overflows: 0  Memory Usage: 3kB
               Buffers: shared hit=164
               ->  Index Scan using articulos_pkey on articulos a  (cost=0.42..0.45 rows=1 width=8)
                     (actual time=0.001..0.001 rows=0 loops=41)
                     Index Cond: (id = p.id_articulo)
                     Filter: (rubro = 15)
                     Rows Removed by Filter: 1
                     Buffers: shared hit=164
 Planning:
   Buffers: shared hit=16
 Planning Time: 0.059 ms
 Execution Time: 0.426 ms
(23 rows)

 precio_maximo
---------------
       5100.00
(1 row)

 precios_con_el_maximo | articulos
-----------------------+-----------
                    16 |        16
(1 row)
```

ítem	variante	mediana ms	mín ms	máx ms	plan
b.7	Sin índices	133.621	131.375	136.441	Finalize Aggregate [Gather [Partial Aggregate [Parallel Hash Join [Parallel Seq Scan, Parallel Hash [Parallel Seq Scan]]]]]

ítem	variante	mediana ms	mín ms	máx ms	plan
b.7	Con índices: Nested Loop	12.737	12.454	13.411	Finalize Aggregate [Gather [Partial Aggregate [Nested Loop [Parallel Bitmap Heap Scan [Bitmap Index Scan (idx_articulos_rubro)], Index Only Scan (idx_precios_articulo_fecha)]]]]
b.7	ORDER BY precio DESC LIMIT 1	0.050	0.050	0.052	Limit [Nested Loop [Index Scan Backward (idx_precios_precio_articulo), Memoize [Index Scan (articulos_pkey)]]]

b.8 y b.9 Máximo precio promedio por rubro y por artículo

Las dos consultas son las del b.6 y el b.5 con un orden descendente y la primera fila. Se usó `FETCH FIRST 1 ROW WITH TIES` en lugar de `LIMIT 1` para que, si hay empate en el primer lugar, salgan todos los empatados y no uno elegido al azar.

b.8. Sobre las tablas, el plan es el del b.6 con un `Sort` y un `Limit` encima, que ordenan 31 filas: el tiempo es el del b.6 (642 ms con 4 MB de `work_mem`, 414 ms con 64 MB). Sobre la vista materializada, 17,3 ms. El rubro con mayor promedio es el 0, con $ 2.604,89, aunque, como se vio en el b.6, la diferencia con los demás rubros es de pocos pesos y se debe al azar de la generación.

b.9. Sobre la tabla precios, `Parallel Index Only Scan` y `GroupAggregate` como en el b.5, más un top-N sobre los 99.001 promedios: 508 ms. Sobre la vista materializada son 99.001 filas en lugar de 8 millones: `Seq Scan` y `Sort` en 14,6 ms. Con un índice sobre el promedio en la vista, el primero del índice en orden descendente es la respuesta y el plan es un `Index Scan` con `Limit`: 0,003 ms. El artículo con mayor promedio es el 93.411, con $ 3.278,32 sobre 62 precios.

Para estas consultas de resumen, que leen toda la tabla, los índices y la memoria dan mejoras moderadas; lo que da órdenes de magnitud es precalcular, siempre que se pueda tolerar que el resultado refleje el último `REFRESH`.

SCRIPT b06-maximos-promedios.sql

```
SET search_path TO public;

DELETE FROM medicion WHERE item IN ('b.8', 'b.9');
DROP INDEX IF EXISTS idx_mv_precios_promedio;

-- b.8) Máximo precio promedio por rubro: el rubro cuyo precio promedio es el más
--      alto. Es la consulta del b.6 más un orden descendente; FETCH FIRST 1 ROW
--      WITH TIES devuelve todos los rubros empatados en el primer lugar, si los hay.

EXPLAIN (ANALYZE, BUFFERS)
SELECT a.rubro, ROUND(AVG(p.precio), 2) AS precio_promedio
FROM precios p
JOIN articulos a ON a.id = p.id_articulo
GROUP BY a.rubro
ORDER BY precio_promedio DESC
FETCH FIRST 1 ROW WITH TIES;

CALL pr_medir('b.8', 'Sobre las tablas, work_mem 4MB', $$
    SELECT a.rubro, ROUND(AVG(p.precio), 2) AS precio_promedio
    FROM precios p JOIN articulos a ON a.id = p.id_articulo
    GROUP BY a.rubro ORDER BY precio_promedio DESC FETCH FIRST 1 ROW WITH TIES
$$, 5);

SET work_mem = '64MB';
CALL pr_medir('b.8', 'Sobre las tablas, work_mem 64MB', $$
    SELECT a.rubro, ROUND(AVG(p.precio), 2) AS precio_promedio
    FROM precios p JOIN articulos a ON a.id = p.id_articulo
    GROUP BY a.rubro ORDER BY precio_promedio DESC FETCH FIRST 1 ROW WITH TIES
$$, 5);
RESET work_mem;

EXPLAIN (ANALYZE, BUFFERS)
SELECT a.rubro, ROUND(SUM(m.suma) / SUM(m.cantidad), 2) AS precio_promedio
FROM mv_precios_por_articulo m
JOIN articulos a ON a.id = m.id_articulo
GROUP BY a.rubro
ORDER BY precio_promedio DESC
FETCH FIRST 1 ROW WITH TIES;

CALL pr_medir('b.8', 'Vista materializada', $$
    SELECT a.rubro, ROUND(SUM(m.suma) / SUM(m.cantidad), 2) AS precio_promedio
    FROM mv_precios_por_articulo m JOIN articulos a ON a.id = m.id_articulo
    GROUP BY a.rubro ORDER BY precio_promedio DESC FETCH FIRST 1 ROW WITH TIES
$$, 20);

SELECT a.rubro, ROUND(SUM(m.suma) / SUM(m.cantidad), 2) AS precio_promedio
FROM mv_precios_por_articulo m
JOIN articulos a ON a.id = m.id_articulo
GROUP BY a.rubro
ORDER BY precio_promedio DESC
FETCH FIRST 1 ROW WITH TIES;
```

```
-- b.9) Máximo precio promedio por artículo: el artículo con el promedio más alto.

EXPLAIN (ANALYZE, BUFFERS)
SELECT p.id_articulo, ROUND(AVG(p.precio), 2) AS precio_promedio
FROM precios p
GROUP BY p.id_articulo
ORDER BY precio_promedio DESC
FETCH FIRST 1 ROW WITH TIES;

CALL pr_medir('b.9', 'Sobre la tabla precios', $$
    SELECT p.id_articulo, ROUND(AVG(p.precio), 2) AS precio_promedio
    FROM precios p GROUP BY p.id_articulo
    ORDER BY precio_promedio DESC FETCH FIRST 1 ROW WITH TIES
$$, 5);

-- Sobre la vista materializada son 99.000 filas en lugar de 8 millones.

EXPLAIN (ANALYZE, BUFFERS)
SELECT m.id_articulo, m.promedio AS precio_promedio
FROM mv_precios_por_articulo m
ORDER BY m.promedio DESC
FETCH FIRST 1 ROW WITH TIES;

CALL pr_medir('b.9', 'Vista materializada', $$
    SELECT m.id_articulo, m.promedio FROM mv_precios_por_articulo m
    ORDER BY m.promedio DESC FETCH FIRST 1 ROW WITH TIES
$$, 20);

-- Un índice sobre el promedio en la vista evita incluso ordenar: el primero del
-- índice en orden descendente es la respuesta.

CREATE INDEX idx_mv_precios_promedio ON mv_precios_por_articulo (promedio DESC);
ANALYZE mv_precios_por_articulo;

EXPLAIN (ANALYZE, BUFFERS)
SELECT m.id_articulo, m.promedio AS precio_promedio
FROM mv_precios_por_articulo m
ORDER BY m.promedio DESC
FETCH FIRST 1 ROW WITH TIES;

CALL pr_medir('b.9', 'Vista materializada + índice', $$
    SELECT m.id_articulo, m.promedio FROM mv_precios_por_articulo m
    ORDER BY m.promedio DESC FETCH FIRST 1 ROW WITH TIES
$$, 20);

SELECT m.id_articulo, m.cantidad, m.promedio AS precio_promedio,
       LEFT(a.descripcion, 30) AS descripcion, a.rubro
FROM mv_precios_por_articulo m
JOIN articulos a ON a.id = m.id_articulo
ORDER BY m.promedio DESC
FETCH FIRST 1 ROW WITH TIES;

SELECT item, variante, mediana_ms, min_ms, max_ms, plan
FROM medicion
WHERE item IN ('b.8', 'b.9')
ORDER BY id;
```

SALIDA

```
                                                                                                                  QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=166949.62..166949.63 rows=1 width=40) (actual time=706.397..712.304 rows=1 loops=1)
   Buffers: shared hit=4272 read=63668, temp read=36824 written=36852
   ->  Sort  (cost=166949.62..166949.70 rows=31 width=40) (actual time=706.396..712.303 rows=2 loops=1)
         Sort Key: (round(avg(p.precio), 2)) DESC
         Sort Method: quicksort  Memory: 25kB
         Buffers: shared hit=4272 read=63668, temp read=36824 written=36852
         ->  Finalize GroupAggregate  (cost=166941.31..166949.47 rows=31 width=40)
               (actual time=706.359..712.295 rows=31 loops=1)
               Group Key: a.rubro
               Buffers: shared hit=4269 read=63668, temp read=36824 written=36852
               ->  Gather Merge  (cost=166941.31..166948.54 rows=62 width=40)
                     (actual time=706.354..712.268 rows=93 loops=1)
                     Workers Planned: 2
                     Workers Launched: 2
                     Buffers: shared hit=4269 read=63668, temp read=36824 written=36852
                     ->  Sort  (cost=165941.28..165941.36 rows=31 width=40)
```

```
                          (actual time=705.308..705.345 rows=31 loops=3)
                          Sort Key: a.rubro
                          Sort Method: quicksort  Memory: 27kB
                          Buffers: shared hit=4269 read=63668, temp read=36824 written=36852
                          Worker 0:  Sort Method: quicksort  Memory: 27kB
                          Worker 1:  Sort Method: quicksort  Memory: 27kB
                          ->  Partial HashAggregate  (cost=165940.13..165940.51 rows=31 width=40)
                                (actual time=705.297..705.335 rows=31 loops=3)
                                Group Key: a.rubro
                                Batches: 1  Memory Usage: 32kB
                                Buffers: shared hit=4253 read=63668, temp read=36824 written=36852
                                Worker 0:  Batches: 1  Memory Usage: 32kB
                                Worker 1:  Batches: 1  Memory Usage: 32kB
                                ->  Parallel Hash Join  (cost=14796.50..149273.59 rows=3333308 width=13)
                                      (actual time=317.956..531.687 rows=2666667 loops=3)
                                      Hash Cond: (p.id_articulo = a.id)
                                      Buffers: shared hit=4253 read=63668, temp read=36824 written=36852
                                      ->  Parallel Seq Scan on precios p
                                            (cost=0.00..92157.08 rows=3333308 width=13)
                                            (actual time=0.026..108.639 rows=2666667 loops=3)
                                            Buffers: shared hit=1469 read=57355
                                      ->  Parallel Hash  (cost=11174.33..11174.33 rows=208333 width=16)
                                            (actual time=18.720..18.720 rows=166667 loops=3)
                                            Buckets: 262144  Batches: 4  Memory Usage: 7968kB
                                            Buffers: shared hit=2778 read=6313, temp written=1660
                                            ->  Parallel Seq Scan on articulos a
                                                  (cost=0.00..11174.33 rows=208333 width=16)
                                                  (actual time=0.003..8.179 rows=166667 loops=3)
                                                  Buffers: shared hit=2778 read=6313
 Planning:
   Buffers: shared hit=167 read=1
 Planning Time: 0.248 ms
 Execution Time: 712.344 ms
(39 rows)

                                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=14620.08..14620.09 rows=1 width=40) (actual time=20.048..20.827 rows=1 loops=1)
   Buffers: shared hit=99422
   ->  Sort  (cost=14620.08..14620.16 rows=31 width=40) (actual time=20.048..20.826 rows=2 loops=1)
         Sort Key: (round((sum(m.suma) / sum(m.cantidad)), 2)) DESC
         Sort Method: quicksort  Memory: 25kB
         Buffers: shared hit=99422
         ->  Finalize GroupAggregate  (cost=14611.30..14619.93 rows=31 width=40)
               (actual time=20.006..20.821 rows=31 loops=1)
               Group Key: a.rubro
               Buffers: shared hit=99422
               ->  Gather Merge  (cost=14611.30..14618.53 rows=62 width=72)
                     (actual time=20.001..20.786 rows=93 loops=1)
                     Workers Planned: 2
                     Workers Launched: 2
                     Buffers: shared hit=99422
                     ->  Sort  (cost=13611.28..13611.35 rows=31 width=72)
                           (actual time=18.669..18.670 rows=31 loops=3)
                           Sort Key: a.rubro
                           Sort Method: quicksort  Memory: 28kB
                           Buffers: shared hit=99422
                           Worker 0:  Sort Method: quicksort  Memory: 28kB
                           Worker 1:  Sort Method: quicksort  Memory: 28kB
                           ->  Partial HashAggregate  (cost=13610.04..13610.51 rows=31 width=72)
                                 (actual time=18.657..18.661 rows=31 loops=3)
                                 Group Key: a.rubro
                                 Batches: 1  Memory Usage: 32kB
                                 Buffers: shared hit=99412
                                 Worker 0:  Batches: 1  Memory Usage: 32kB
                                 Worker 1:  Batches: 1  Memory Usage: 32kB
                                 ->  Merge Join  (cost=1.20..13300.67 rows=41250 width=22)
                                       (actual time=0.294..16.314 rows=33000 loops=3)
                                       Merge Cond: (a.id = m.id_articulo)
                                       Buffers: shared hit=99412
                                       ->  Parallel Index Scan using articulos_pkey on articulos a
                                             (cost=0.42..46426.68 rows=208333 width=16)
                                             (actual time=0.004..5.506 rows=33001 loops=3)
                                             Buffers: shared hit=96116
                                       ->  Index Scan using idx_mv_precios_articulo on mv_precios_por_articulo m
                                             (cost=0.29..3412.31 rows=99001 width=22)
                                             (actual time=0.005..5.882 rows=99001 loops=3)
                                             Buffers: shared hit=3296
 Planning:
   Buffers: shared hit=56
 Planning Time: 0.196 ms
 Execution Time: 20.840 ms
(36 rows)
```

```
 rubro | precio_promedio
-------+-----------------
     0 |         2604.89
(1 row)

                                                                                                                                                  QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=276529.63..276529.63 rows=1 width=40) (actual time=593.117..593.129 rows=1 loops=1)
   Buffers: shared hit=3952575 read=36885 written=5, temp read=303 written=304
   ->  Sort  (cost=276529.63..276773.12 rows=97395 width=40) (actual time=593.117..593.128 rows=2 loops=1)
         Sort Key: (round(avg(precio), 2)) DESC
         Sort Method: external merge  Disk: 2424kB
         Buffers: shared hit=3952575 read=36885 written=5, temp read=303 written=304
         ->  Finalize GroupAggregate  (cost=1000.46..276042.66 rows=97395 width=40)
               (actual time=3.181..577.715 rows=99001 loops=1)
               Group Key: id_articulo
               Buffers: shared hit=3952575 read=36885 written=5
               ->  Gather Merge  (cost=1000.46..273120.81 rows=194790 width=40)
                     (actual time=3.165..547.313 rows=136719 loops=1)
                     Workers Planned: 2
                     Workers Launched: 2
                     Buffers: shared hit=3952575 read=36885 written=5
                     ->  Partial GroupAggregate  (cost=0.43..249637.18 rows=97395 width=40)
                           (actual time=0.040..392.266 rows=45573 loops=3)
                           Group Key: id_articulo
                           Buffers: shared hit=3952575 read=36885 written=5
                           ->  Parallel Index Only Scan using idx_precios_articulo_fecha on precios p
                                 (cost=0.43..231753.21 rows=3333308 width=13)
                                 (actual time=0.029..290.328 rows=2666667 loops=3)
                                 Heap Fetches: 0
                                 Buffers: shared hit=3952575 read=36885 written=5
 Planning Time: 0.022 ms
 Execution Time: 593.655 ms
(21 rows)

                                                                                                                                  QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=2317.02..2317.02 rows=1 width=14) (actual time=17.703..17.705 rows=1 loops=1)
   Buffers: shared read=832, temp read=303 written=304
   ->  Sort  (cost=2317.02..2564.52 rows=99001 width=14) (actual time=17.703..17.703 rows=2 loops=1)
         Sort Key: promedio DESC
         Sort Method: external merge  Disk: 2424kB
         Buffers: shared read=832, temp read=303 written=304
         ->  Seq Scan on mv_precios_por_articulo m  (cost=0.00..1822.01 rows=99001 width=14)
               (actual time=0.004..3.430 rows=99001 loops=1)
               Buffers: shared read=832
 Planning:
   Buffers: shared read=3
 Planning Time: 0.068 ms
 Execution Time: 18.065 ms
(12 rows)

                                                                                                                                                QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=0.42..0.48 rows=1 width=14) (actual time=0.007..0.007 rows=1 loops=1)
   Buffers: shared hit=2 read=3
   ->  Index Scan using idx_mv_precios_promedio on mv_precios_por_articulo m
         (cost=0.42..5913.42 rows=99001 width=14) (actual time=0.006..0.007 rows=2 loops=1)
         Buffers: shared hit=2 read=3
 Planning:
   Buffers: shared hit=10 read=1
 Planning Time: 0.019 ms
 Execution Time: 0.010 ms
(8 rows)

 id_articulo | cantidad | precio_promedio |          descripcion           | rubro
-------------+----------+-----------------+--------------------------------+-------
       93411 |       62 |         3278.32 | FbrLckfLTjfqRsHdXD26cCk5I7s0nP |    18
(1 row)
```

ítem	variante	mediana ms	mín ms	máx ms	plan
b.8	Sobre las tablas, work_mem 4MB	642.264	627.843	650.410	Limit [Sort [Finalize GroupAggregate [Gather Merge [Sort [Partial HashAggregate [Parallel Hash Join [Parallel Seq Scan, Parallel Hash [Parallel Seq Scan]]]]]]]]
b.8	Sobre las tablas, work_mem 64MB	414.131	411.938	420.099	Limit [Sort [Finalize GroupAggregate [Gather Merge [Sort [Partial HashAggregate [Parallel Hash Join [Parallel Seq Scan, Parallel Hash [Parallel Seq Scan]]]]]]]]

ítem	variante	mediana ms	mín ms	máx ms	plan
b.8	Vista materializada	17.311	16.499	18.040	Limit [Sort [Finalize GroupAggregate [Gather Merge [Sort [Partial HashAggregate [Merge Join [Parallel Index Scan (articulos_pkey), Index Scan (idx_mv_precios_articulo)]]]]]]]
b.9	Sobre la tabla precios	507.899	497.870	515.104	Limit [Sort [Finalize GroupAggregate [Gather Merge [Partial GroupAggregate [Parallel Index Only Scan (idx_precios_articulo_fecha)]]]]]
b.9	Vista materializada	14.578	14.336	17.069	Limit [Sort [Seq Scan]]
b.9	Vista materializada + índice	0.003	0.003	0.003	Limit [Index Scan (idx_mv_precios_promedio)]

Parte c. Tabla personas

Se creó una tabla `personas` con un millón de filas (65 MB) y cuatro columnas con valores repetidos: nombre (40 distintos), apellido (50), edad (de 0 a 99, uniforme) y ciudad (16 ciudades de Entre Ríos). Las ciudades no tienen todas la misma frecuencia, como pasaría con datos reales: Paraná tiene el 25 % de las filas y Colón, la ciudad que pide el enunciado, el 3 % (29.972 personas). Como en la parte b, los valores se generan al azar y no tienen relación con el orden físico de las filas.

c.0 Creación de la tabla personas

Las ciudades se sortean con pesos armando un arreglo de 100 elementos en el que cada ciudad aparece tantas veces como su peso; sortear una posición de ese arreglo es sortear una ciudad con esa probabilidad. La salida muestra la distribución obtenida, que coincide con los pesos.

SCRIPT c00-personas.sql

```
SET search_path TO public;

-- c.0 Tabla personas con un millón de filas y cuatro columnas con valores
--     repetidos: nombre, apellido, edad y ciudad. Las ciudades se reparten con
--     pesos distintos para que no todas tengan la misma frecuencia, como pasaría
--     con datos reales: Paraná es la más grande y Colón tiene el 3 % de las filas.
--     La edad es uniforme entre 0 y 99 años.

DROP TABLE IF EXISTS personas;
DELETE FROM medicion WHERE item LIKE 'c.%';

CREATE TABLE personas (
    id        BIGINT PRIMARY KEY,
    nombre    VARCHAR(30) NOT NULL,
    apellido  VARCHAR(30) NOT NULL,
    edad      SMALLINT    NOT NULL,
    ciudad    VARCHAR(40) NOT NULL
);

SELECT setseed(0.2026);

WITH ciudades (ciudad, peso) AS (
    VALUES ('Paraná', 25), ('Concordia', 16), ('Gualeguaychú', 11),
           ('Concepción del Uruguay', 10), ('Gualeguay', 5), ('Villaguay', 5),
           ('Chajarí', 5), ('Victoria', 4), ('Colón', 3), ('La Paz', 3),
           ('Nogoyá', 3), ('San José', 2), ('Federación', 2), ('Crespo', 2),
           ('Villa Elisa', 2), ('Basavilbaso', 2)
),
urna AS (
    -- Cada ciudad aparece tantas veces como su peso: sortear una posición de este
    -- arreglo de 100 elementos es sortear una ciudad con esa probabilidad.
    SELECT array_agg(ciudad) AS ciudades
    FROM ciudades, generate_series(1, peso)
),
nombres AS (
    SELECT ARRAY['Juan','María','José','Ana','Carlos','Laura','Luis','Sofía','Jorge',
                 'Lucía','Miguel','Valentina','Pedro','Camila','Diego','Martina',
                 'Pablo','Julieta','Andrés','Florencia','Ricardo','Paula','Sergio',
                 'Carolina','Martín','Gabriela','Fernando','Natalia','Leandro',
                 'Agustina','Manuel','Victoria','Nicolás','Micaela','Federico',
                 'Rocío','Gustavo','Daniela','Alejandro','Romina'] AS v
),
apellidos AS (
    SELECT ARRAY['González','Rodríguez','Gómez','Fernández','López','Díaz',
                 'Martínez','Pérez','García','Sánchez','Romero','Sosa','Álvarez',
                 'Torres','Ruiz','Ramírez','Flores','Acosta','Benítez','Medina',
                 'Suárez','Herrera','Aguirre','Pereyra','Gutiérrez','Giménez',
                 'Molina','Silva','Castro','Rojas','Ortiz','Núñez','Luna','Juárez',
                 'Cabrera','Ríos','Morales','Godoy','Moreno','Ferreyra','Domínguez',
                 'Carrizo','Peralta','Castillo','Ledesma','Quiroga','Vega','Vera',
                 'Muñoz','Ojeda'] AS v
)
INSERT INTO personas (id, nombre, apellido, edad, ciudad)
SELECT g,
       n.v[1 + floor(random() * cardinality(n.v))::INTEGER],
       a.v[1 + floor(random() * cardinality(a.v))::INTEGER],
       floor(random() * 100)::SMALLINT,
       u.ciudades[1 + floor(random() * 100)::INTEGER]
FROM generate_series(1, 1000000) AS g
CROSS JOIN nombres n
CROSS JOIN apellidos a
CROSS JOIN urna u;

VACUUM ANALYZE personas;

SELECT c.relname                              AS tabla,
       c.reltuples::BIGINT                    AS filas,
       c.relpages                             AS paginas,
```

```
       pg_size_pretty(pg_relation_size(c.oid)) AS tamanio
FROM pg_class c
WHERE c.relname = 'personas';

SELECT ciudad, COUNT(*) AS personas,
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 2) AS porcentaje
FROM personas
GROUP BY ciudad
ORDER BY personas DESC;

SELECT COUNT(DISTINCT nombre)   AS nombres,
       COUNT(DISTINCT apellido) AS apellidos,
       COUNT(DISTINCT edad)     AS edades,
       COUNT(DISTINCT ciudad)   AS ciudades
FROM personas;
```

SALIDA

```
 setseed
---------

(1 row)

  tabla   |  filas  | paginas | tamanio
----------+---------+---------+---------
 personas | 1000000 |    8368 | 65 MB
(1 row)

         ciudad         | personas | porcentaje
------------------------+----------+------------
 Paraná                 |   249455 |      24.95
 Concordia              |   159664 |      15.97
 Gualeguaychú           |   110133 |      11.01
 Concepción del Uruguay |    99748 |       9.97
 Villaguay              |    50477 |       5.05
 Chajarí                |    50331 |       5.03
 Gualeguay              |    49704 |       4.97
 Victoria               |    40291 |       4.03
 La Paz                 |    30110 |       3.01
 Nogoyá                 |    29985 |       3.00
 Colón                  |    29972 |       3.00
 Federación             |    20201 |       2.02
 Crespo                 |    20110 |       2.01
 Basavilbaso            |    20092 |       2.01
 San José               |    19888 |       1.99
 Villa Elisa            |    19839 |       1.98
(16 rows)

 nombres | apellidos | edades | ciudades
---------+-----------+--------+----------
      40 |        50 |    100 |       16
(1 row)
```

c.1, c.2 y c.3 Personas de Colón

c.1, sin índice. El script ejecuta primero la consulta exactamente como la da el enunciado, con `EXPLAIN ANALYZE`, y después con `BUFFERS`. El plan es un `Gather` con un `Parallel Seq Scan`: el líder y dos procesos auxiliares se reparten las 8.368 páginas, cada uno evalúa la condición sobre su tercio de la tabla (descarta 323.343 filas y se queda con unas 10.000) y el `Gather` reúne las 29.972 filas. La estimación (31.567) es buena gracias a las estadísticas de valores frecuentes. Mediana: 12,5 ms.

c.2, con B-Tree sobre ciudad. `Bitmap Index Scan` sobre el índice y `Bitmap Heap Scan`. Mediana: 2,68 ms, 4,7 veces menos. Como en el b.1, el plan lee 8.133 de las 8.368 páginas (`Heap Blocks`): con unas 120 personas por página, casi todas tienen alguien de Colón. Lo que se ahorra no es lectura sino evaluar la condición sobre el millón de filas y comparar cadenas.

Se probó también con Paraná, que tiene el 25 % de las filas. El planificador sigue eligiendo el índice (8,7 ms), y en este caso con razón: deshabilitándolo, el `Seq Scan` tarda 29,4 ms. El mapa de bits termina abarcando todas las páginas, pero las lee en orden físico, como un Seq Scan, y sólo procesa las filas marcadas. Además, al devolver 250.000 filas el planificador ni siquiera usa procesos paralelos, porque pasar tantas filas por el `Gather` es caro.

c.3, con índice hash. Se eliminó el B-Tree y se creó un hash. El plan tiene la misma forma: `Bitmap Index Scan` sobre el hash y `Bitmap Heap Scan` con `Recheck Cond`, que en el hash es imprescindible porque el índice guarda códigos hash, no las ciudades. Mediana: 3,38 ms, un poco más lenta que con el B-Tree.

La teoría dice que la búsqueda en un hash es O(1) y en un B-Tree logarítmica, pero en la práctica la diferencia no aparece, por dos razones: el B-Tree tiene tres niveles y bajar por él son tres lecturas de páginas que están en memoria, y en las dos versiones la mayor parte del tiempo se va en el `Bitmap Heap Scan`, que es igual. Lo que sí es muy distinto es el tamaño: el hash ocupa 47 MB y el B-Tree 6,9 MB. Con sólo 16 valores distintos, la deduplicación del B-Tree guarda cada ciudad una vez con la lista de sus filas, mientras que el hash guarda una entrada por fila. Para una columna con pocos valores distintos el B-Tree es mejor en todo, y además sirve para rangos y ordenamientos.

SCRIPT c01-ciudad.sql

```
SET search_path TO public;

DELETE FROM medicion WHERE item IN ('c.1', 'c.2', 'c.3');
DROP INDEX IF EXISTS idx_personas_ciudad_btree;
DROP INDEX IF EXISTS idx_personas_ciudad_hash;
DROP INDEX IF EXISTS idx_personas_edad;
DROP INDEX IF EXISTS idx_personas_ciudad_edad;
ANALYZE personas;

-- c.1) Personas de Colón, sin índices sobre ciudad.

EXPLAIN ANALYZE SELECT * FROM personas WHERE ciudad = 'Colón';

EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM personas WHERE ciudad = 'Colón';

CALL pr_medir('c.1', 'Sin índice', $$
    SELECT * FROM personas WHERE ciudad = 'Colón'
$$, 10);

-- c.2) Índice B-Tree sobre ciudad.

CREATE INDEX idx_personas_ciudad_btree ON personas USING btree (ciudad);
ANALYZE personas;

EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM personas WHERE ciudad = 'Colón';

CALL pr_medir('c.2', 'Índice B-Tree', $$
    SELECT * FROM personas WHERE ciudad = 'Colón'
$$, 10);

-- c.2) Paraná tiene el 25 % de las filas. El planificador sigue eligiendo el
--      Bitmap Heap Scan; se compara con el Seq Scan deshabilitando el bitmap.

EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM personas WHERE ciudad = 'Paraná';

CALL pr_medir('c.2', 'Paraná, índice B-Tree', $$
    SELECT * FROM personas WHERE ciudad = 'Paraná'
```

```
$$, 10);

SET enable_bitmapscan = off;
SET enable_indexscan = off;
CALL pr_medir('c.2', 'Paraná, sin usar el índice', $$
    SELECT * FROM personas WHERE ciudad = 'Paraná'
$$, 10);
RESET enable_bitmapscan;
RESET enable_indexscan;

-- c.3) Se elimina el B-Tree y se crea un índice hash sobre ciudad.

DROP INDEX idx_personas_ciudad_btree;
CREATE INDEX idx_personas_ciudad_hash ON personas USING hash (ciudad);
ANALYZE personas;

EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM personas WHERE ciudad = 'Colón';

CALL pr_medir('c.3', 'Índice hash', $$
    SELECT * FROM personas WHERE ciudad = 'Colón'
$$, 10);

-- Tamaño de cada índice sobre la misma columna. El B-Tree se recrea sólo para
-- medirlo y se vuelve a borrar.

CREATE INDEX idx_personas_ciudad_btree ON personas USING btree (ciudad);

SELECT c.relname AS relacion, am.amname AS metodo,
       pg_size_pretty(pg_relation_size(c.oid)) AS tamanio
FROM pg_class c
LEFT JOIN pg_am am ON am.oid = c.relam
WHERE c.relname IN ('personas', 'idx_personas_ciudad_btree', 'idx_personas_ciudad_hash')
ORDER BY pg_relation_size(c.oid) DESC;

DROP INDEX idx_personas_ciudad_btree;

SELECT COUNT(*) AS personas_de_colon FROM personas WHERE ciudad = 'Colón';

SELECT item, variante, mediana_ms, min_ms, max_ms, plan
FROM medicion
WHERE item IN ('c.1', 'c.2', 'c.3')
ORDER BY id;
```

SALIDA

```
                                                        QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------
 Gather  (cost=1000.00..17586.33 rows=30100 width=35) (actual time=0.067..11.893 rows=29972 loops=1)
   Workers Planned: 2
   Workers Launched: 2
   ->  Parallel Seq Scan on personas  (cost=0.00..13576.33 rows=12542 width=35)
         (actual time=0.004..9.949 rows=9991 loops=3)
         Filter: ((ciudad)::text = 'Colón'::text)
         Rows Removed by Filter: 323343
 Planning Time: 0.025 ms
 Execution Time: 12.341 ms
(8 rows)

                                                        QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------
 Gather  (cost=1000.00..17586.33 rows=30100 width=35) (actual time=0.036..12.609 rows=29972 loops=1)
   Workers Planned: 2
   Workers Launched: 2
   Buffers: shared hit=8368
   ->  Parallel Seq Scan on personas  (cost=0.00..13576.33 rows=12542 width=35)
         (actual time=0.004..10.867 rows=9991 loops=3)
         Filter: ((ciudad)::text = 'Colón'::text)
         Rows Removed by Filter: 323343
         Buffers: shared hit=8368
 Planning Time: 0.021 ms
 Execution Time: 13.116 ms
(10 rows)

                                                                QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on personas  (cost=344.80..9094.05 rows=30500 width=35)
     (actual time=1.051..4.143 rows=29972 loops=1)
   Recheck Cond: ((ciudad)::text = 'Colón'::text)
   Heap Blocks: exact=8133
```

```
   Buffers: shared hit=8133 read=28
   ->  Bitmap Index Scan on idx_personas_ciudad_btree  (cost=0.00..337.18 rows=30500 width=0)
         (actual time=0.557..0.557 rows=29972 loops=1)
         Index Cond: ((ciudad)::text = 'Colón'::text)
         Buffers: shared read=28
 Planning:
   Buffers: shared hit=19 read=1
 Planning Time: 0.030 ms
 Execution Time: 4.642 ms
(11 rows)

                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on personas  (cost=2765.64..14219.48 rows=246867 width=35)
     (actual time=2.658..11.162 rows=249455 loops=1)
   Recheck Cond: ((ciudad)::text = 'Paraná'::text)
   Heap Blocks: exact=8368
   Buffers: shared hit=8369 read=212
   ->  Bitmap Index Scan on idx_personas_ciudad_btree  (cost=0.00..2703.93 rows=246867 width=0)
         (actual time=2.156..2.156 rows=249455 loops=1)
         Index Cond: ((ciudad)::text = 'Paraná'::text)
         Buffers: shared hit=1 read=212
 Planning Time: 0.008 ms
 Execution Time: 15.189 ms
(9 rows)

                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on personas  (cost=1003.87..9765.21 rows=31467 width=35)
     (actual time=1.109..4.657 rows=29972 loops=1)
   Recheck Cond: ((ciudad)::text = 'Colón'::text)
   Heap Blocks: exact=8133
   Buffers: shared hit=8208
   ->  Bitmap Index Scan on idx_personas_ciudad_hash  (cost=0.00..996.00 rows=31467 width=0)
         (actual time=0.610..0.610 rows=29972 loops=1)
         Index Cond: ((ciudad)::text = 'Colón'::text)
         Buffers: shared hit=75
 Planning:
   Buffers: shared hit=17
 Planning Time: 0.028 ms
 Execution Time: 5.158 ms
(11 rows)

         relacion          | metodo | tamanio
---------------------------+--------+---------
 personas                  | heap   | 65 MB
 idx_personas_ciudad_hash  | hash   | 47 MB
 idx_personas_ciudad_btree | btree  | 6880 kB
(3 rows)

 personas_de_colon
-------------------
             29972
(1 row)
```

ítem	variante	mediana ms	mín ms	máx ms	plan
c.1	Sin índice	12.537	11.954	12.741	`Gather [Parallel Seq Scan]`
c.2	Índice B-Tree	2.676	2.588	2.930	`Bitmap Heap Scan [Bitmap Index Scan (idx_personas_ciudad_btree)]`
c.2	Paraná, índice B-Tree	8.719	8.432	8.935	`Bitmap Heap Scan [Bitmap Index Scan (idx_personas_ciudad_btree)]`
c.2	Paraná, sin usar el índice	29.361	28.580	29.777	`Seq Scan`
c.3	Índice hash	3.382	3.206	3.714	`Bitmap Heap Scan [Bitmap Index Scan (idx_personas_ciudad_hash)]`

c.4 y c.5 Personas en un rango de edad

c.4, sin índice. Personas de 30 a 34 años (49.754, el 5 %). El índice hash del c.3 no puede usarse para un rango, porque el hash no conserva el orden de los valores. El plan es un `Parallel Seq Scan` con el filtro `edad >= 30 AND edad <= 34`: 12,0 ms. Una sola edad (`edad = 30`), 9,6 ms.

c.5, con B-Tree sobre edad. El B-Tree resuelve el rango como un recorrido de sus hojas desde la primera entrada con edad 30 hasta la última con 34, que el plan muestra como `Index Cond: ((edad >= 30) AND (edad <= 34))`. Con eso arma el mapa de bits y el `Bitmap Heap Scan` trae las filas: 3,66 ms, 3,3 veces menos. Para una sola edad (1 %) son 1,30 ms, 7,4 veces menos; cuanto más selectiva la condición, más gana el índice. Para un rango amplio, de 18 a 64 años (47 % de la tabla), el planificador todavía usa el índice, pero el mapa de bits ya cubre todas las páginas y el plan equivale a un recorrido secuencial.

La última prueba cuenta las personas del rango. Como no necesita ninguna columna fuera del índice, es un `Index Only Scan` sin acceso a la tabla (`Heap Fetches: 0`): 1,0 ms, doce veces menos que recorriendo la tabla.

SCRIPT c02-edad.sql

```
SET search_path TO public;

DELETE FROM medicion WHERE item IN ('c.4', 'c.5');
DROP INDEX IF EXISTS idx_personas_edad;
DROP INDEX IF EXISTS idx_personas_ciudad_edad;
ANALYZE personas;

-- c.4) Personas de entre 30 y 34 años (5 de las 100 edades, un 5 % de la tabla).
--      El índice hash del c.3 no sirve para rangos: sólo resuelve igualdades.

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM personas
WHERE edad BETWEEN 30 AND 34;

CALL pr_medir('c.4', 'Rango 30-34, sin índice', $$
    SELECT * FROM personas WHERE edad BETWEEN 30 AND 34
$$, 10);

CALL pr_medir('c.4', 'Edad = 30, sin índice', $$
    SELECT * FROM personas WHERE edad = 30
$$, 10);

-- c.5) Índice B-Tree sobre edad.

CREATE INDEX idx_personas_edad ON personas USING btree (edad);
ANALYZE personas;

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM personas
WHERE edad BETWEEN 30 AND 34;

CALL pr_medir('c.5', 'Rango 30-34, B-Tree', $$
    SELECT * FROM personas WHERE edad BETWEEN 30 AND 34
$$, 10);

-- Una sola edad (1 %) y un rango amplio (de 18 a 64, casi la mitad de la tabla).

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM personas
WHERE edad = 30;

CALL pr_medir('c.5', 'Edad = 30, B-Tree', $$
    SELECT * FROM personas WHERE edad = 30
$$, 10);

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM personas
WHERE edad BETWEEN 18 AND 64;

-- Si la consulta sólo necesita columnas que están en el índice, se resuelve sin
-- leer la tabla: cuántas personas hay en el rango.
```

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT COUNT(*)
FROM personas
WHERE edad BETWEEN 30 AND 34;

CALL pr_medir('c.5', 'COUNT(*) rango 30-34, B-Tree', $$
    SELECT COUNT(*) FROM personas WHERE edad BETWEEN 30 AND 34
$$, 10);

SELECT edad, COUNT(*) AS personas
FROM personas
WHERE edad BETWEEN 30 AND 34
GROUP BY edad
ORDER BY edad;

SELECT item, variante, mediana_ms, min_ms, max_ms, plan
FROM medicion
WHERE item IN ('c.4', 'c.5')
ORDER BY id;
```

SALIDA

```
                                                                 QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------------------------
 Gather  (cost=1000.00..20308.00 rows=46900 width=35) (actual time=0.065..12.491 rows=49754 loops=1)
   Workers Planned: 2
   Workers Launched: 2
   Buffers: shared hit=8368
   ->  Parallel Seq Scan on personas  (cost=0.00..14618.00 rows=19542 width=35)
         (actual time=0.003..10.445 rows=16585 loops=3)
         Filter: ((edad >= 30) AND (edad <= 34))
         Rows Removed by Filter: 316749
         Buffers: shared hit=8368
 Planning:
   Buffers: shared hit=18
 Planning Time: 0.025 ms
 Execution Time: 13.338 ms
(12 rows)

                                                             QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on personas  (cost=676.72..9782.73 rows=49200 width=35)
     (actual time=1.331..5.399 rows=49754 loops=1)
   Recheck Cond: ((edad >= 30) AND (edad <= 34))
   Heap Blocks: exact=8344
   Buffers: shared hit=8347 read=45
   ->  Bitmap Index Scan on idx_personas_edad  (cost=0.00..664.42 rows=49200 width=0)
         (actual time=0.823..0.823 rows=49754 loops=1)
         Index Cond: ((edad >= 30) AND (edad <= 34))
         Buffers: shared hit=3 read=45
 Planning:
   Buffers: shared hit=23 read=1
 Planning Time: 0.041 ms
 Execution Time: 6.226 ms
(11 rows)

                                                             QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on personas  (cost=103.21..8956.97 rows=9133 width=35)
     (actual time=0.540..1.372 rows=9752 loops=1)
   Recheck Cond: (edad = 30)
   Heap Blocks: exact=5794
   Buffers: shared hit=5805
   ->  Bitmap Index Scan on idx_personas_edad  (cost=0.00..100.92 rows=9133 width=0)
         (actual time=0.225..0.225 rows=9752 loops=1)
         Index Cond: (edad = 30)
         Buffers: shared hit=11
 Planning Time: 0.008 ms
 Execution Time: 1.530 ms
(9 rows)

                                                               QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on personas  (cost=6406.55..21802.05 rows=468500 width=35)
     (actual time=5.846..20.575 rows=469304 loops=1)
   Recheck Cond: ((edad >= 18) AND (edad <= 64))
   Heap Blocks: exact=8368
   Buffers: shared hit=8412 read=357
   ->  Bitmap Index Scan on idx_personas_edad  (cost=0.00..6289.43 rows=468500 width=0)
```

```
         (actual time=5.344..5.344 rows=469304 loops=1)
         Index Cond: ((edad >= 18) AND (edad <= 64))
         Buffers: shared hit=44 read=357
 Planning Time: 0.008 ms
 Execution Time: 28.140 ms
(9 rows)

                                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------
 Aggregate  (cost=1279.42..1279.43 rows=1 width=8) (actual time=2.021..2.021 rows=1 loops=1)
   Buffers: shared hit=46
   ->  Index Only Scan using idx_personas_edad on personas  (cost=0.42..1156.42 rows=49200 width=0)
         (actual time=0.004..1.080 rows=49754 loops=1)
         Index Cond: ((edad >= 30) AND (edad <= 34))
         Heap Fetches: 0
         Buffers: shared hit=46
 Planning:
   Buffers: shared hit=6
 Planning Time: 0.017 ms
 Execution Time: 2.027 ms
(10 rows)

 edad | personas
------+----------
   30 |     9752
   31 |     9880
   32 |     9990
   33 |    10077
   34 |    10055
(5 rows)
```

ítem	variante	mediana ms	mín ms	máx ms	plan
c.4	Rango 30-34, sin índice	12.000	11.761	12.672	Gather [Parallel Seq Scan]
c.4	Edad = 30, sin índice	9.565	9.486	9.773	Gather [Parallel Seq Scan]
c.5	Rango 30-34, B-Tree	3.664	3.584	4.026	Bitmap Heap Scan [Bitmap Index Scan (idx_personas_edad)]
c.5	Edad = 30, B-Tree	1.298	1.228	1.350	Bitmap Heap Scan [Bitmap Index Scan (idx_personas_edad)]
c.5	COUNT(*) rango 30-34, B-Tree	0.995	0.990	1.007	Aggregate [Index Only Scan (idx_personas_edad)]

c.6 Funciones agregadas sobre la edad

Se plantearon dos consultas: totales de toda la tabla (`COUNT`, `SUM` y `AVG` de la edad) y los mismos agregados por ciudad, y se midieron sin índices, con un B-Tree sobre edad y con uno compuesto sobre (ciudad, edad).

Sin índices. Los totales se calculan con agregación en paralelo: cada proceso lee su parte con `Parallel Seq Scan` y calcula sus totales parciales (`Partial Aggregate`), y el `Finalize Aggregate` los combina (para el promedio combina sumas y cantidades, no promedios). Mediana: 14,2 ms. Por ciudad es igual pero con `Partial HashAggregate` de 16 grupos por proceso, un `Sort` y un `Gather Merge` que junta los parciales ordenados por ciudad: 28,3 ms.

Con índice sobre edad. El planificador no lo usa: el plan y el tiempo son los mismos. Calcular agregados de toda la tabla obliga a leer todas las edades, y el índice no evita eso; lo único que podría ofrecer es leerlas de una estructura más chica (6,8 MB contra 65 MB de la tabla). Con el compuesto (ciudad, edad) pasa lo mismo con la consulta por ciudad.

Con `random_page_cost = 1.1` el planificador sí elige leer los índices: `Parallel Index Only Scan` sobre el de edad para los totales (12,2 ms) y sobre el compuesto para la consulta por ciudad, que además llega ordenada por ciudad y se agrupa con `Partial GroupAggregate` sin tabla hash ni `Sort` (23,7 ms). Las mejoras son del 14 % y el 16 %: con todo en memoria, el costo está en procesar el millón de valores, no en leerlos.

Donde el índice sobre la columna de cálculo cambia el resultado es cuando hay un filtro: los agregados de una sola ciudad (Colón) se resuelven con un `Index Only Scan` sobre el tramo de Colón del índice (ciudad, edad), en 0,85 ms. Un índice sobre la columna que se agrega sirve, entonces, para que la consulta se responda sólo con el índice; para que sirva de verdad tiene que haber además una condición sobre la columna que lo encabeza.

SCRIPT c03-agregados.sql

```
SET search_path TO public;

DELETE FROM medicion WHERE item = 'c.6';
DROP INDEX IF EXISTS idx_personas_edad;
DROP INDEX IF EXISTS idx_personas_ciudad_edad;
ANALYZE personas;

-- c.6) Funciones agregadas sobre la columna edad, sin y con índices sobre ella.
--      Consulta 1: totales de toda la tabla. Consulta 2: los mismos agregados por
--      ciudad.

EXPLAIN (ANALYZE, BUFFERS)
SELECT COUNT(*) AS personas, SUM(edad) AS suma_edades, ROUND(AVG(edad), 2) AS edad_promedio
FROM personas;

CALL pr_medir('c.6', 'Totales, sin índice', $$
    SELECT COUNT(*), SUM(edad), AVG(edad) FROM personas
$$, 10);

EXPLAIN (ANALYZE, BUFFERS)
SELECT ciudad, COUNT(*) AS personas, SUM(edad) AS suma_edades,
       ROUND(AVG(edad), 2) AS edad_promedio
FROM personas
GROUP BY ciudad
ORDER BY ciudad;

CALL pr_medir('c.6', 'Por ciudad, sin índice', $$
    SELECT ciudad, COUNT(*), SUM(edad), AVG(edad) FROM personas
    GROUP BY ciudad ORDER BY ciudad
$$, 10);

-- Índice B-Tree sobre edad: los totales se pueden calcular leyendo sólo el
-- índice (Index Only Scan), que es más chico que la tabla.

CREATE INDEX idx_personas_edad ON personas USING btree (edad);
VACUUM ANALYZE personas;

EXPLAIN (ANALYZE, BUFFERS)
SELECT COUNT(*) AS personas, SUM(edad) AS suma_edades, ROUND(AVG(edad), 2) AS edad_promedio
FROM personas;

CALL pr_medir('c.6', 'Totales, índice (edad)', $$
    SELECT COUNT(*), SUM(edad), AVG(edad) FROM personas
```

```
$$, 10);

-- Por ciudad, el índice sobre edad sola no alcanza: falta la ciudad.

EXPLAIN (ANALYZE, BUFFERS)
SELECT ciudad, COUNT(*) AS personas, SUM(edad) AS suma_edades,
       ROUND(AVG(edad), 2) AS edad_promedio
FROM personas
GROUP BY ciudad
ORDER BY ciudad;

CALL pr_medir('c.6', 'Por ciudad, índice (edad)', $$
    SELECT ciudad, COUNT(*), SUM(edad), AVG(edad) FROM personas
    GROUP BY ciudad ORDER BY ciudad
$$, 10);

-- Índice compuesto (ciudad, edad): tiene las dos columnas y además entrega las
-- filas ordenadas por ciudad, así que el agrupamiento no necesita hash ni sort.

CREATE INDEX idx_personas_ciudad_edad ON personas USING btree (ciudad, edad);
VACUUM ANALYZE personas;

EXPLAIN (ANALYZE, BUFFERS)
SELECT ciudad, COUNT(*) AS personas, SUM(edad) AS suma_edades,
       ROUND(AVG(edad), 2) AS edad_promedio
FROM personas
GROUP BY ciudad
ORDER BY ciudad;

CALL pr_medir('c.6', 'Por ciudad, índice (ciudad, edad)', $$
    SELECT ciudad, COUNT(*), SUM(edad), AVG(edad) FROM personas
    GROUP BY ciudad ORDER BY ciudad
$$, 10);

-- Ni con el índice sobre edad ni con el compuesto cambia el plan de las dos
-- consultas que recorren toda la tabla. Con random_page_cost = 1.1 (SSD, o datos en
-- memoria, que es el caso) el planificador pasa a leer sólo los índices.

SET random_page_cost = 1.1;

EXPLAIN (ANALYZE, BUFFERS)
SELECT COUNT(*) AS personas, SUM(edad) AS suma_edades, ROUND(AVG(edad), 2) AS edad_promedio
FROM personas;

CALL pr_medir('c.6', 'Totales, índices, random_page_cost 1.1', $$
    SELECT COUNT(*), SUM(edad), AVG(edad) FROM personas
$$, 10);

EXPLAIN (ANALYZE, BUFFERS)
SELECT ciudad, COUNT(*) AS personas, SUM(edad) AS suma_edades,
       ROUND(AVG(edad), 2) AS edad_promedio
FROM personas
GROUP BY ciudad
ORDER BY ciudad;

CALL pr_medir('c.6', 'Por ciudad, índices, random_page_cost 1.1', $$
    SELECT ciudad, COUNT(*), SUM(edad), AVG(edad) FROM personas
    GROUP BY ciudad ORDER BY ciudad
$$, 10);

RESET random_page_cost;

-- Con el filtro por ciudad, el índice compuesto da los agregados de una sola
-- ciudad leyendo únicamente su tramo del índice.

EXPLAIN (ANALYZE, BUFFERS)
SELECT COUNT(*) AS personas, SUM(edad) AS suma_edades, ROUND(AVG(edad), 2) AS edad_promedio
FROM personas
WHERE ciudad = 'Colón';

CALL pr_medir('c.6', 'Colón, índice (ciudad, edad)', $$
    SELECT COUNT(*), SUM(edad), AVG(edad) FROM personas WHERE ciudad = 'Colón'
$$, 10);

SELECT pg_size_pretty(pg_relation_size('personas'))                AS tabla,
       pg_size_pretty(pg_relation_size('idx_personas_edad'))       AS indice_edad,
       pg_size_pretty(pg_relation_size('idx_personas_ciudad_edad')) AS indice_ciudad_edad;
```

```
SELECT ciudad, COUNT(*) AS personas, SUM(edad) AS suma_edades,
       ROUND(AVG(edad), 2) AS edad_promedio
FROM personas
GROUP BY ciudad
ORDER BY ciudad;

SELECT item, variante, mediana_ms, min_ms, max_ms, plan
FROM medicion
WHERE item = 'c.6'
ORDER BY id;
```

SALIDA

```
                                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------
 Finalize Aggregate  (cost=16659.90..16659.91 rows=1 width=48) (actual time=17.961..18.664 rows=1 loops=1)
   Buffers: shared hit=8368
   ->  Gather  (cost=16659.67..16659.88 rows=2 width=48) (actual time=17.916..18.648 rows=3 loops=1)
         Workers Planned: 2
         Workers Launched: 2
         Buffers: shared hit=8368
         ->  Partial Aggregate  (cost=15659.67..15659.68 rows=1 width=48)
               (actual time=17.002..17.002 rows=1 loops=3)
               Buffers: shared hit=8368
               ->  Parallel Seq Scan on personas  (cost=0.00..12534.67 rows=416667 width=2)
                     (actual time=0.002..7.743 rows=333333 loops=3)
                     Buffers: shared hit=8368
 Planning:
   Buffers: shared hit=26 read=1
 Planning Time: 0.064 ms
 Execution Time: 18.676 ms
(14 rows)

                                                                       QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------
 Finalize GroupAggregate  (cost=17701.84..17706.13 rows=16 width=59) (actual time=31.080..31.837 rows=16 loops=1)
   Group Key: ciudad
   Buffers: shared hit=8384
   ->  Gather Merge  (cost=17701.84..17705.57 rows=32 width=59) (actual time=31.076..31.820 rows=48 loops=1)
         Workers Planned: 2
         Workers Launched: 2
         Buffers: shared hit=8384
         ->  Sort  (cost=16701.82..16701.86 rows=16 width=59) (actual time=30.012..30.012 rows=16 loops=3)
               Sort Key: ciudad
               Sort Method: quicksort  Memory: 26kB
               Buffers: shared hit=8384
               Worker 0:  Sort Method: quicksort  Memory: 26kB
               Worker 1:  Sort Method: quicksort  Memory: 26kB
               ->  Partial HashAggregate  (cost=16701.34..16701.50 rows=16 width=59)
                     (actual time=29.983..29.984 rows=16 loops=3)
                     Group Key: ciudad
                     Batches: 1  Memory Usage: 24kB
                     Buffers: shared hit=8368
                     Worker 0:  Batches: 1  Memory Usage: 24kB
                     Worker 1:  Batches: 1  Memory Usage: 24kB
                     ->  Parallel Seq Scan on personas  (cost=0.00..12534.67 rows=416667 width=13)
                           (actual time=0.001..7.661 rows=333333 loops=3)
                           Buffers: shared hit=8368
 Planning:
   Buffers: shared hit=9
 Planning Time: 0.022 ms
 Execution Time: 31.845 ms
(25 rows)

                                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------
 Finalize Aggregate  (cost=16659.90..16659.91 rows=1 width=48) (actual time=18.281..19.058 rows=1 loops=1)
   Buffers: shared hit=8368
   ->  Gather  (cost=16659.67..16659.88 rows=2 width=48) (actual time=18.248..19.053 rows=3 loops=1)
         Workers Planned: 2
         Workers Launched: 2
         Buffers: shared hit=8368
         ->  Partial Aggregate  (cost=15659.67..15659.68 rows=1 width=48)
               (actual time=17.071..17.071 rows=1 loops=3)
               Buffers: shared hit=8368
               ->  Parallel Seq Scan on personas  (cost=0.00..12534.67 rows=416667 width=2)
                     (actual time=0.002..7.810 rows=333333 loops=3)
                     Buffers: shared hit=8368
 Planning:
   Buffers: shared hit=6
 Planning Time: 0.022 ms
```

```
 Execution Time: 19.066 ms
(14 rows)

                                                                                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------------------
 Finalize GroupAggregate  (cost=17701.84..17706.13 rows=16 width=59) (actual time=32.800..33.507 rows=16 loops=1)
   Group Key: ciudad
   Buffers: shared hit=8384
   ->  Gather Merge  (cost=17701.84..17705.57 rows=32 width=59) (actual time=32.796..33.492 rows=48 loops=1)
         Workers Planned: 2
         Workers Launched: 2
         Buffers: shared hit=8384
         ->  Sort  (cost=16701.82..16701.86 rows=16 width=59) (actual time=31.694..31.695 rows=16 loops=3)
               Sort Key: ciudad
               Sort Method: quicksort  Memory: 26kB
               Buffers: shared hit=8384
               Worker 0:  Sort Method: quicksort  Memory: 26kB
               Worker 1:  Sort Method: quicksort  Memory: 26kB
               ->  Partial HashAggregate  (cost=16701.34..16701.50 rows=16 width=59)
                     (actual time=31.666..31.667 rows=16 loops=3)
                     Group Key: ciudad
                     Batches: 1  Memory Usage: 24kB
                     Buffers: shared hit=8368
                     Worker 0:  Batches: 1  Memory Usage: 24kB
                     Worker 1:  Batches: 1  Memory Usage: 24kB
                     ->  Parallel Seq Scan on personas  (cost=0.00..12534.67 rows=416667 width=13)
                           (actual time=0.001..7.993 rows=333333 loops=3)
                           Buffers: shared hit=8368
 Planning:
   Buffers: shared hit=8
 Planning Time: 0.030 ms
 Execution Time: 33.516 ms
(25 rows)

                                                                                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------------------
 Finalize GroupAggregate  (cost=17701.84..17706.13 rows=16 width=59) (actual time=30.978..31.767 rows=16 loops=1)
   Group Key: ciudad
   Buffers: shared hit=8384
   ->  Gather Merge  (cost=17701.84..17705.57 rows=32 width=59) (actual time=30.973..31.756 rows=48 loops=1)
         Workers Planned: 2
         Workers Launched: 2
         Buffers: shared hit=8384
         ->  Sort  (cost=16701.82..16701.86 rows=16 width=59) (actual time=29.774..29.775 rows=16 loops=3)
               Sort Key: ciudad
               Sort Method: quicksort  Memory: 26kB
               Buffers: shared hit=8384
               Worker 0:  Sort Method: quicksort  Memory: 26kB
               Worker 1:  Sort Method: quicksort  Memory: 26kB
               ->  Partial HashAggregate  (cost=16701.34..16701.50 rows=16 width=59)
                     (actual time=29.747..29.748 rows=16 loops=3)
                     Group Key: ciudad
                     Batches: 1  Memory Usage: 24kB
                     Buffers: shared hit=8368
                     Worker 0:  Batches: 1  Memory Usage: 24kB
                     Worker 1:  Batches: 1  Memory Usage: 24kB
                     ->  Parallel Seq Scan on personas  (cost=0.00..12534.67 rows=416667 width=13)
                           (actual time=0.001..7.755 rows=333333 loops=3)
                           Buffers: shared hit=8368
 Planning:
   Buffers: shared hit=9
 Planning Time: 0.030 ms
 Execution Time: 31.778 ms
(25 rows)

                                                                                                              QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------------------
 Finalize Aggregate  (cost=14232.82..14232.83 rows=1 width=48) (actual time=17.225..17.833 rows=1 loops=1)
   Buffers: shared hit=3 read=850
   ->  Gather  (cost=14232.59..14232.80 rows=2 width=48) (actual time=17.199..17.825 rows=3 loops=1)
         Workers Planned: 2
         Workers Launched: 2
         Buffers: shared hit=3 read=850
         ->  Partial Aggregate  (cost=13232.59..13232.60 rows=1 width=48)
               (actual time=16.123..16.123 rows=1 loops=3)
               Buffers: shared hit=3 read=850
               ->  Parallel Index Only Scan using idx_personas_edad on personas
                     (cost=0.42..10107.59 rows=416667 width=2) (actual time=0.026..8.274 rows=333333 loops=3)
                     Heap Fetches: 0
                     Buffers: shared hit=3 read=850
 Planning Time: 0.016 ms
 Execution Time: 17.841 ms
(13 rows)
```

```
                                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Finalize GroupAggregate  (cost=1000.45..15322.70 rows=16 width=59) (actual time=33.245..33.784 rows=16 loops=1)
   Group Key: ciudad
   Buffers: shared hit=3 read=889
   ->  Gather Merge  (cost=1000.45..15322.14 rows=32 width=59) (actual time=33.214..33.776 rows=32 loops=1)
         Workers Planned: 2
         Workers Launched: 2
         Buffers: shared hit=3 read=889
         ->  Partial GroupAggregate  (cost=0.42..14318.42 rows=16 width=59)
               (actual time=1.441..21.554 rows=11 loops=3)
               Group Key: ciudad
               Buffers: shared hit=3 read=889
               ->  Parallel Index Only Scan using idx_personas_ciudad_edad on personas
                     (cost=0.42..10151.59 rows=416667 width=13) (actual time=0.024..9.326 rows=333333 loops=3)
                     Heap Fetches: 0
                     Buffers: shared hit=3 read=889
 Planning Time: 0.020 ms
 Execution Time: 33.792 ms
(15 rows)

                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Aggregate  (cost=850.10..850.12 rows=1 width=48) (actual time=1.335..1.336 rows=1 loops=1)
   Buffers: shared hit=30
   ->  Index Only Scan using idx_personas_ciudad_edad on personas  (cost=0.42..627.60 rows=29667 width=2)
         (actual time=0.005..0.673 rows=29972 loops=1)
         Index Cond: (ciudad = 'Colón'::text)
         Heap Fetches: 0
         Buffers: shared hit=30
 Planning Time: 0.016 ms
 Execution Time: 1.339 ms
(8 rows)

 tabla | indice_edad | indice_ciudad_edad
-------+-------------+--------------------
 65 MB | 6840 kB     | 7160 kB
(1 row)

         ciudad         | personas | suma_edades | edad_promedio
------------------------+----------+-------------+---------------
 Basavilbaso            |    20092 |      990260 |         49.29
 Chajarí                |    50331 |     2491440 |         49.50
 Colón                  |    29972 |     1487157 |         49.62
 Concepción del Uruguay |    99748 |     4943781 |         49.56
 Concordia              |   159664 |     7911587 |         49.55
 Crespo                 |    20110 |      993863 |         49.42
 Federación             |    20201 |     1006512 |         49.82
 Gualeguay              |    49704 |     2462179 |         49.54
 Gualeguaychú           |   110133 |     5451608 |         49.50
 La Paz                 |    30110 |     1498108 |         49.75
 Nogoyá                 |    29985 |     1475761 |         49.22
 Paraná                 |   249455 |    12340381 |         49.47
 San José               |    19888 |      980835 |         49.32
 Victoria               |    40291 |     1991740 |         49.43
 Villa Elisa            |    19839 |      982576 |         49.53
 Villaguay              |    50477 |     2494849 |         49.43
(16 rows)
```

ítem	variante	mediana ms	mín ms	máx ms	plan
c.6	Totales, sin índice	14.165	14.046	15.077	Finalize Aggregate [Gather [Partial Aggregate [Parallel Seq Scan]]]
c.6	Por ciudad, sin índice	28.269	27.358	28.962	Finalize GroupAggregate [Gather Merge [Sort [Partial HashAggregate [Parallel Seq Scan]]]]
c.6	Totales, índice (edad)	13.787	13.532	14.440	Finalize Aggregate [Gather [Partial Aggregate [Parallel Seq Scan]]]
c.6	Por ciudad, índice (edad)	29.331	28.442	30.365	Finalize GroupAggregate [Gather Merge [Sort [Partial HashAggregate [Parallel Seq Scan]]]]
c.6	Por ciudad, índice (ciudad, edad)	28.032	27.384	28.526	Finalize GroupAggregate [Gather Merge [Sort [Partial HashAggregate [Parallel Seq Scan]]]]
c.6	Totales, índices, random_page_cost 1.1	12.181	11.791	12.514	Finalize Aggregate [Gather [Partial Aggregate [Parallel Index Only Scan (idx_personas_edad)]]]
c.6	Por ciudad, índices, random_page_cost 1.1	23.727	21.721	25.682	Finalize GroupAggregate [Gather Merge [Partial GroupAggregate [Parallel Index Only Scan (idx_personas_ciudad_edad)]]]
c.6	Colón, índice (ciudad, edad)	0.845	0.820	0.861	Aggregate [Index Only Scan (idx_personas_ciudad_edad)]

Parte d. Clientes y pedidos

Las tablas se crean como las da el enunciado, con su clave primaria, la foránea de pedidos a clientes y el email único. Se cargaron 500.000 clientes y 5.000.000 de pedidos, cada uno de un cliente al azar, con fecha uniforme entre 2020 y 2025 y total entre $ 500 y $ 100.000. Como son seis años, 2024 tiene la sexta parte de los pedidos (834.878, el 16,7 %), que es el dato que define todo lo que sigue.

d.1 Carga de 500.000 clientes y 5.000.000 de pedidos

La salida muestra los tamaños (pedidos ocupa 249 MB, su clave primaria 107 MB), la distribución por año y que casi todos los clientes (499.974) tienen pedidos, unos 10 cada uno. La carga de pedidos verifica la clave foránea fila por fila, y es lo más lento del script, que en total tardó unos 30 s.

SCRIPT d00-clientes-pedidos.sql

```
SET search_path TO public;

-- d.0 / d.1) Tablas clientes y pedidos como las define el enunciado, con 500.000
--            clientes y 5.000.000 de pedidos. Cada pedido se asigna a un cliente
--            al azar, con fecha uniforme entre 2020 y 2025 (seis años, así que 2024
--            tiene alrededor de la sexta parte de los pedidos) y total entre
--            $ 500 y $ 100.000.

DROP TABLE IF EXISTS pedidos;
DROP TABLE IF EXISTS clientes;
DELETE FROM medicion WHERE item LIKE 'd.%';

CREATE TABLE clientes (
    id SERIAL PRIMARY KEY,
    nombre TEXT,
    email TEXT UNIQUE
);

CREATE TABLE pedidos (
    id SERIAL PRIMARY KEY,
    cliente_id INT REFERENCES clientes(id),
    fecha DATE,
    total NUMERIC
);

SELECT setseed(0.2026);

INSERT INTO clientes (nombre, email)
SELECT 'Cliente ' || lpad(g::TEXT, 6, '0'),
       'cliente' || g || '@correo.com.ar'
FROM generate_series(1, 500000) AS g;

INSERT INTO pedidos (cliente_id, fecha, total)
SELECT 1 + floor(random() * 500000)::INTEGER,
       DATE '2020-01-01' + floor(random() * (DATE '2026-01-01' - DATE '2020-01-01'))::INTEGER,
       ROUND((500 + random() * 99500)::NUMERIC, 2)
FROM generate_series(1, 5000000);

VACUUM ANALYZE clientes;
VACUUM ANALYZE pedidos;

SELECT c.relname                              AS relacion,
       c.relkind                              AS tipo,
       c.reltuples::BIGINT                    AS filas,
       c.relpages                             AS paginas,
       pg_size_pretty(pg_relation_size(c.oid)) AS tamanio
FROM pg_class c
WHERE c.relname IN ('clientes', 'pedidos', 'clientes_pkey', 'clientes_email_key', 'pedidos_pkey')
ORDER BY c.relkind DESC, c.relname;

SELECT EXTRACT(YEAR FROM fecha)::INTEGER AS anio,
       COUNT(*)                         AS pedidos,
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 2) AS porcentaje,
       ROUND(SUM(total), 2)             AS facturacion
FROM pedidos
GROUP BY anio
ORDER BY anio;

SELECT COUNT(DISTINCT cliente_id)                       AS clientes_con_pedidos,
       ROUND(COUNT(*)::NUMERIC / COUNT(DISTINCT cliente_id), 1) AS pedidos_por_cliente
FROM pedidos;
```

SALIDA

```
 setseed
---------
```

```
(1 row)

      relacion      | tipo |  filas  | paginas | tamanio
--------------------+------+---------+---------+---------
 clientes           | r    |  500000 |    4673 | 37 MB
 pedidos            | r    | 4999992 |   31848 | 249 MB
 clientes_email_key | i    |  500000 |    5332 | 42 MB
 clientes_pkey      | i    |  500000 |    1374 | 11 MB
 pedidos_pkey       | i    | 4999992 |   13712 | 107 MB
(5 rows)

 anio | pedidos | porcentaje |  facturacion
------+---------+------------+---------------
 2020 |  835966 |      16.72 | 41995165124.79
 2021 |  832029 |      16.64 | 41790116233.90
 2022 |  832267 |      16.65 | 41811575955.46
 2023 |  832360 |      16.65 | 41834782832.37
 2024 |  834878 |      16.70 | 41947763308.26
 2025 |  832500 |      16.65 | 41849864633.40
(6 rows)

 clientes_con_pedidos | pedidos_por_cliente
----------------------+---------------------
               499974 |                10.0
(1 row)
```

d.2 y d.3 Los diez clientes con mayor facturación en 2024

El plan de la consulta del enunciado, leído de abajo hacia arriba:

- `Parallel Seq Scan on pedidos`: los tres procesos recorren las 31.848 páginas de pedidos y se quedan con los de 2024 (cada uno descarta 1,39 millones y conserva unos 278.000). No hay índice sobre fecha, así que no hay otra forma.
- `Parallel Hash` y `Parallel Seq Scan on clientes`: en paralelo se construye una tabla hash con los 500.000 clientes. No entra en `work_mem` y se divide en 4 lotes (`Batches: 4`), que pasan por archivos temporales.
- `Parallel Hash Join`: cada pedido de 2024 se une con su cliente por `cliente_id`.
- `Sort` por `c.id` y `Partial GroupAggregate`: como el `GROUP BY` es por cliente, cada proceso ordena sus 278.000 filas por cliente (`external merge`, unos 10 MB a disco por proceso) y suma el total de cada uno.
- `Gather Merge` y `Finalize GroupAggregate`: se juntan los resultados de los tres procesos, ordenados, y se combinan las sumas parciales del mismo cliente. Quedan 405.670 clientes con pedidos en 2024.
- `Sort` (`top-N heapsort`) y `Limit`: de esos 405.670 se quedan los diez de mayor facturación sin ordenar todo; el `top-N heapsort` mantiene sólo los diez mejores mientras recorre.

Mediana: 359 ms. El `GROUP BY c.id` con `c.nombre` en el `SELECT` es válido porque `c.id` es clave primaria: el nombre depende funcionalmente de ella.

SCRIPT d01-top-clientes.sql

```
SET search_path TO public;

DELETE FROM medicion WHERE item IN ('d.2', 'd.3');
DROP INDEX IF EXISTS idx_pedidos_cliente;
DROP INDEX IF EXISTS idx_pedidos_cliente_fecha;
DROP INDEX IF EXISTS idx_pedidos_fecha_cliente;
DROP INDEX IF EXISTS idx_pedidos_fecha_cliente_total;
DROP INDEX IF EXISTS idx_pedidos_cliente_fecha_total;
ANALYZE pedidos;

-- d.2) Los 10 clientes con mayor facturación en 2024, con la consulta del
--      enunciado. pedidos sólo tiene los índices de las restricciones (clave
--      primaria de pedidos, clave primaria y email único de clientes).

SELECT c.nombre, SUM(p.total) AS facturacion
FROM clientes c
JOIN pedidos p ON c.id = p.cliente_id
WHERE p.fecha BETWEEN '2024-01-01' AND '2024-12-31'
GROUP BY c.id
ORDER BY facturacion DESC
LIMIT 10;

-- d.3) Plan de ejecución.

EXPLAIN (ANALYZE, BUFFERS)
SELECT c.nombre, SUM(p.total) AS facturacion
FROM clientes c
JOIN pedidos p ON c.id = p.cliente_id
WHERE p.fecha BETWEEN '2024-01-01' AND '2024-12-31'
GROUP BY c.id
ORDER BY facturacion DESC
LIMIT 10;

CALL pr_medir('d.3', 'Consulta del enunciado, sin índices', $$
    SELECT c.nombre, SUM(p.total) AS facturacion
    FROM clientes c JOIN pedidos p ON c.id = p.cliente_id
    WHERE p.fecha BETWEEN '2024-01-01' AND '2024-12-31'
    GROUP BY c.id ORDER BY facturacion DESC LIMIT 10
$$, 5);

SELECT item, variante, mediana_ms, min_ms, max_ms, plan
FROM medicion
WHERE item = 'd.3'
ORDER BY id;
```

SALIDA

```
     nombre     | facturacion
----------------+-------------
 Cliente 128142 |   635379.85
 Cliente 404752 |   623336.19
 Cliente 180988 |   619212.06
 Cliente 306973 |   603802.09
 Cliente 056838 |   597978.40
 Cliente 028050 |   595147.87
 Cliente 095025 |   593192.36
 Cliente 158871 |   588046.15
 Cliente 147649 |   575036.84
 Cliente 073558 |   570155.82
(10 rows)

                                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=230842.62..230842.64 rows=10 width=51) (actual time=385.784..389.217 rows=10 loops=1)
   Buffers: shared hit=14788 read=21827 written=68, temp read=9338 written=9387
   ->  Sort  (cost=230842.62..232092.62 rows=500000 width=51) (actual time=385.784..389.216 rows=10 loops=1)
         Sort Key: (sum(p.total)) DESC
         Sort Method: top-N heapsort  Memory: 25kB
         Buffers: shared hit=14788 read=21827 written=68, temp read=9338 written=9387
         ->  Finalize GroupAggregate  (cost=120807.69..220037.80 rows=500000 width=51)
               (actual time=174.158..358.783 rows=405670 loops=1)
               Group Key: c.id
               Buffers: shared hit=14788 read=21827 written=68, temp read=9338 written=9387
               ->  Gather Merge  (cost=120807.69..208541.59 rows=699494 width=51)
                     (actual time=174.156..254.646 rows=465028 loops=1)
                     Workers Planned: 2
                     Workers Launched: 2
                     Buffers: shared hit=14788 read=21827 written=68, temp read=9338 written=9387
                     ->  Partial GroupAggregate  (cost=119807.66..126802.60 rows=349747 width=51)
                           (actual time=171.649..225.358 rows=155009 loops=3)
                           Group Key: c.id
                           Buffers: shared hit=14788 read=21827 written=68, temp read=9338 written=9387
                           ->  Sort  (cost=119807.66..120682.03 rows=349747 width=27)
                                 (actual time=171.640..184.807 rows=278293 loops=3)
                                 Sort Key: c.id
                                 Sort Method: external merge  Disk: 10424kB
                                 Buffers: shared hit=14788 read=21827 written=68, temp read=9338 written=9387
                                 Worker 0:  Sort Method: external merge  Disk: 10200kB
                                 Worker 1:  Sort Method: external merge  Disk: 10312kB
                                 ->  Parallel Hash Join  (cost=10581.50..79234.54 rows=349747 width=27)
                                       (actual time=102.644..134.538 rows=278293 loops=3)
                                       Hash Cond: (p.cliente_id = c.id)
                                       Buffers: shared hit=14716 read=21827 written=68, temp read=5471 written=5508
                                       ->  Parallel Seq Scan on pedidos p
                                             (cost=0.00..63097.95 rows=349747 width=12)
                                             (actual time=0.323..57.390 rows=278293 loops=3)
                                             Filter: ((fecha >= '2024-01-01'::date) AND (fecha <=
                                                 '2024-12-31'::date))
                                             Rows Removed by Filter: 1388374
                                             Buffers: shared hit=10021 read=21827 written=68
                                       ->  Parallel Hash  (cost=6756.33..6756.33 rows=208333 width=19)
                                             (actual time=20.978..20.979 rows=166667 loops=3)
                                             Buckets: 131072  Batches: 4  Memory Usage: 7968kB
                                             Buffers: shared hit=4673, temp written=1856
                                             ->  Parallel Seq Scan on clientes c
                                                   (cost=0.00..6756.33 rows=208333 width=19)
                                                   (actual time=0.003..6.261 rows=166667 loops=3)
                                                   Buffers: shared hit=4673
 Planning:
   Buffers: shared hit=8
 Planning Time: 0.126 ms
 Execution Time: 389.647 ms
(38 rows)
```

ítem	variante	mediana ms	mín ms	máx ms	plan
d.3	Consulta del enunciado, sin índices	358.843	345.440	361.723	`Limit [Sort [Finalize GroupAggregate [Gather Merge [Partial GroupAggregate [Sort [Parallel Hash Join [Parallel Seq Scan, Parallel Hash [Parallel Seq Scan]]]]]]]]`

d.4 y d.5 Índices sobre pedidos

Se probaron los tres índices del enunciado de a uno por vez. Cuando el planificador no usa el índice, se muestra también el plan que resulta de forzarlo.

INDEX(cliente_id). El planificador no lo usa (347 ms, el mismo plan de antes). El índice no sirve para el filtro, que es por fecha. Lo único que podría aportar es el orden por cliente para agrupar, y forzándolo se ve por qué no conviene: `Merge Join` entre la clave primaria de clientes y el índice de pedidos, que recorre los 5 millones de pedidos en orden de cliente, saltando de una página de la tabla a otra para leer la fecha y el total de cada uno, y descarta 4,17 millones por fecha. Son 5 millones de accesos a páginas: 3.327 ms, más de nueve veces peor.

INDEX(cliente_id, fecha). Tampoco lo usa (352 ms). Forzándolo, el planificador lo aprovecha de una forma poco habitual: recorre el índice completo con la condición de fecha sobre la segunda columna (`Index Cond` sólo sobre fecha), arma el mapa de bits y lee la tabla. La fecha no encabeza el índice, así que no puede ir directo al tramo de 2024 y tiene que leer las 13.660 páginas del índice. 501 ms.

INDEX(fecha, cliente_id). Es el único que el planificador elige, porque la condición es sobre su primera columna: con un `Bitmap Index Scan` va directo al tramo de 2024 del índice y lee sólo 2.284 de sus páginas. Pero el resultado es peor que sin índice: 426 ms contra 359. Hay tres razones. Primero, los 834.878 pedidos de 2024 están repartidos por toda la tabla y el `Bitmap Heap Scan` igual lee sus 31.848 páginas (hay que ir a buscar el total). Segundo, el plan deja de ser paralelo. Tercero, la agregación pasa a ser un `HashAggregate` de 405.670 grupos que no entra en memoria y escribe 62 MB a disco. El planificador estimó este plan más barato (costo 187.705 contra 230.843 del original) y se equivocó.

Cuál es mejor y por qué. De los tres, el único utilizable para esta consulta es (fecha, cliente_id), porque es el único cuya primera columna coincide con la condición del `WHERE`; (cliente_id) y (cliente_id, fecha) sólo podrían aportar el orden por cliente, y a costa de recorrer los 5 millones de pedidos. Pero ninguno de los tres mejora la consulta, porque a ninguno le alcanza para responderla: falta la columna `total` y cualquier plan por índice tiene que visitar la tabla por cada pedido. Con una selectividad del 17 % y los pedidos de 2024 dispersos en todas las páginas, eso equivale a leer la tabla entera, y el recorrido secuencial en paralelo es más barato.

Para comprobarlo, en el d.5 se repitieron los dos compuestos agregando `total` con `INCLUDE`, de modo que el índice cubra la consulta:

- **(fecha, cliente_id) INCLUDE (total)**: `Index Only Scan` del tramo de 2024 (lee 4.020 páginas en lugar de las 31.848 de la tabla), pero los pedidos salen ordenados por fecha y hay que agruparlos con el mismo `HashAggregate` que derrama a disco. 375 ms: tampoco mejora.
- **(cliente_id, fecha) INCLUDE (total)**: recorre el índice entero (24.057 páginas), pero sin tocar la tabla y entregando los pedidos ordenados por cliente. El agrupamiento pasa a ser un `GroupAggregate` que suma mientras recorre, sin tabla hash, sin `Sort` y sin disco, y el join con clientes es un `Merge Join` contra su clave primaria, que tiene el mismo orden. 230 ms, un 36 % menos que sin índices.

El índice que gana es el que entrega los datos en el orden del `GROUP BY`, no el que lee menos: en esta consulta el costo dominante es agrupar 835.000 pedidos en 405.670 clientes, y el orden por cliente lo elimina.

Reescritura. El nombre del cliente sólo hace falta para los diez primeros, pero la consulta del enunciado une los 835.000 pedidos con los 500.000 clientes antes de agrupar. Calculando el ranking sobre pedidos solos y buscando después los diez nombres por clave primaria, el plan con el índice (cliente_id, fecha) INCLUDE (total) queda en `Index Only Scan`, `GroupAggregate`, top-N y un `Nested Loop` de diez búsquedas en clientes: 165 ms. Sin índices la reescritura sola da 329 ms. Combinadas, el índice cubriente y la reescritura dejan la consulta en menos de la mitad del tiempo original (359 a 165 ms).

El índice tiene su precio: ocupa 189 MB, tres cuartos de la tabla, y hay que mantenerlo en cada `INSERT` de pedidos. Si este reporte se pidiera muchas veces por día, una vista materializada con la facturación anual por cliente, como en la parte b, sería todavía más rápida.

SCRIPT d02-indices-pedidos.sql

```
SET search_path TO public;

DELETE FROM medicion WHERE item IN ('d.4', 'd.5');
DROP INDEX IF EXISTS idx_pedidos_cliente;
```

```
DROP INDEX IF EXISTS idx_pedidos_cliente_fecha;
DROP INDEX IF EXISTS idx_pedidos_fecha_cliente;
DROP INDEX IF EXISTS idx_pedidos_fecha_cliente_total;
DROP INDEX IF EXISTS idx_pedidos_cliente_fecha_total;

-- d.4) Se prueban los tres índices del enunciado de a uno por vez. Para cada uno:
--      el plan que elige el planificador y, si no lo usa, el plan y el tiempo que
--      se obtienen obligándolo (enable_seqscan = off encarece artificialmente el
--      Seq Scan y fuerza a buscar un camino por índice).

-- 1. INDEX(cliente_id)

CREATE INDEX idx_pedidos_cliente ON pedidos (cliente_id);
VACUUM ANALYZE pedidos;

EXPLAIN (ANALYZE, BUFFERS)
SELECT c.nombre, SUM(p.total) AS facturacion
FROM clientes c
JOIN pedidos p ON c.id = p.cliente_id
WHERE p.fecha BETWEEN '2024-01-01' AND '2024-12-31'
GROUP BY c.id
ORDER BY facturacion DESC
LIMIT 10;

CALL pr_medir('d.4', '(cliente_id), plan elegido', $$
    SELECT c.nombre, SUM(p.total) AS facturacion
    FROM clientes c JOIN pedidos p ON c.id = p.cliente_id
    WHERE p.fecha BETWEEN '2024-01-01' AND '2024-12-31'
    GROUP BY c.id ORDER BY facturacion DESC LIMIT 10
$$, 5);

SET enable_seqscan = off;

EXPLAIN (ANALYZE, BUFFERS)
SELECT c.nombre, SUM(p.total) AS facturacion
FROM clientes c
JOIN pedidos p ON c.id = p.cliente_id
WHERE p.fecha BETWEEN '2024-01-01' AND '2024-12-31'
GROUP BY c.id
ORDER BY facturacion DESC
LIMIT 10;

CALL pr_medir('d.4', '(cliente_id), forzado', $$
    SELECT c.nombre, SUM(p.total) AS facturacion
    FROM clientes c JOIN pedidos p ON c.id = p.cliente_id
    WHERE p.fecha BETWEEN '2024-01-01' AND '2024-12-31'
    GROUP BY c.id ORDER BY facturacion DESC LIMIT 10
$$, 3);

RESET enable_seqscan;
DROP INDEX idx_pedidos_cliente;

-- 2. INDEX(cliente_id, fecha)

CREATE INDEX idx_pedidos_cliente_fecha ON pedidos (cliente_id, fecha);
VACUUM ANALYZE pedidos;

EXPLAIN (ANALYZE, BUFFERS)
SELECT c.nombre, SUM(p.total) AS facturacion
FROM clientes c
JOIN pedidos p ON c.id = p.cliente_id
WHERE p.fecha BETWEEN '2024-01-01' AND '2024-12-31'
GROUP BY c.id
ORDER BY facturacion DESC
LIMIT 10;

CALL pr_medir('d.4', '(cliente_id, fecha), plan elegido', $$
    SELECT c.nombre, SUM(p.total) AS facturacion
    FROM clientes c JOIN pedidos p ON c.id = p.cliente_id
    WHERE p.fecha BETWEEN '2024-01-01' AND '2024-12-31'
    GROUP BY c.id ORDER BY facturacion DESC LIMIT 10
$$, 5);

SET enable_seqscan = off;

EXPLAIN (ANALYZE, BUFFERS)
SELECT c.nombre, SUM(p.total) AS facturacion
FROM clientes c
```

```
JOIN pedidos p ON c.id = p.cliente_id
WHERE p.fecha BETWEEN '2024-01-01' AND '2024-12-31'
GROUP BY c.id
ORDER BY facturacion DESC
LIMIT 10;

CALL pr_medir('d.4', '(cliente_id, fecha), forzado', $$
    SELECT c.nombre, SUM(p.total) AS facturacion
    FROM clientes c JOIN pedidos p ON c.id = p.cliente_id
    WHERE p.fecha BETWEEN '2024-01-01' AND '2024-12-31'
    GROUP BY c.id ORDER BY facturacion DESC LIMIT 10
$$, 5);

RESET enable_seqscan;
DROP INDEX idx_pedidos_cliente_fecha;

-- 3. INDEX(fecha, cliente_id)

CREATE INDEX idx_pedidos_fecha_cliente ON pedidos (fecha, cliente_id);
VACUUM ANALYZE pedidos;

EXPLAIN (ANALYZE, BUFFERS)
SELECT c.nombre, SUM(p.total) AS facturacion
FROM clientes c
JOIN pedidos p ON c.id = p.cliente_id
WHERE p.fecha BETWEEN '2024-01-01' AND '2024-12-31'
GROUP BY c.id
ORDER BY facturacion DESC
LIMIT 10;

CALL pr_medir('d.4', '(fecha, cliente_id), plan elegido', $$
    SELECT c.nombre, SUM(p.total) AS facturacion
    FROM clientes c JOIN pedidos p ON c.id = p.cliente_id
    WHERE p.fecha BETWEEN '2024-01-01' AND '2024-12-31'
    GROUP BY c.id ORDER BY facturacion DESC LIMIT 10
$$, 5);

SELECT c.relname AS indice, pg_size_pretty(pg_relation_size(c.oid)) AS tamanio
FROM pg_class c
WHERE c.relname IN ('pedidos', 'idx_pedidos_fecha_cliente');

DROP INDEX idx_pedidos_fecha_cliente;

-- d.5) Ninguno de los tres mejora la consulta tal como están. Les falta la columna
--      total, así que cualquier plan por índice tiene que visitar la tabla por cada
--      pedido de 2024. Se repiten los dos índices compuestos agregando total con
--      INCLUDE, para que el índice cubra la consulta (Index Only Scan).

-- Antes, la reescritura que se usa más abajo, todavía sin índices sobre pedidos,
-- para separar lo que aporta cada cambio.

CALL pr_medir('d.5', 'Reescrita, sin índices', $$
    WITH ranking AS (
        SELECT p.cliente_id, SUM(p.total) AS facturacion
        FROM pedidos p
        WHERE p.fecha BETWEEN '2024-01-01' AND '2024-12-31'
        GROUP BY p.cliente_id ORDER BY facturacion DESC LIMIT 10
    )
    SELECT c.nombre, r.facturacion
    FROM ranking r JOIN clientes c ON c.id = r.cliente_id
    ORDER BY r.facturacion DESC
$$, 5);

CREATE INDEX idx_pedidos_fecha_cliente_total ON pedidos (fecha, cliente_id) INCLUDE (total);
VACUUM ANALYZE pedidos;

EXPLAIN (ANALYZE, BUFFERS)
SELECT c.nombre, SUM(p.total) AS facturacion
FROM clientes c
JOIN pedidos p ON c.id = p.cliente_id
WHERE p.fecha BETWEEN '2024-01-01' AND '2024-12-31'
GROUP BY c.id
ORDER BY facturacion DESC
LIMIT 10;

CALL pr_medir('d.5', '(fecha, cliente_id) INCLUDE (total)', $$
    SELECT c.nombre, SUM(p.total) AS facturacion
    FROM clientes c JOIN pedidos p ON c.id = p.cliente_id
```

```
    WHERE p.fecha BETWEEN '2024-01-01' AND '2024-12-31'
    GROUP BY c.id ORDER BY facturacion DESC LIMIT 10
$$, 5);

DROP INDEX idx_pedidos_fecha_cliente_total;

CREATE INDEX idx_pedidos_cliente_fecha_total ON pedidos (cliente_id, fecha) INCLUDE (total);
VACUUM ANALYZE pedidos;

EXPLAIN (ANALYZE, BUFFERS)
SELECT c.nombre, SUM(p.total) AS facturacion
FROM clientes c
JOIN pedidos p ON c.id = p.cliente_id
WHERE p.fecha BETWEEN '2024-01-01' AND '2024-12-31'
GROUP BY c.id
ORDER BY facturacion DESC
LIMIT 10;

CALL pr_medir('d.5', '(cliente_id, fecha) INCLUDE (total)', $$
    SELECT c.nombre, SUM(p.total) AS facturacion
    FROM clientes c JOIN pedidos p ON c.id = p.cliente_id
    WHERE p.fecha BETWEEN '2024-01-01' AND '2024-12-31'
    GROUP BY c.id ORDER BY facturacion DESC LIMIT 10
$$, 5);

-- d.5) Reescritura con el índice anterior: el nombre del cliente sólo hace falta
--      para los diez primeros. Se calcula el ranking sobre pedidos solo y después
--      se buscan esos diez clientes por clave primaria, en lugar de unir los
--      835.000 pedidos de 2024 con los 500.000 clientes.

EXPLAIN (ANALYZE, BUFFERS)
WITH ranking AS (
    SELECT p.cliente_id, SUM(p.total) AS facturacion
    FROM pedidos p
    WHERE p.fecha BETWEEN '2024-01-01' AND '2024-12-31'
    GROUP BY p.cliente_id
    ORDER BY facturacion DESC
    LIMIT 10
)
SELECT c.nombre, r.facturacion
FROM ranking r
JOIN clientes c ON c.id = r.cliente_id
ORDER BY r.facturacion DESC;

CALL pr_medir('d.5', 'Reescrita + (cliente_id, fecha) INCLUDE (total)', $$
    WITH ranking AS (
        SELECT p.cliente_id, SUM(p.total) AS facturacion
        FROM pedidos p
        WHERE p.fecha BETWEEN '2024-01-01' AND '2024-12-31'
        GROUP BY p.cliente_id ORDER BY facturacion DESC LIMIT 10
    )
    SELECT c.nombre, r.facturacion
    FROM ranking r JOIN clientes c ON c.id = r.cliente_id
    ORDER BY r.facturacion DESC
$$, 5);

SELECT c.relname AS indice, pg_size_pretty(pg_relation_size(c.oid)) AS tamanio
FROM pg_class c
WHERE c.relname IN ('pedidos', 'idx_pedidos_cliente_fecha_total');

WITH ranking AS (
    SELECT p.cliente_id, SUM(p.total) AS facturacion
    FROM pedidos p
    WHERE p.fecha BETWEEN '2024-01-01' AND '2024-12-31'
    GROUP BY p.cliente_id
    ORDER BY facturacion DESC
    LIMIT 10
)
SELECT c.nombre, r.facturacion
FROM ranking r
JOIN clientes c ON c.id = r.cliente_id
ORDER BY r.facturacion DESC;

SELECT item, variante, mediana_ms, min_ms, max_ms, plan
FROM medicion
WHERE item IN ('d.3', 'd.4', 'd.5')
ORDER BY id;
```

SALIDA

```
                                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=228324.05..228324.07 rows=10 width=51) (actual time=383.353..384.751 rows=10 loops=1)
   Buffers: shared hit=15780 read=20835, temp read=9336 written=9376
   ->  Sort  (cost=228324.05..229574.05 rows=500000 width=51) (actual time=383.352..384.750 rows=10 loops=1)
         Sort Key: (sum(p.total)) DESC
         Sort Method: top-N heapsort  Memory: 25kB
         Buffers: shared hit=15780 read=20835, temp read=9336 written=9376
         ->  Finalize GroupAggregate  (cost=119957.85..217519.23 rows=500000 width=51)
               (actual time=170.195..354.008 rows=405670 loops=1)
               Group Key: c.id
               Buffers: shared hit=15780 read=20835, temp read=9336 written=9376
               ->  Gather Merge  (cost=119957.85..206117.18 rows=686940 width=51)
                     (actual time=170.192..248.358 rows=466182 loops=1)
                     Workers Planned: 2
                     Workers Launched: 2
                     Buffers: shared hit=15780 read=20835, temp read=9336 written=9376
                     ->  Partial GroupAggregate  (cost=118957.83..125827.23 rows=343470 width=51)
                           (actual time=167.131..220.716 rows=155394 loops=3)
                           Group Key: c.id
                           Buffers: shared hit=15780 read=20835, temp read=9336 written=9376
                           ->  Sort  (cost=118957.83..119816.51 rows=343470 width=27)
                                 (actual time=167.121..179.958 rows=278293 loops=3)
                                 Sort Key: c.id
                                 Sort Method: external merge  Disk: 10280kB
                                 Buffers: shared hit=15780 read=20835, temp read=9336 written=9376
                                 Worker 0:  Sort Method: external merge  Disk: 10424kB
                                 Worker 1:  Sort Method: external merge  Disk: 10240kB
                                 ->  Parallel Hash Join  (cost=10581.50..79158.07 rows=343470 width=27)
                                       (actual time=95.509..129.328 rows=278293 loops=3)
                                       Hash Cond: (p.cliente_id = c.id)
                                       Buffers: shared hit=15708 read=20835, temp read=5468 written=5496
                                       ->  Parallel Seq Scan on pedidos p
                                             (cost=0.00..63097.95 rows=343470 width=12)
                                             (actual time=0.031..54.550 rows=278293 loops=3)
                                             Filter: ((fecha >= '2024-01-01'::date) AND (fecha <=
                                                 '2024-12-31'::date))
                                             Rows Removed by Filter: 1388374
                                             Buffers: shared hit=11013 read=20835
                                       ->  Parallel Hash  (cost=6756.33..6756.33 rows=208333 width=19)
                                             (actual time=17.694..17.694 rows=166667 loops=3)
                                             Buckets: 131072  Batches: 4  Memory Usage: 7936kB
                                             Buffers: shared hit=4673, temp written=1852
                                             ->  Parallel Seq Scan on clientes c
                                                   (cost=0.00..6756.33 rows=208333 width=19)
                                                   (actual time=0.003..6.353 rows=166667 loops=3)
                                                   Buffers: shared hit=4673
 Planning:
   Buffers: shared hit=129 read=5
 Planning Time: 0.279 ms
 Execution Time: 385.138 ms
(38 rows)

                                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=300458.36..300458.38 rows=10 width=51) (actual time=3474.297..3474.298 rows=10 loops=1)
   Buffers: shared hit=2549334 read=2461572 written=11640
   ->  Sort  (cost=300458.36..301708.36 rows=500000 width=51) (actual time=3474.297..3474.297 rows=10 loops=1)
         Sort Key: (sum(p.total)) DESC
         Sort Method: top-N heapsort  Memory: 25kB
         Buffers: shared hit=2549334 read=2461572 written=11640
         ->  GroupAggregate  (cost=19.82..289653.54 rows=500000 width=51)
               (actual time=0.025..3447.517 rows=405670 loops=1)
               Group Key: c.id
               Buffers: shared hit=2549334 read=2461572 written=11640
               ->  Merge Join  (cost=19.82..279281.90 rows=824327 width=27)
                     (actual time=0.017..3351.159 rows=834878 loops=1)
                     Merge Cond: (c.id = p.cliente_id)
                     Buffers: shared hit=2549334 read=2461572 written=11640
                     ->  Index Scan using clientes_pkey on clientes c
                           (cost=0.42..17672.42 rows=500000 width=19)
                           (actual time=0.002..33.815 rows=500000 loops=1)
                           Buffers: shared hit=7 read=6035 written=27
                     ->  Index Scan using idx_pedidos_cliente on pedidos p
                           (cost=0.43..250066.40 rows=824327 width=12)
                           (actual time=0.013..3246.149 rows=834878 loops=1)
                           Filter: ((fecha >= '2024-01-01'::date) AND (fecha <= '2024-12-31'::date))
                           Rows Removed by Filter: 4165122
                           Buffers: shared hit=2549327 read=2455537 written=11613
 Planning:
```

```
   Buffers: shared hit=16
 Planning Time: 0.067 ms
 Execution Time: 3474.306 ms
(22 rows)

                                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=230036.47..230036.50 rows=10 width=51) (actual time=384.582..386.076 rows=10 loops=1)
   Buffers: shared hit=16108 read=20507, temp read=9338 written=9379
   ->  Sort  (cost=230036.47..231286.47 rows=500000 width=51) (actual time=384.582..386.075 rows=10 loops=1)
         Sort Key: (sum(p.total)) DESC
         Sort Method: top-N heapsort  Memory: 25kB
         Buffers: shared hit=16108 read=20507, temp read=9338 written=9379
         ->  Finalize GroupAggregate  (cost=120536.97..219231.65 rows=500000 width=51)
               (actual time=176.844..352.545 rows=405670 loops=1)
               Group Key: c.id
               Buffers: shared hit=16108 read=20507, temp read=9338 written=9379
               ->  Gather Merge  (cost=120536.97..207765.66 rows=695466 width=51)
                     (actual time=176.841..254.031 rows=469473 loops=1)
                     Workers Planned: 2
                     Workers Launched: 2
                     Buffers: shared hit=16108 read=20507, temp read=9338 written=9379
                     ->  Partial GroupAggregate  (cost=119536.94..126491.60 rows=347733 width=51)
                           (actual time=175.018..227.874 rows=156491 loops=3)
                           Group Key: c.id
                           Buffers: shared hit=16108 read=20507, temp read=9338 written=9379
                           ->  Sort  (cost=119536.94..120406.27 rows=347733 width=27)
                                 (actual time=175.007..187.643 rows=278293 loops=3)
                                 Sort Key: c.id
                                 Sort Method: external merge  Disk: 10288kB
                                 Buffers: shared hit=16108 read=20507, temp read=9338 written=9379
                                 Worker 0:  Sort Method: external merge  Disk: 10272kB
                                 Worker 1:  Sort Method: external merge  Disk: 10376kB
                                 ->  Parallel Hash Join  (cost=10581.50..79209.26 rows=347733 width=27)
                                       (actual time=106.481..138.047 rows=278293 loops=3)
                                       Hash Cond: (p.cliente_id = c.id)
                                       Buffers: shared hit=16041 read=20502, temp read=5471 written=5500
                                       ->  Parallel Seq Scan on pedidos p
                                             (cost=0.00..63097.95 rows=347733 width=12)
                                             (actual time=0.038..67.407 rows=278293 loops=3)
                                             Filter: ((fecha >= '2024-01-01'::date) AND (fecha <=
                                                 '2024-12-31'::date))
                                             Rows Removed by Filter: 1388374
                                             Buffers: shared hit=16014 read=15834
                                       ->  Parallel Hash  (cost=6756.33..6756.33 rows=208333 width=19)
                                             (actual time=18.344..18.344 rows=166667 loops=3)
                                             Buckets: 131072  Batches: 4  Memory Usage: 7904kB
                                             Buffers: shared hit=8 read=4665, temp written=1848
                                             ->  Parallel Seq Scan on clientes c
                                                   (cost=0.00..6756.33 rows=208333 width=19)
                                                   (actual time=0.016..6.272 rows=166667 loops=3)
                                                   Buffers: shared hit=8 read=4665
 Planning:
   Buffers: shared hit=28 read=16
 Planning Time: 0.284 ms
 Execution Time: 386.419 ms
(38 rows)

                                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=284515.27..284515.29 rows=10 width=51) (actual time=558.243..558.245 rows=10 loops=1)
   Buffers: shared hit=670 read=50883 written=9, temp read=9185 written=16796
   ->  Sort  (cost=284515.27..285765.27 rows=500000 width=51) (actual time=558.243..558.244 rows=10 loops=1)
         Sort Key: (sum(p.total)) DESC
         Sort Method: top-N heapsort  Memory: 26kB
         Buffers: shared hit=670 read=50883 written=9, temp read=9185 written=16796
         ->  HashAggregate  (cost=256050.46..273710.45 rows=500000 width=51)
               (actual time=425.866..534.107 rows=405670 loops=1)
               Group Key: c.id
               Planned Partitions: 32  Batches: 33  Memory Usage: 8209kB  Disk Usage: 62632kB
               Buffers: shared hit=670 read=50883 written=9, temp read=9185 written=16796
               ->  Hash Join  (cost=131909.41..189546.54 rows=834559 width=27)
                     (actual time=133.986..326.077 rows=834878 loops=1)
                     Hash Cond: (p.cliente_id = c.id)
                     Buffers: shared hit=670 read=50883 written=9, temp read=4554 written=4554
                     ->  Bitmap Heap Scan on pedidos p  (cost=105056.99..149423.38 rows=834559 width=12)
                           (actual time=73.287..141.922 rows=834878 loops=1)
                           Recheck Cond: ((fecha >= '2024-01-01'::date) AND (fecha <= '2024-12-31'::date))
                           Heap Blocks: exact=31848
                           Buffers: shared hit=3 read=45508 written=9
                           ->  Bitmap Index Scan on idx_pedidos_cliente_fecha
                                 (cost=0.00..104848.35 rows=834559 width=0)
                                 (actual time=71.053..71.053 rows=834878 loops=1)
```

```
                                Index Cond: ((fecha >= '2024-01-01'::date) AND (fecha <= '2024-12-31'::date))
                                Buffers: shared hit=3 read=13660 written=3
                     ->  Hash  (cost=17672.42..17672.42 rows=500000 width=19)
                           (actual time=60.691..60.691 rows=500000 loops=1)
                           Buckets: 131072  Batches: 4  Memory Usage: 7382kB
                           Buffers: shared hit=667 read=5375, temp written=1829
                           ->  Index Scan using clientes_pkey on clientes c
                                 (cost=0.42..17672.42 rows=500000 width=19)
                                 (actual time=0.010..31.483 rows=500000 loops=1)
                                 Buffers: shared hit=667 read=5375
 Planning:
   Buffers: shared hit=16
 Planning Time: 0.072 ms
 Execution Time: 563.267 ms
(29 rows)

                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=187704.80..187704.82 rows=10 width=51) (actual time=457.609..457.610 rows=10 loops=1)
   Buffers: shared read=38805 written=7, temp read=9185 written=16796
   ->  Sort  (cost=187704.80..188954.80 rows=500000 width=51) (actual time=457.608..457.609 rows=10 loops=1)
         Sort Key: (sum(p.total)) DESC
         Sort Method: top-N heapsort  Memory: 26kB
         Buffers: shared read=38805 written=7, temp read=9185 written=16796
         ->  HashAggregate  (cost=159381.26..176899.98 rows=500000 width=51)
               (actual time=335.575..436.540 rows=405670 loops=1)
               Group Key: c.id
               Planned Partitions: 32  Batches: 33  Memory Usage: 8209kB  Disk Usage: 62632kB
               Buffers: shared read=38805 written=7, temp read=9185 written=16796
               ->  Hash Join  (cost=36345.75..93700.75 rows=824226 width=27)
                     (actual time=61.360..243.179 rows=834878 loops=1)
                     Hash Cond: (p.cliente_id = c.id)
                     Buffers: shared read=38805 written=7, temp read=4554 written=4554
                     ->  Bitmap Heap Scan on pedidos p  (cost=17492.75..61704.14 rows=824226 width=12)
                           (actual time=19.691..87.876 rows=834878 loops=1)
                           Recheck Cond: ((fecha >= '2024-01-01'::date) AND (fecha <= '2024-12-31'::date))
                           Heap Blocks: exact=31848
                           Buffers: shared read=34132 written=7
                           ->  Bitmap Index Scan on idx_pedidos_fecha_cliente
                                 (cost=0.00..17286.69 rows=824226 width=0)
                                 (actual time=17.527..17.527 rows=834878 loops=1)
                                 Index Cond: ((fecha >= '2024-01-01'::date) AND (fecha <= '2024-12-31'::date))
                                 Buffers: shared read=2284 written=4
                     ->  Hash  (cost=9673.00..9673.00 rows=500000 width=19)
                           (actual time=41.663..41.663 rows=500000 loops=1)
                           Buckets: 131072  Batches: 4  Memory Usage: 7382kB
                           Buffers: shared read=4673, temp written=1829
                           ->  Seq Scan on clientes c  (cost=0.00..9673.00 rows=500000 width=19)
                                 (actual time=0.004..17.121 rows=500000 loops=1)
                                 Buffers: shared read=4673
 Planning:
   Buffers: shared hit=24 read=6
 Planning Time: 0.128 ms
 Execution Time: 462.464 ms
(29 rows)

          indice           | tamanio
---------------------------+---------
 idx_pedidos_fecha_cliente | 107 MB
 pedidos                   | 249 MB
(2 rows)

                                                                                                QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=160382.62..160382.65 rows=10 width=51) (actual time=408.007..408.008 rows=10 loops=1)
   Buffers: shared hit=11 read=8683, temp read=9186 written=16793
   ->  Sort  (cost=160382.62..161632.62 rows=500000 width=51) (actual time=408.006..408.007 rows=10 loops=1)
         Sort Key: (sum(p.total)) DESC
         Sort Method: top-N heapsort  Memory: 26kB
         Buffers: shared hit=11 read=8683, temp read=9186 written=16793
         ->  HashAggregate  (cost=131872.96..149577.80 rows=500000 width=51)
               (actual time=285.959..386.631 rows=405670 loops=1)
               Group Key: c.id
               Planned Partitions: 32  Batches: 33  Memory Usage: 8209kB  Disk Usage: 62632kB
               Buffers: shared hit=11 read=8683, temp read=9186 written=16793
               ->  Hash Join  (cost=18853.43..65107.58 rows=837840 width=27)
                     (actual time=41.786..191.810 rows=834878 loops=1)
                     Hash Cond: (p.cliente_id = c.id)
                     Buffers: shared hit=11 read=8683, temp read=4554 written=4554
                     ->  Index Only Scan using idx_pedidos_fecha_cliente_total on pedidos p
                           (cost=0.43..32941.23 rows=837840 width=12)
                           (actual time=0.021..37.518 rows=834878 loops=1)
                           Index Cond: ((fecha >= '2024-01-01'::date) AND (fecha <= '2024-12-31'::date))
```

```
                        Heap Fetches: 0
                        Buffers: shared hit=1 read=4020
                  ->  Hash  (cost=9673.00..9673.00 rows=500000 width=19)
                        (actual time=41.652..41.652 rows=500000 loops=1)
                        Buckets: 131072  Batches: 4  Memory Usage: 7382kB
                        Buffers: shared hit=10 read=4663, temp written=1829
                        ->  Seq Scan on clientes c  (cost=0.00..9673.00 rows=500000 width=19)
                              (actual time=0.003..17.109 rows=500000 loops=1)
                              Buffers: shared hit=10 read=4663
 Planning:
   Buffers: shared hit=26 read=4
 Planning Time: 0.122 ms
 Execution Time: 412.857 ms
(26 rows)

                                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=205281.09..205281.11 rows=10 width=51) (actual time=273.707..273.708 rows=10 loops=1)
   Buffers: shared hit=16 read=30087
   ->  Sort  (cost=205281.09..206531.09 rows=500000 width=51) (actual time=273.706..273.707 rows=10 loops=1)
         Sort Key: (sum(p.total)) DESC
         Sort Method: top-N heapsort  Memory: 25kB
         Buffers: shared hit=16 read=30087
         ->  GroupAggregate  (cost=13.00..194476.27 rows=500000 width=51)
               (actual time=0.012..250.868 rows=405670 loops=1)
               Group Key: c.id
               Buffers: shared hit=16 read=30087
               ->  Merge Join  (cost=13.00..184093.38 rows=826578 width=27)
                     (actual time=0.009..174.429 rows=834878 loops=1)
                     Merge Cond: (c.id = p.cliente_id)
                     Buffers: shared hit=16 read=30087
                     ->  Index Scan using clientes_pkey on clientes c
                           (cost=0.42..17672.42 rows=500000 width=19)
                           (actual time=0.004..27.904 rows=500000 loops=1)
                           Buffers: shared hit=12 read=6030
                     ->  Index Only Scan using idx_pedidos_cliente_fecha_total on pedidos p
                           (cost=0.43..154846.13 rows=826578 width=12)
                           (actual time=0.003..93.485 rows=834878 loops=1)
                           Index Cond: ((fecha >= '2024-01-01'::date) AND (fecha <= '2024-12-31'::date))
                           Heap Fetches: 0
                           Buffers: shared hit=4 read=24057
 Planning:
   Buffers: shared hit=33 read=5
 Planning Time: 0.147 ms
 Execution Time: 273.718 ms
(22 rows)

                                                                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Nested Loop  (cost=172113.74..172197.84 rows=10 width=47) (actual time=181.169..181.197 rows=10 loops=1)
   Buffers: shared hit=17 read=24084 written=1
   ->  Limit  (cost=172113.31..172113.34 rows=10 width=36) (actual time=181.155..181.156 rows=10 loops=1)
         Buffers: shared hit=2 read=24059 written=1
         ->  Sort  (cost=172113.31..173075.97 rows=385061 width=36) (actual time=181.155..181.155 rows=10 loops=1)
               Sort Key: (sum(p.total)) DESC
               Sort Method: top-N heapsort  Memory: 25kB
               Buffers: shared hit=2 read=24059 written=1
               ->  GroupAggregate  (cost=0.43..163792.29 rows=385061 width=36)
                     (actual time=0.008..164.421 rows=405670 loops=1)
                     Group Key: p.cliente_id
                     Buffers: shared hit=2 read=24059 written=1
                     ->  Index Only Scan using idx_pedidos_cliente_fecha_total on pedidos p
                           (cost=0.43..154846.13 rows=826578 width=12)
                           (actual time=0.007..92.457 rows=834878 loops=1)
                           Index Cond: ((fecha >= '2024-01-01'::date) AND (fecha <= '2024-12-31'::date))
                           Heap Fetches: 0
                           Buffers: shared hit=2 read=24059 written=1
   ->  Index Scan using clientes_pkey on clientes c  (cost=0.42..8.44 rows=1 width=19)
         (actual time=0.004..0.004 rows=1 loops=10)
         Index Cond: (id = p.cliente_id)
         Buffers: shared hit=15 read=25
 Planning Time: 0.049 ms
 Execution Time: 181.204 ms
(20 rows)

             indice              | tamanio
---------------------------------+---------
 idx_pedidos_cliente_fecha_total | 189 MB
 pedidos                         | 249 MB
(2 rows)

     nombre     | facturacion
----------------+-------------
```

```
 Cliente 128142 |   635379.85
 Cliente 404752 |   623336.19
 Cliente 180988 |   619212.06
 Cliente 306973 |   603802.09
 Cliente 056838 |   597978.40
 Cliente 028050 |   595147.87
 Cliente 095025 |   593192.36
 Cliente 158871 |   588046.15
 Cliente 147649 |   575036.84
 Cliente 073558 |   570155.82
(10 rows)
```

ítem	variante	mediana ms	mín ms	máx ms	plan
d.3	Consulta del enunciado, sin índices	358.843	345.440	361.723	Limit [Sort [Finalize GroupAggregate [Gather Merge [Partial GroupAggregate [Sort [Parallel Hash Join [Parallel Seq Scan, Parallel Hash [Parallel Seq Scan]]]]]]]
d.4	(cliente_id), plan elegido	347.252	338.317	364.472	Limit [Sort [Finalize GroupAggregate [Gather Merge [Partial GroupAggregate [Sort [Parallel Hash Join [Parallel Seq Scan, Parallel Hash [Parallel Seq Scan]]]]]]]
d.4	(cliente_id), forzado	3327.219	3322.628	3348.205	Limit [Sort [GroupAggregate [Merge Join [Index Scan (clientes_pkey), Index Scan (idx_pedidos_cliente)]]]]
d.4	(cliente_id, fecha), plan elegido	352.115	347.695	361.393	Limit [Sort [Finalize GroupAggregate [Gather Merge [Partial GroupAggregate [Sort [Parallel Hash Join [Parallel Seq Scan, Parallel Hash [Parallel Seq Scan]]]]]]]
d.4	(cliente_id, fecha), forzado	500.937	497.117	503.265	Limit [Sort [HashAggregate [Hash Join [Bitmap Heap Scan [Bitmap Index Scan (idx_pedidos_cliente_fecha)], Hash [Index Scan (clientes_pkey)]]]]]
d.4	(fecha, cliente_id), plan elegido	425.860	425.185	428.365	Limit [Sort [HashAggregate [Hash Join [Bitmap Heap Scan [Bitmap Index Scan (idx_pedidos_fecha_cliente)], Hash [Seq Scan]]]]]
d.5	Reescrita, sin índices	328.669	328.304	330.493	Nested Loop [Limit [Sort [HashAggregate [Seq Scan]]], Index Scan (clientes_pkey)]
d.5	(fecha, cliente_id) INCLUDE (total)	375.049	373.723	376.012	Limit [Sort [HashAggregate [Hash Join [Index Only Scan (idx_pedidos_fecha_cliente_total), Hash [Seq Scan]]]]]
d.5	(cliente_id, fecha) INCLUDE (total)	229.771	229.375	230.872	Limit [Sort [GroupAggregate [Merge Join [Index Scan (clientes_pkey), Index Only Scan (idx_pedidos_cliente_fecha_total)]]]]
d.5	Reescrita + (cliente_id, fecha) INCLUDE (total)	164.907	164.540	165.536	Nested Loop [Limit [Sort [GroupAggregate [Index Only Scan (idx_pedidos_cliente_fecha_total)]]], Index Scan (clientes_pkey)]

Resumen de resultados

Mediana del tiempo de ejecución de la versión inicial de cada consulta y de la mejor variante encontrada. En la parte a las mejoras relativas son grandes pero los tiempos absolutos son de microsegundos, por el tamaño de la tabla.

Ítem	Consulta	Inicial	Mejor	Qué la mejoró
a.1-a.2	Comercios de calle Mitre	0,070 ms	0,008 ms	Índice hash sobre calle
a.3-a.5	Comercios por producto principal	0,162 ms	0,083 ms	B-Tree: Index Only Scan y GroupAggregate (el hash no sirve)
a.6-a.7	Comercios del barrio 1	0,077 ms	0,003 ms	Índice hash sobre barrio
a.8	Kioscos del Centro en Galarza	0,084 ms	0,005 ms	Códigos literales y B-Tree (calle, clanae5)
a.9	Pares cercanos del mismo rubro	4,42 ms	1,22 ms	B-Tree (clanae5, calle, numero) con random_page_cost 1.1
a.10-a.11	Calle 8 de junio, CLANAE 47119	0,008 ms	0,004 ms	B-Tree (calle, clanae5)
a.12.1	Superficie sobre el promedio del rubro	142 ms	0,36 ms	CTE con GROUP BY en vez de subconsulta correlacionada
a.12.2	Top 10 barrio y rubro	0,459 ms	0,326 ms	Agrupar por códigos antes del join
a.12.3	Rubros sin comercios	0,130 ms	0,115 ms	NOT EXISTS (y es la forma correcta ante NULL)
b.1	Artículos del rubro 5	7,63 ms	0,444 ms	B-Tree sobre rubro y CLUSTER
b.2	Precio actual del artículo 10	66,3 ms	0,003 ms	(id_articulo, fecha DESC) INCLUDE (precio)
b.3	Precio actual y descripción	7,70 ms	0,004 ms	Clave primaria en articulos
b.4	Artículos con precios entre 1000 y 1500	326 ms	107 ms	EXISTS e índice (precio, id_articulo)
b.5	Precio promedio por artículo	663 ms	2,6 ms	Vista materializada
b.6	Precio promedio por rubro	636 ms	16,2 ms	Vista materializada (405 ms con work_mem 64MB)
b.7	Máximo del rubro 15 desde abril 2023	134 ms	0,050 ms	ORDER BY precio DESC LIMIT 1 sobre índice
b.8	Máximo promedio por rubro	642 ms	17,3 ms	Vista materializada
b.9	Máximo promedio por artículo	508 ms	0,003 ms	Vista materializada con índice sobre el promedio
c.1-c.3	Personas de Colón	12,5 ms	2,68 ms	B-Tree sobre ciudad (hash: 3,38 ms)
c.4-c.5	Personas de 30 a 34 años	12,0 ms	3,66 ms	B-Tree sobre edad (el hash no sirve para rangos)
c.6	Agregados por ciudad	28,3 ms	23,7 ms	Índice (ciudad, edad) con random_page_cost 1.1
d.2-d.5	Top 10 de facturación 2024	359 ms	165 ms	(cliente_id, fecha) INCLUDE (total) y ranking antes del join

Conclusiones

- **Medir bien antes de concluir.** Una sola ejecución de `EXPLAIN ANALYZE` puede llevar a la conclusión opuesta, como pasó en el a.4 del borrador, donde la primera lectura del índice desde disco hacía parecer más lento al plan más rápido. Comparar medianas de varias ejecuciones con la caché en el mismo estado evita ese error.
- **B-Tree o hash.** Para igualdades los dos rinden parecido (a.2, c.2 y c.3), pero el B-Tree además resuelve rangos, prefijos, ordenamientos y agrupamientos, permite Index Only Scan, admite índices compuestos y, con la deduplicación, puede ocupar mucho menos (6,9 MB contra 47 MB en el c.3). El hash sólo tiene sentido para igualdades sobre columnas con muchos valores distintos.
- **La selectividad y el orden físico deciden.** Un índice rinde cuando la condición es selectiva y las filas que busca están juntas. Con filas dispersas, un 3 % ya obliga a leer casi todas las páginas (b.1, c.2); un 17 % hace que el índice sea peor que el recorrido secuencial (d.4). `CLUSTER` puede agrupar físicamente las filas, a costa de bloquear la tabla y de que el orden no se mantenga.
- **Índices cubrientes.** Las mejoras más grandes con índices se obtuvieron cuando el índice tenía todas las columnas que la consulta necesita y la respondía sin ir a la tabla (b.2, b.4, b.7, d.5), en varios casos agregando columnas con `INCLUDE`. En los índices compuestos importa el orden de las columnas: la primera tiene que ser la de la condición o la del orden que se quiere aprovechar.

- **Reescribir suele ganar más que indexar.** Cambiar una subconsulta correlacionada por un CTE (a.12.1, unas 400 veces), un join con DISTINCT por EXISTS (b.4), un MAX por un ORDER BY con LIMIT (b.7) o calcular el ranking antes del join (d.5) mejoró más, o lo mismo, que los índices, y sin costo de mantenimiento.
- **El planificador estima, no sabe.** Eligió planes peores que otros disponibles en el a.9, el c.6 y el d.4. En los dos primeros la causa es `random_page_cost = 4`, pensado para discos rotativos; con datos en memoria o en SSD, el valor 1.1 hace que valore mejor los índices. En el a.9 también pesa que no modela el costo de comparar texto con una collation lingüística.
- **Para consultas que resumen toda la tabla, precalcular.** Promedios y máximos sobre 8 millones de filas cuestan medio segundo con cualquier índice; una vista materializada los deja en milisegundos, a cambio de tener que refrescarla. Aumentar `work_mem` para la sesión evita que los hash y los sort pasen por disco y es una mejora intermedia barata (b.6, b.8).